Universidade Federal do Piauí
Centro de Educação Aberta e a Distância

PROJETO E ANÁLISE DE ALGORITMOS

Francisco José de Araújo
José Messias Alves da Silva



Ministério da Educação - MEC
Universidade Aberta do Brasil - UAB
Universidade Federal do Piauí - UFPI
Universidade Aberta do Piauí - UAPI
Centro de Educação Aberta e a Distância - CEAD

Projeto e Análise de Algoritmos

Francisco José de Araújo
José Messias Alves da Silva

UFPI
2013

PRESIDENTE DA REPÚBLICA	*Dilma Vana Rousseff Linhares*
MINISTRO DA EDUCAÇÃO	*Aloisio Mercadante*
GOVERNADOR DO ESTADO	*Wilson Nunes Martins*
REITOR DA UNIVERSIDADE FEDERAL DO PIAUÍ	*José Arimatéia Dantas Lopes*
SECRETÁRIO DE EDUCAÇÃO A DISTÂNCIA DO MEC	*Carlos Eduardo Bielshowsky*
PRESIDENTE DA CAPES	*Jorge Almeida Guimarães*
COORDENADOR GERAL DA UNIVERSIDADE ABERTA DO BRASIL	*João Carlos Teatini de S. Clímaco*
DIRETOR DO CENTRO DE EDUCAÇÃO ABERTA E A DISTÂNCIA DA UFPI	*Gildásio Guedes Fernandes*

ADMINISTRAÇÃO	*Antonella Maria das Chagas Sousa*
ADMINISTRAÇÃO PÚBLICA	*Fabiana Rodrigues de Almeida Castro*
CIÊNCIAS BIOLÓGICAS	*Maria da Conceição Prado de Oliveira*
FILOSOFIA	*Zoraida Maria Lopes Feitosa*
FÍSICA	*Miguel Arcanjo Costa*
LETRAS PORTUGUÊS	*José Vanderlei Carneiro*
LETRAS INGLÊS	*Lívia Fernanda Nery da Silva*
MATEMÁTICA	*João Benício de Melo Neto*
PEDAGOGIA	*Vera Lúcia Costa Oliveira*
QUÍMICA	*Milton Batista da Silva*
SISTEMAS DE INFORMAÇÃO	*Leonardo Ramon Nunes de Sousa*

EQUIPE DE DESENVOLVIMENTO

TÉCNICA EM ASSUNTOS EDUCACIONAIS	*Zilda Vieira Chaves*
EDIÇÃO	*Roberto Denes Quaresma Rêgo*
PROJETO GRÁFICO	*Samuel Falcão Silva*
DIAGRAMAÇÃO	*Antonio F. de Carvalho Filho*
REVISÃO ORTOGRÁFICA	*Djane Lemos Ferreira Gabriel*
REVISÃO GRÁFICA	*Gesiel dos Santos Sobrinho*

CONSELHO EDITORIAL DA EDUFPI

Prof. Dr. Ricardo Alaggio Ribeiro (Presidente)
Des. Tomaz Gomes Campelo
Prof. Dr. José Renato de Araújo Sousa
Profª. Drª. Teresinha de Jesus Mesquita Queiroz
Profª. Francisca Maria Soares Mendes
Profª. Iracildes Maria de Moura Fé Lima
Prof. Dr. João Renór Ferreira de Carvalho

A663p Araújo, Francisco José de.
Projetos e análise de algoritimos / Francisco José de Araújo.
– Teresina : EDUFPI/CEAD, 2013.
128p.

1. Algoritimos. 2. Algoritimos - Análise. 3. Educação a Distância. I. Título.

CDD 005.1

© 2013. Universidade Federal do Piauí - UFPI. Todos os direitos reservados.

A responsabilidade pelo texto e imagens desta obra é do autor. O conteúdo desta obra foi licenciado, temporária e gratuitamente, para utilização no âmbito do Sistema Universidade Aberta do Brasil, através da UFPI. O leitor se compromete a utilizar o conteúdo desta obra para aprendizado pessoal, sendo que a reprodução e distribuição ficarão limitadas ao âmbito interno dos cursos. A citação desta obra em trabalhos acadêmicos e/ou profissionais poderá ser feita, com indicação da fonte. A cópia desta obra sem autorização expressa, ou com intuito de lucro, constitui crime contra a propriedade intelectual, com sanções previstas no Código Penal.
É proibida a venda deste material.

Apresentação

Ao desenvolver um sistema computacional, não podemos deixar de levar em consideração todos os aspectos que influem positiva ou negativamente na sua execução. Projetar bem um sistema antes de sua implementação pode reduzir drasticamente o tempo de sua conclusão, além de utilizar mais eficientemente todos os recursos computacionais que se tem à disposição.

O objetivo desta apostila é proporcionar ao leitor um entendimento no que se refere ao desenvolvimento de bons algoritmos e sua análise. O texto foi escrito de forma simples e objetiva. Cada capítulo é acompanhado de embasamento teórico e prático dos fundamentos de análise, bem como exemplos de boas implementações e exercícios. A bibliografia e a *web* bibliografia ao fim das notas são suficientes para que o leitor se aprofunde na teoria apresentada em cada unidade.

Na Unidade I são apresentados os conceitos relacionados aos fundamentos de análise de algoritmos, os quais serão descritos suas definições e principais situações relacionadas aos problemas de análise.

Na Unidade II é descrita a forma como analisar as principais estruturas contidas em algoritmos, de maneira que possa determinar uma ordem de grandeza do seu desempenho.

Na Unidade III são apresentadas as principais estratégias para elaboração de algoritmos com bom desempenho, conforme a natureza dos problemas tomados.

Por fim, na Unidade IV é apresentada uma classificação dos principais problemas computacionais em estudo e as suas ordens de complexidade, enfocando a natureza de sua resolução.

Sumário

11 **UNIDADE 1**
FUNDAMENTOS DE ANÁLISE DE ALGORITMOS

Fundamentos de algoritmos 11
Conceitos básicos 18
Recorrências 32

35 **UNIDADE 2**
TÉCNICAS DE ANÁLISE DE ALGORITMOS

Análise de algoritmos 47
Complexidade de algoritmos 48

35 **UNIDADE 3**
TÉCNICAS DE PROJETO DE ALGORITMOS

Introdução 63
Força bruta 63
Dividir- e-conquistar 64
Programação dinâmica 70
Algoritmos gulosos 76

35

UNIDADE 4
CLASSES DE PROBLEMAS

Introdução 103
Solucionabilidade de problemas 103
Formas de problemas 105
Problemas de decisão classe p 106
Classe np 108
Classe co-np 109
Classe np-completo 110
Algumas reduções 112
A classe np-difícil 113
Relações entre classes de problemas 113
Backtracking e branch-and-bound 114

UNIDADE 1

Fundamentos de Análise de Algoritmo

Resumindo

Esta unidade é dedicada aos conceitos iniciais relacionados à análise de algoritmos, noções de função de complexidade e suas variações, eficiências e avaliação empírica de algoritmos e às variáveis envolvidas nesse processo.

1 FUNDAMENTOS DE ANÁLISE DE ALGORITMO

FUNDAMENTOS DE ALGORITMOS

> "Ao verificar que um dado programa está muito lento, uma pessoa prática pede uma máquina mais rápida ao seu chefe, mas o ganho potencial que uma máquina mais rápida pode proporcionar é tipicamente limitado por um fator de 10 por razões técnicas ou econômicas. Para obter um ganho maior, é preciso buscar melhores algoritmos. Um bom algoritmo, mesmo rodando em uma máquina lenta,sempre acaba derrotando (para instâncias grandes do problema) um algoritmo pior rodando em uma máquina rápida. Sempre."
>
> - S. S. Skiena, The Algorithm Design Manual

Introdução

Neste capítulo, apresentaremos alguns fundamentos de algoritmos e algumas ideias iniciais sobre função de complexidade, eficiência de algoritmos, etapas para o desenvolvimento de algoritmos eficientes, medida de complexidade e análise empírica.

Algoritmo

O que é um Algoritmo?

Definições:

Segundo o dicionário de Aurélio, algoritmo sob o ponto de vista da matemática é "processo de cálculo ou de resolução de um grupo de problemas

semelhantes em que se manipulam, com generalidade e sem restrições, regras formais para a obtenção do resultado ou da solução do problema".

Um algoritmo é uma sequência de instruções não ambíguas para resolver um problema, isto é, para obter uma saída desejada para qualquer entrada legítima em um intervalo de tempo finito.

Um algoritmo é qualquer procedimento computacional que recebe como entrada um valor ou um conjunto de valores e produz como saída um valor ou um conjunto de valores.

Podemos concluir que um algoritmo é uma sequência de passos computacionais que transformam a entrada em saída.

Exemplo: Considere a seguinte função:

$F(x)= x^3/5$

A sua entrada é o x e a sua saída e o y ou f(x), o valor que a função retorna. O algoritmo seria o seguinte:

1. Entrada: receber o valor de x.
2. Elevar x ao cúbico e guardar o número resultante como z.
3. Dividir z por 5 e guardar o número resultante como y.
4. Saída: imprimir o valor de y.

O algoritmo, portanto, é a lógica do nosso problema matemático ou informático. É a sequência de passos que eu faço na minha cabeça (ou no papel) antes de escrever para uma das linguagens. Se formos pensar, veremos que tudo o que fazemos é um algoritmo, é um procedimento que recebe uma entrada e envia uma saída, não só no computador, mas na vida.

Exemplo: Encontrar o maior e o menor valor de um vetor com n valores. Formalmente, o problema poderia ser colocado da seguinte forma:

Entrada: uma sequência de n números $< a_1, a_2, a_3,...,a_n>$

Saída: os valores Min e Max, o menor e o maior valor, respectivamente, dentre os valores da entrada.

Podem existir vários algoritmos diferentes para resolver o mesmo problema. Nos casos acima, poderíamos ter um algoritmo que fizesse a mesma coisa de maneira diferente.

Os algoritmos descritos neste trabalho serão escritos em uma linguagem de pseudocódigo, por está mais próximo da linguagem natural.

Por que estudar algoritmos?

Devemos conhecer um conjunto de algoritmos de diferentes áreas, além de sermos capazes de projetar novos algoritmos e analisar suas eficiências. O estudo de algoritmos é, reconhecidamente, a pedra

fundamental da ciência da computação. Algoritmo é muito mais que um ramo da ciência da computação. É o núcleo da ciência da computação e com toda a imparcialidade, pode ser considerado relevante para a maioria das ciências, negócios e tecnologia. Programas de computadores não existiriam sem algoritmos.

Instância

Instância de um problema consiste de todas as entradas necessárias para se calcular uma solução para o problema. Uma instância de um problema computacional é um possível valor para a entrada.

Alguns exemplos de problemas e suas instâncias:

Os números 25, -30 e 10 definem uma instância do problema da equação do segundo grau. A instância consiste em encontrar um número inteiro x tal que $25x^2 -30x +10=0$.

< 42, 6, 11,17, 4> é uma instância para o problema da ordenação.

Um algoritmo é dito correto se, para cada instância de entrada, ele para com a saída correta.

ATENÇÃO!!
Nem todos os problemas podem ser resolvidos por algoritmos. Exemplo. Como se tornar rico e famoso?

Que tipos de Problemas são resolvidos com algoritmos?

A ordenação não é o único problema computacional para o qual foram desenvolvidos algoritmos.

Algumas aplicações práticas de algoritmos. Segundo Cormen (2002):

1. O Projeto Genoma Humano: tem como objetivo identificar todos os 100.000 genes do DNA humano, determinar as sequencias dos 3 bilhões de pares de bases químicas que constituem o DNA humano, armazenar essas informações em bancos de dados e desenvolver ferramentas para análise de dados. Cada uma dessas etapas exige Algoritmos sofisticados.
2. A Internet: permite que pessoas de todo mundo acessem e obtenham com rapidez grandes quantidades de informações. Para isso, são empregados Algoritmos inteligentes com a finalidade de gerenciar e manipular esse grande volume de dados.
3. O Comércio Eletrônico: permite que mercadorias e serviços sejam negociados e trocados eletronicamente. Possui a capacidade de manter privadas as informações, como números de cartões de crédito, senhas e extratos bancários. As tecnologias usadas são a criptografia e as

assinaturas digitais e são baseadas em Algoritmos numéricos e teoria dos números.

4. Na indústria: alocar recursos escassos de maneira mais eficiente. Localizar poços, designar as tripulações para os vôos, instalar um provedor de serviços da *Internet*, etc. Esses exemplos podem ser resolvidos com o uso da programação linear.

Alguns exemplos de problemas concretos:

1. Mapa rodoviário no qual a distância entre cada par de pontos é marcado, tendo como objetivo determinar a menor rota de um ponto a outro do número de rotas;
2. Determinação do produto de n matrizes $A_1, A_2, \ldots, A_n$. Como a multiplicação de matrizes é associativa, existem várias ordens de multiplicação;
3. Temos n pontos no plano e desejamos encontrar a envoltória convexa desses pontos, ou seja, é um polígono convexo que contém os pontos.

Essas listas estão longe de esgotar os exemplos, mas exibem duas características comuns a muitos algoritmos interessantes:

1. Existem muitas soluções candidatas, porém a maioria não é aquilo que desejamos. Encontrar a solução que queremos pode representar um desafio.
2. Existem aplicações práticas. Dos problemas anteriores, o caminho mais curto fornece os exemplos mais fáceis. Uma empresa de transportes que utiliza caminhões tem interesse financeiro em encontrar os caminhos mais curtos em uma rede ferroviária ou rodoviária porque menores caminhos resultam em menor trabalho e menor consumo de combustível.

Complexidade e custo de um algoritmo

Determinando o menor custo possível para resolver problemas de uma dada classe, temos a medida de dificuldade inerente para resolver o problema. Quando o custo de um algoritmo é igual ao menor custo possível, o algoritmo é ótimo para a medida de custo considerada. Podem existir vários algoritmos para resolver o mesmo problema.

Medida do custo para execução do programa

Em muitas situações podem existir vários algoritmos para resolver o mesmo problema, sendo necessário escolher o melhor. Se a mesma medida de custo é aplicada a diferentes algoritmos, então é possível compará-los e escolher o mais adequado para resolver o problema em questão.

O custo de utilização de um algoritmo pode ser medido de várias maneiras. Uma delas é mediante a execução do programa em um computador, sendo o tempo de execução medido diretamente. As medidas de tempo obtidas são bastante inadequadas e os resultados jamais devem ser generalizados: os resultados são dependentes do compilador que podem favorecer algumas construções em detrimento de outras; os resultados dependem de *hardware*; quando grandes quantidades de memória são utilizadas, as medidas de tempo dependem deste aspecto.

Uma maneira mais adequada de medir o custo de utilização de um algoritmo é por meio do uso de um modelo matemático baseado em um computador idealizado. Devendo especificar o conjunto de operações e seus custos de execuções, é mais usual ignorar o custo de algumas das operações e considerar apenas as operações mais significativas.

Função de Complexidade

Para medir o custo de execução de um algoritmo, é comum definir uma função de custo ou função de complexidade f. A função f(n) é a medida do tempo necessário para executar um algoritmo para um problema de tamanho n.

Existem dois tipos de funções de complexidade a saber: a função de complexidade de tempo, onde f(n) mede o tempo necessário para executar um algoritmo em um problema de tamanho n e a função de complexidade de espaço, onde f(n) mede a memória necessária para executar um algoritmo em um problema de tamanho n.

Utilizaremos f para denotar uma função de complexidade de tempo daqui para frente. A complexidade de tempo na realidade não representa tempo diretamente, mas o número de vezes que determinada operação considerada relevante é executada.

Função de Complexidade de Espaço f(n): mede a memória necessária para executar um algoritmo em um problema de tamanho n.

Complexidade de um algoritmo é o tempo requerido para a execução deste algoritmo.

Eficiência de Algoritmos

Algoritmos criados para resolver o mesmo problema muitas vezes diferem de forma drástica em sua eficiência. Essas diferenças podem ser muito mais significativas que as diferenças relativas a hardware e software.

Dado um problema a ser resolvido, é interessante procurar diversos algoritmos que o faça escolher o melhor deles. Mas como decidir quais dos algoritmos é melhor?

Exemplo: Vamos comparar um computador mais rápido (computador A) que execute a ordenação por inserção com um computador mais lento (computador B) que execute a ordenação por intercalação. Cada um deles deve ordenar um milhão de números.

Suponha que o computador A execute um bilhão de instruções por segundo e o computador B execute apenas dez milhões de instruções por segundo. Assim, o computador A será 100 vezes mais rápido que o computador B.

Suponha que o programador mais esperto codifique a ordenação por inserção em linguagem de máquina para o Computador A e que o código resultante exija $2n^2$ instruções para ordenar n números. Por outro lado, a ordenação por intercalação é programada para o computador B por um programador médio que utiliza uma linguagem de alto nível com um compilador ineficiente, com o código resultante de 50nlogn instruções. Para ordenar um milhão de números, o

Computador A demora:

$$\frac{2 \cdot \left(10^6\right)^2}{10^9 \, instr / seg} = 2000 seg$$

Computador B demora:

$$\frac{50 \cdot 10^6 \cdot log\, 10^6}{10^7 \, instr / seg} = 100 seg$$

Usando o algoritmo cujo tempo de execução é mais lento, até mesmo com um compilador fraco, o Computador B funciona 20 vezes mais rápido que o computador A. Portanto, o algoritmo de ordenação por intercalação gasta

menos tempo computacional, ou seja, é mais eficiente do que o algoritmo de ordenação por inserção e esta vantagem é ainda maior à proporção que n cresce.

Metodologia para Desenvolver Algoritmos Eficientes.

Os passos necessários para procurar elaborar algoritmos que sejam eficientes são: Análise do Problema, Concepção do algoritmo, Análise de eficiência, Verificação e refinamento.

Passo 1: Análise do Problema

A análise do problema é uma etapa importante, pois permite uma compreensão do que se pretende e facilita o compartilhamento com outros pesquisadores.

Passo 2: Concepção do Algoritmo

A concepção é uma etapa criativa. Nesta fase, podem ser aplicadas as principais técnicas de projeto de algoritmos, as quais serão estudadas posteriormente.

Passo 3: Análise de Eficiência

Por outro lado, o algoritmo pode estar correto, mas não ser eficiente. A busca por algoritmos eficientes é de suma importância, pois uma pequena alteração no algoritmo poderá trazer grande melhoria no desempenho do mesmo.

Passo 4: Verificação e Refinamento

A verificação é uma etapa importante para garantir que o algoritmo trabalhe corretamente e faça o que deve fazer. O refinamento consiste em introduzir alterações no algoritmo com vistas a torná-lo correto e melhorar sua eficiência em tempo de execução e/ou espaço de memória utilizada.

Um algoritmo resolve um problema quando, para qualquer entrada, produz uma resposta correta quando concedidos tempo e memória suficientes para sua execução.

Um algoritmo que resolve um problema (teoricamente) não significa

ser aceitável na prática. Os recursos de espaço e tempo têm grande importância em casos práticos.

Às vezes, o algoritmo mais direto está longe de ser razoável em termos de eficiência. Um exemplo é o caso da solução de sistemas de equações lineares. O método de Cramer, resolvendo o sistema através de sua definição, requer dezenas de milhões de anos para resolver um sistema 20x20. O mesmo sistema pode ser resolvido em tempo razoável pelo método de Gauss, como mostra a Tabela 1.1.

Tabela 1.1 TAMANHO DO PROBLEMA X TEMPO DE EXECUÇÃO

N	Método de Crames	Método de Gauss
2	20µs	50µs
3	102µs	159µs
4	456µs	353µs
5	2,35ms	**666µs**

O avanço tecnológico concebe máquinas cada vez mais rápidas e que passam a resolver problemas maiores, pois a complexidade do algoritmo é que determina o novo tamanho máximo do problema resolvível.

Uma base sólida de conhecimento e técnicas de algoritmos é uma característica que separa os programadores qualificados dos não qualificados. Com a moderna tecnologia computacional, você pode executar alguns trabalhos sem saber muito sobre algoritmos, porém, com uma boa base em algoritmos, é possível fazer muito mais.

CONCEITOS BÁSICOS

> "A arte de programar consiste em organizar e dominar a complexidade"
>
> - Edsger W. Dijkstra

Introdução

A análise de algoritmos tem como objetivo melhorar, se possível, seu

desempenho e escolher, entre os algoritmos disponíveis, o melhor. Existem vários critérios de avaliação de um algoritmo como: quantidade de trabalho requerido, quantidade de espaço requerido, simplicidade, exatidão de resposta e otimização (TOSCANI, 2001).

As medidas de complexidade introduzem as ideias de complexidade de pessimista (pior caso), bem como as medidas de complexidade de tempo e espaço.

Comportamento Assintótico de Funções

O parâmetro n fornece uma medida da dificuldade para se resolver o problema. Para valores suficientemente pequenos de n, qualquer algoritmo custa pouco para ser executado, mesmo os ineficientes. A escolha do algoritmo não é um problema crítico para problemas de tamanho pequeno. Logo, a análise de algoritmos é realizada para valores grandes de n. O comportamento assintótico de função representa o limite do comportamento do custo quando n cresce. Para saber o comportamento de um algoritmo ou problema em relação ao tamanho da entrada, o que é relevante?

Exemplo: Suponha dois algoritmos A e B cujos tempos de execução sejam TA(n)=3n+10 e TB(n)=½ n +1. A Figura 1.1 mostra a representação gráfica,

Qual o maior deles? A Tabela 1.2 mostra onde o algoritmo A é maior do algoritmo B.

Tabela 1.2

n	$T_A(n)$	$T_B(n)$
0	10	1
2	16	3
4	22	9
6	28	19
8	34	33
9	37	41,5

Para n < 9, $T_A(n) > T_B(n)$, ou seja, o algoritmo A é maior do que B para todo n< 9.

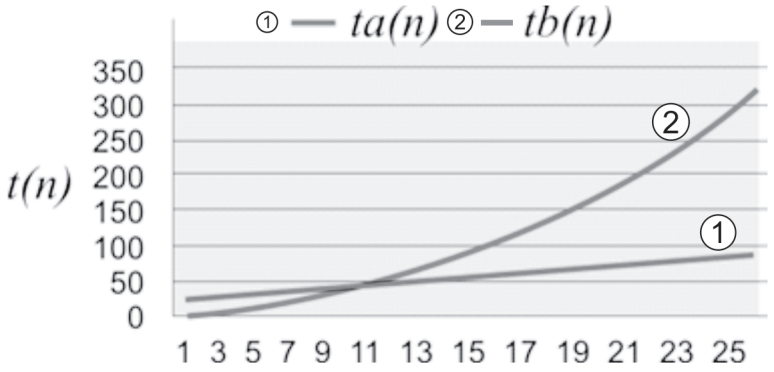


Figura 1.1 $T_A(n) > T_B(n)$

Exemplo: Considere a existência de dois algoritmos A e B para a solução de um problema. Se a complexidade de um é expressa por $T_A(n)=^{n2}$ e a do outro por $T_B(n)=100n$. Isso significa que, em função do tamanho da entrada de dados n, o algoritmo A cresce como uma parábola e o B cresce linearmente. Desta forma, se os algoritmos forem usados para um conjunto de 30 dados, teremos: $T_B(30)=3000$ e $T_A(30)=900$, neste caso, $T_A<T_B$. No entanto, se n=30000, teremos: $T_A(30000)=900.000.000$ e $T_B(30000)=3000.000$, ou seja TA>TB.

Exemplo: Suponha $T_C(n)$ =45n+15 e $T_D(n)$=0,1n2+0,5. Qual delas é maior?

Ordens Assintóticas

A análise de um algoritmo geralmente conta com apenas algumas operações elementares. A medida de custo ou medida de complexidade relata o crescimento assintótico da operação considerada.

A complexidade assintótica é definida pelo crescimento da complexidade para entradas suficientemente grandes. O comportamento assintótico de um algoritmo é o mais procurado, pois para um volume grande de dados, a complexidade torna-se mais importante. Algoritmo assintoticamente mais eficiente é melhor para as entradas, exceto para entradas relativamente pequenas.

Consideremos as funções f e g mapeando naturais em reais não

negativos: de N em R_+ .

Uma cota assintótica superior (CAS) é uma função que cresce mais rapidamente que outra e a partir de certo ponto está acima. Por exemplo, uma função cúbica n^3 cresce mais rapidamente do que uma quadrática n2. Dizemos que a cúbica n^3 é CAS para n^2. Do mesmo modo, uma função exponencial 2n é CAS para n^2.

Definição:

Em geral, define-se que g é uma cota assintótica superior para f, se e somente se $(\exists n_0 \in N)(\forall n \geq n_0)\ f(n) \leq g(n)$

Para n suficientemente grande, g(n) domina f(n).

Exemplo: O gráfico da Figura 1.2 mostra esta notação O:

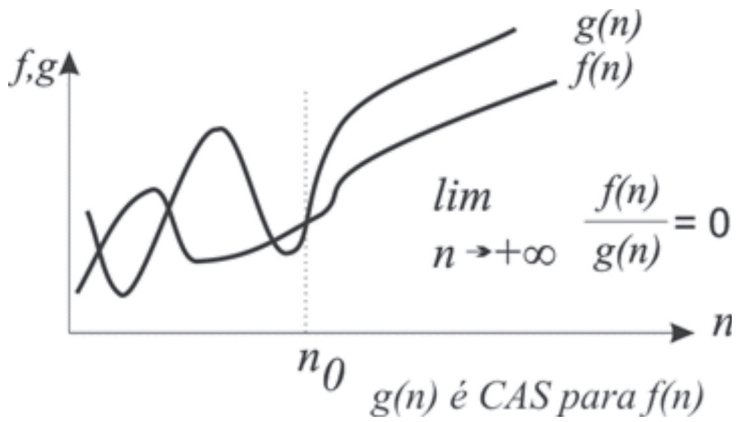


Figura 1.2:

Exemplo: Mostre que a função exponencial 2^n é CAS para n^2.

Exercício: Para cada um dos seguintes pares de funções f e g, verifique se é possível encontrar uma constante $n_0 \in N$ tal que:

$$(\forall n \geq n0)\ f\ (n) \leq g\ (n)$$

a) n, $n\log_2 n$

b) 2^n, 3^{n+1}

Notação O

A notação O define uma cota assintótica superior.

Seja N o conjunto dos números naturais e R o conjunto dos números reais. O conjunto N* denota o conjunto dos números naturais estritamente positivos, R_+* representa o conjunto dos números reais estritamente positivos

e R_+ o conjunto dos reais não negativos.

Seja f: N *→ R+ uma função arbitrária.

Definição:

Dadas duas funções assintoticamente não negativas f e g, dizemos que f está na ordem de g. Escrevemos f=O(g), se f(n) ≤ c.g(n) para algum c positivo e para todo n suficientemente grande.

Em outras palavras, existe um número positivo c e um número natural n_o, tais que f(n) ≤ c.g(n) para todo n maior que n_o.

Alternativamente, f(n) ∈ O(g(n)) se $\lim\left(\frac{f(n)}{g(n)}\right)$ é constante (mas não infinito).

Exemplo: Seja $f(n)=13n^3+2n^2+5n\log n$ e $g(n)=n^3$, então:

$$\lim\left(\frac{f(n)}{g(n)}\right)=\lim\left(\frac{13n^3+2n^2+5n\log n}{n^3}\right)=\lim\left(13+\frac{2}{n}+\frac{5\log n}{n^2}\right)=13$$

Simbolicamente:

$O(g(n)) = \{f : N \to R_+ \mid (\exists c \in R_+^*)(\exists n_0 \in N)(\forall n \geq n_0)[f(n) \leq c.g(n)]\}$

Normalmente diz-se que "f(n) é O(g(n))" ou f(n) ∈ O(g(n)).

Exemplo gráfico da Figura 1.3 de dominação assintótica que ilustra a notação O.

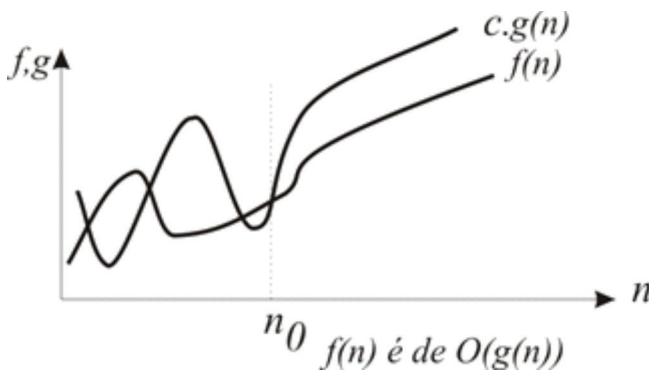


Figura 1.3.

O valor constante n0 mostrado é o menor valor possível, mas qualquer valor maior é válido.

Exemplos de Notação O

A notação O é usada para estabelecer limites superiores de complexidade.

Exemplo: Verifique, se $g(n)=(n+1)^2$, então:

$g(n)$ é $O(n^2)$ ou $g(n)=O(n^2)$, ou seja, $(\exists c \in R^*_+)((\exists n_0 \in N)(\forall n \geq n0)\ g(n) \leq cf(n)$

$f(n)=n^2$

$(n+1)^2 \leq c.n^2$

$n^2+2n+1 \leq c.n^2 \rightarrow c \geq 1 + 2/n + 1/n^2$

Logo, $n_0=1$ e $c=4$

Isto porque $(n+1)^2 \leq 4n^2$ para $n \geq 1$.

Exemplo: $g(n)=3n^3 + 2n^2 + n$ é $O(n^3)$

Basta mostrar que $3n^3 + 2n^2 + n \leq 6n^3$, para $n \geq 1$

A função $g(n) = 3n^3 + 2n^2 + n$ é também $O(n^4)$, entretanto, esta afirmação é mais fraca do que dizer que $g(n)$ é $O(n^3)$.

Exemplo: $g(n)=\log_5 n$ é $O(\log n)$.

O $\log_b n$ difere do $\log_c n$ por uma contante no caso é $\log_b c$.

Como $n=c^{\log_c n}$, tomando o logaritmo base b em ambos os lados da igualdade, temos que $\log_b n=\log_b c^{\log_c n} = \log_c n \times \log_b c$

Exemplo: Suponha que $f(n)=2n^2 + 3n +4$ e que $g(n)=n^2$. Observe que $2n^2 =3n + 4 \leq 2n^2 + 3n^2 + 4n^2 = 9n^2$ desde que $n \geq 1$. Resumindo, $f(n) \leq 9g(n)$ para todo $n \geq 1$. Portanto, $f(n)=O(g(n))$.

Exemplo: Suponha que $f(n)= 3 + 2/n$ e que $g(n)= n^0$, ou seja, $g(n)=1$. Então, $3 + 2/n \leq 3 + 1 =4 = 4n0$ desde que $n \geq 2$. Resumindo, $f(n) \leq 4g(n)$ para $n \geq 2$. Portanto, $f(n)=O(gn))$.

Exemplo: Suponha que $f(n)=n3$ e que $g(n)=500n2$. Não é verdade que $f(n)=O(g(n))$. De fato, se existissem c e n0 tais que $f(n) \leq cg(n)$, teríamos $n \leq 500c$ para todo $n \geq n0$. Mas isso é absurdo!

Exemplo: $7n – 2$ é $O(n)$

Prova: Pela definição da notação O, precisou achar uma constante $c>0$ e uma constante inteira $n_0 >1$, tais que $7n – 2 \leq cn$ para todo inteiro $n \geq n_0$. É fácil perceber que uma escolha poderia ser $c=7$ e $n_0=1$. De fato, esta é uma das infinitas escolhas possíveis porque qualquer número real maior ou igual a 7 será uma escolha possível para c e qualquer inteiro maior ou igual a 1 é uma escolha possível para n_0.

Algumas regras se aplicam na avaliação O(.)

Regra das Somas

Proposição 1

Se determinado algoritmo P se divide em partes independentes, digamos P1 e P2 e sendo $T_1(n)$ e $T_2(n)$ respectivamente de ordem O(f(n)) e O(g(n)) então: $T(n)=T_1(n)+T_2(n)$ e P será de ordem O(máximo{f(n), g(n)}) (CAMPELLO & MACULAN, 1994).

Demonstração:

Para c_1,c_2,n_1 e n_2 constantes

$T_1(n) \leq c_1.f(n), \ \forall \ n \geq n_1$

$T_2(n) \leq c_2.g(n), \ \forall \ n \geq n_2$

Sendo n_o= máximo{n_1,n_2} e com n ≥ no.

Então $T_1(n)+T_2(n) \leq c_1.f(n)+c_2.g(n) \leq (c_1+c_2)$máximo{f(n),g(n)}

Portanto, T(n) é de ordem O(máximo{f(n),g(n)}.

Exemplo: Calcular o tempo de execução de uma sequência de trechos de programas usando a regra da soma para O(f(n))+O(g(n)).

Suponha três trechos cujos tempos de execução são O(n), $O(n^2)$ e O(n.logn).

O tempo de execução dos dois primeiros é $O(max(n,n^2))$ que é $O(n^2)$.

Exemplo: Considere o caso em que as funções f(.) e g(.) são dadas a seguir:

$$f(n)=\begin{cases} n^4, \text{se } n \text{ é par} \\ n^2, \text{se } n \text{ é ímpar} \end{cases} \qquad g(n)=\begin{cases} n^2, \text{se } n \text{ é par} \\ n^3, \text{se } n \text{ é ímpar} \end{cases}$$

Neste caso:

$$\text{máximo}\{f(n), g(n)\}=\begin{cases} n^4, \text{se } n \text{ é par} \\ n^3, \text{se } n \text{ é ímpar} \end{cases}$$

O tempo de execução de todos os três trechos é $O(max(n^2,n.logn))$ que é $O(n^2)$.

Regra dos Produtos

Proposição 2

Se $T_1(n)$ e $T_2(n)$ são de ordem O(f(n)) e O(g(n)) respectivamente, então:

$T(n) = T_1(n) . T_2(n)$ é de ordem O(f(n).g(n)).

Demonstração

Por hipótese ∃ constantes c1 , c2, n1, n2 tais que:

$T_1(n) = O(f(n)) \rightarrow T_1(n) \leq c1.f(n), \quad \forall n \geq n1$

$T_2(n) = O(g(n)) \Rightarrow T_2(n) \leq c2.g(n), \quad \forall n \geq n2$

Fazendo no = máximo{n1,n2} e c = c1.c2

$T(n)= T_1(n).T_2(n) \leq c(f(n).g(n)), n \geq no$

E, portanto, T(n) é de ordem O(f(n).g(n))

Segue-se que O(k.f(n)) é o mesmo que O(f(n)) quando k é uma constante positiva.

Outras propriedades:

f(n)=O(f(n))

k. O(f(n))= O(f(n)) k = constate

O(f(n)) + O(f(n)) = O(f(n))

O(O(f(n))) = O(f(n))

f(n)O(g(n)) = O(f(n)g(n))

Teorema:

Se $T(n)=t_m n^m+t_{m-1}n^{m-1}+...+ t_1 n+t_o$ é um polinômio de grau m então $T(n)=O(n^m)$.

Demonstração:

Usando a definição:

$T(n) = O(n^m) \Rightarrow (\exists c \in R_+) T(n) \leq c.n^2, \quad \forall n \geq no$

$|T(n)| \leq |t_m|n^m+ |t_{m-1}|n^{m-1}+...+ |t_1|n+|t_o|$

$|T(n)| \leq (|t_m|+ |t_{m-1}|/n+...+ |t_o|/n^m)n^m$

$|T(n)| = (|t_m|+...+ |t_o|)n^m, \forall n \geq 1$

Substituindo $c=|t_m|+...+ |t_o|$ e $n_o=1$

Temos $|T(n)| \leq c|n^m| \Rightarrow T(n) \leq c.n^m \Rightarrow T(n) = O(n^m)$

Exemplo: Seja $T(n)= 2x^5+45x^4 + 100008x^2 -8888888$ um polinômio de grau 5, logo $T(n)=O(n^5)$, ou seja, despreza todos os termos de grau menor do que 5 e a constante.

Uma ferramenta poderosa e versátil para provar que alguma função é de ordem de outra é a "regra do limite", dadas as funções arbitrárias $f,g:N\rightarrow R_+^*$.

1. Se $\lim(f(n)/g(n)) \in R_+^*$ então $f(n) \in O(g(n))$ e $g(n) \in O(f(n))$
2. Se $\lim(f(n)/g(n)) = 0$ então $f(n) \in O(g(n))$ mas $g(n) \notin O(f(n))$
3. Se $\lim(f(n)/g(n)) = + \in$ então $f(n) \notin O(g(n))$ e $g(n) \in O(f(n))$

Exemplo: Sejam $f(n) = \log n$ e $g(n) = \sqrt{n}$

Deseja-se determinar a ordem relativa das duas funções.

Desde que f(n) e g(n) tendem ao infinito quando n tende ao infinito, deve-se usar regra de l'Hôpital para calcular este limite.

Resolução:

Provar que este limite existe.

$$\lim_{n \to \infty} f(n)/g(n) = \lim_{n \to \infty} \tilde{f}(n)/\tilde{g}(n)$$

$$\lim\left(\log n / \sqrt{n}\right) = \lim(1/n)/\frac{1}{2\sqrt{n}}) = \lim\left(2/\sqrt{n}\right) = 0$$

Agora, a regra do limite nos mostra que $\log n \in O(\sqrt{n})$ e $\sqrt{n} \notin O(\log n)$.

Em outras palavras, a função $\sqrt{n}$ cresce assintoticamente mais rápido que log n.

1. Notação Ω (Ômega)

A notação O nos dá um limite superior para o tempo que algum algoritmo gastará para qualquer instância de tamanho n. Para estimar um limite inferior, podemos usar a seguinte notação: Ω(f(n)).

Exemplo: $f(n)=7n^3+5$ cresce menos rapidamente do que uma exponencial $g(n)=2^n$.

Diz-se que a exponencial g(n) é Ωf(n)).

Definição:

Diz-se que g(n) é Ω(f(n)), se e somente se, para alguma constante c ∈ R^*_+ e n_o ∈ N tal que g(n) ≥ c.f(n)

Isto é: Ω(f(n)) = {g: N→R_+ |(∃ c ∈ R^*_+)(∃ n_o ∈ N) (∀ n ≥ n_o)[g(n) ≥ c.f(n)]}

Em outra palavras, Ω(f(n)) é um conjunto de todas as funções g(n) limitadas inferiormente por múltiplo real positivo de f(n), para n suficientemente grande.

Exemplo: Para mostrar que g(n)= $3n^3+2n^2$ é Ω(n^3), basta fazer c=1, e então $3n^3+2n^2 \geq n^3$ para n ≥ 0.

Exemplo: Seja g(n)=n para n ímpar (n ≥ 1) e g(n) = $n^2/10$ para n par (n ≥ 0). Neste caso, g(n) é Ω(n^2), bastando considerar c=1/10 e n=0,2,4,...

Exemplo: A Figura 1.4 mostra o gráfico para a notação Ω.

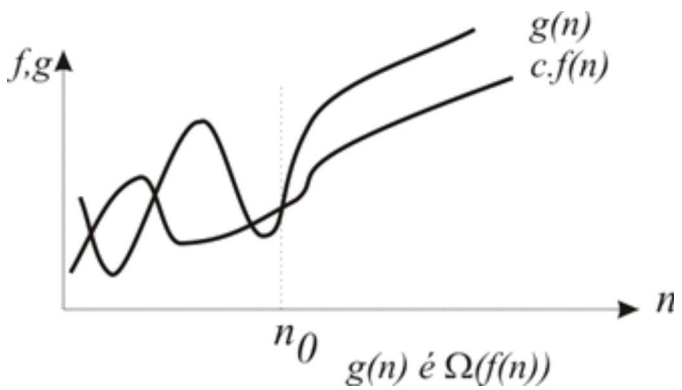


Figura 1.4

Exemplo: Seja $t(n)=n^3-2n^2+4n$, podemos afirmar que t(n) é Ω(n^3), pois $n^3-2n^2+4n \geq 0.5n3$ para n>1.

Exemplo: Se $f(n)=n^2-1$, então são válidas as igualdades f(n)=Ω(n^2), f(n)=Ω(n) e f(n)=Ω(1), mas não vale f(n)=Ω(n^3).

Exercício: Para as funções exponencial $f(n)=2^n$ e cúbica $g(n)=7n^3+5$, verifique que f(n) é Ω(g(n)), determinando as constantes c e no.

1. Notação θ (Theta)

A notação θ define um limite assintótico exato. Por exemplo, as funções quadrática $f(n)=5n^2 + 4$ e $g(n)=n^2 + 1$ crescem com a mesma rapidez.

Diz-se que f(n) é θ(f(n)), ou seja, θ(f(n)) = O(f(n)) ∩ θ(f(n)), se e somente se, θg(n)) = {f: N⟶R_+|(∃ c, d ∈R^*_+)(∃ n_o ∈ N) (∀n ≥ no)[c.g(n) ≤

f(n) ≤ d.g(n)]}

Podemos afirmar que duas funções f(n) e g(n), f(n)= θ(g(n)), se e somente se, f(n)=O(g(n)) e f(n)= Ω(g(n)).

Na prática, normalmente aplicamos a notação Ω para descrever um limite inferior para o melhor caso e a notação O para descrever um limite superior para o pior caso. A Figura 1.5 abaixo ilustra a notação θ

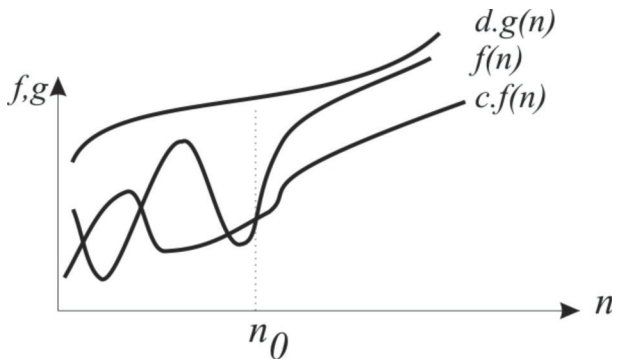


Figura 1.5 f(n) é θ(g(n))

Exemplo: Seja g(n)=n²/3-2n. Vamos mostrar que g(n) = θ(n²).

Temos de obter constantes c, d e n_0 tais que c.n² ≤ (1/3)n² - 2n ≤ d.n² para todo n ≥ n_0.

Dividindo por n², temos c ≤ 1/3 - 2/n ≤ d.

O lado direito da desigualdade será sempre válido para qualquer valor de n ≥ 1 quando escolhemos d ≥1/3. Da mesma maneira, escolhendo c ≤ 1/21, o lado esquerdo da desigualdade será válido para qualquer valor de n ≥ 7. Logo, escolhendo c = 1/21, d = 1/3 e n_0 = 7, verifica-se que n²/3 - 2n= θ(n²).

Outras constantes podem existir, mas o importante é que existe alguma escolha para as três constantes.

A regra do limite para a notação θ é reformulada da seguinte forma:

1. Se lim(f(n)/g(n)) ∈ R_+^* então f(n) ∈ θ (g(n))
2. Se lim(f(n)/g(n)) = 0 então f(n) ∈ O(g(n)), mas f(n) ∉ θ (g(n))
3. Se lim(f(n)/g(n)) = + ∞ então f(n) ∈ Ω(g(n)), mas f(n) ∉ θ(g(n))

Comparação de Funções

Algumas das propriedades relacionadas a números reais também se aplicam a comparação de funções assintóticas. Nas propriedades seguintes, suponha que f(n) e g(n) sejam assintoticamente positivas.

As notações apresentadas respeitam as seguintes propriedades:

Reflexividade:

1. $f(n)= \theta(f(n))$
2. $f(n)= O(f(n))$
3. $f(n)= \Omega(f(n))$

Simetria:

1. $f(n)=O(g(n))$ se e somente se $g(n)=O(f(n))$

Transitividade:

2. $f(n) = \theta(g(n))$ e $g(n) = \theta(h(n))$ implicam $f(n) = \theta(h(n))$
3. $f(n) = O(g(n))$ e $g(n) = O(h(n))$ implicam $f(n) = O(h(n))$
4. $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ implicam $f(n) = \Omega(h(n))$

Comportamento Assintótico

Se f é uma função de complexidade para um algoritmo A, então O(f) é considerada a complexidade assintótica ou o comportamento assintótico do algoritmo A. A relação de dominação assintótica permite comparar funções de complexidade. Entretanto, se as funções f e g dominam assintoticamente uma a outra, então os algoritmos associados são equivalentes. Nestes casos, o comportamento assintótico não serve para comparar algoritmos.

Desafio:
Dê um exemplo de função positiva f(n) de tal forma que f(n) não seja nem O(n) nem Ω(n).

Exemplo: Dois algoritmos C e D aplicados à mesma classe de problemas, sendo que C leva três vezes o tempo de D ao ser executado, isto é, $f(n) = 3g(n)$, sendo que $O(f(n)) = O(g(n))$. Logo, o comportamento assintótico não serve para comparar os algoritmos C e D porque eles diferem apenas por uma constante.

Podemos avaliar programas, comparando as funções de complexidade. Um programa com tempo de execução O(n) é melhor que outro com tempo O (n^2). Porém, as constantes de proporcionalidade podem alterar esta consideração.

Exemplo: Um programa leva 100n unidades de tempo para ser executado e outro leva $2n^2$. Qual dos dois é o melhor?

A resposta a essa pergunta depende do tamanho do problema a ser executado. Para $n<50$, o programa com tempo $2n^2$ é melhor que 100n. Para problemas com entrada de dados pequena, é preferível usar o programa cujo tempo de execução é $O(n^2)$. Entretanto, quando n cresce, o programa com tempo $O(n^2)$ leva muito mais tempo que o programa O(n).

Classes de Comportamentos Assintóticos

As principais classes de problemas possuem as funções de complexidade descritas a seguir. Segundo Zivianni (2007),

1. f(n)=O(1)

1. Algoritmos de complexidade O(1) são ditos de complexidade constante. O uso do algoritmo independe do tamanho de n. As instruções do algoritmo são executadas um número fixo de vezes.

2. f(n) = O(log n)

1. Um algoritmo de complexidade O(log n) é dito ter complexidade logarítmica. Esse tipo de execução ocorre em algoritmos que resolvem um problema transformando-o em problemas pequenos.

1. f(n) = O(n)

1. Um algoritmo de complexidade O(n) é dito ter complexidade linear.

1. f(n) = O(nlog n)

1. Típico em algoritmos que quebram um problema em outros menores resolve cada um deles independentemente e depois unindo as soluções.

2. f(n) = $O(n^2)$

1. Um algoritmo de complexidade $O(n^2)$ é dito ter complexidade quadrática, os quais ocorrem quando os itens de dados são processados aos pares, sendo muitas vezes em um ninho dentro do outro. São úteis para resolver problemas de tamanhos pequenos.

3. f(n) = $O(n^3)$

1. Um algoritmo de complexidade $O(n^3)$ é dito ter complexidade cúbica. Úteis para resolver pequenos problemas.

4. f(n) = $O(2^n)$

1. Um algoritmo de complexidade $O(2^n)$ é dito ter complexidade exponencial. Não são úteis do ponto de vista prático.

5. f(n) = O(n!)

1. Um algoritmo de complexidade O(n!) é dito ter complexidade exponencial também, apesar de a complexidade fatorial O(n!) ter

comportamento muito pior que $O(2^n)$.

Segue a ordem de complexidade dos algoritmos.

2. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$

Um Algoritmo cuja complexidade é $O(c^n)$, $c>1$ é chamado de algoritmo exponencial no tempo de execução. O algoritmo cuja função de complexidade é um polinômio, isto é, $O(p(n))$ é $p(n)$ e é chamado de algoritmo polinomial em tempo de execução. A diferença entre esses algoritmos cresce quando o tamanho do problema a ser resolvido aumenta, conforme ilustra a Tabela 1.3.

TABELA 1.3 COMPARAÇÃO DE VÁRIAS FUNÇÕES

***log* n**	**n**	**n *log* n**	**n^2**	**n^3**	**2^n**
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65536
5	32	160	1024	3276	4294967296

Um problema é considerado intratável quando ele é tão difícil que não existe um algoritmo polinomial para resolvê-lo, enquanto um problema é considerado tratável quando existe um algoritmo polinomial para resolvê-lo.

Exemplo: um algoritmo com função de complexidade $f(n)=2^n$ é mais rápido que um algoritmo $g(n)=n^5$ para valores de n menores ou iguais a 20. Também existem algoritmos exponenciais que são muito úteis na prática.

Exemplo: O algoritmo Simplex para programação linear possui complexidade de tempo exponencial para o pior caso, mas executa muito rápido na prática.

Exemplo: Um caixeiro viajante deseja visitar n cidades de tal forma que sua viagem inicie e termine em uma mesma cidade, sendo que cada cidade deve ser visitada uma única vez. Supondo que sempre exista uma estrada entre duas cidades quaisquer, o problema é encontrar a menor rota para a viagem.

A Figura 1.4 abaixo ilustra o exemplo para quatro cidades c1, c2, c3 e c4 em que os números nos arcos indicam a distância entre as cidades. O percurso <c1, c3, c4, c2, c1> é uma solução para o problema, cujo percurso total em distância é 24.

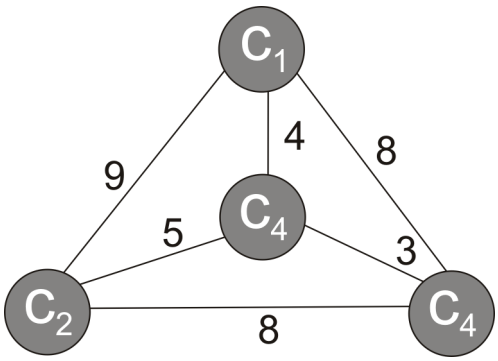


Figura 1.6 Problema do caixeiro viajante

Saiba Mais
O uso da notação O iniciou várias discussões na comunidade de análise de algoritmos e teoria da computação, como por exemplo, a de que a igualdade f(n) = g(n) é de "mão única", ou seja, apenas no sentido esquerdo para direita, mesmo adotando-se o fato de que a notação O defina um conjunto de funções.

RECORRÊNCIAS

Introdução

Quando um algoritmo contém uma chamada recursiva, o seu tempo de execução pode ser descrito por uma recorrência. Uma recorrência é uma equação ou uma inequação que descreve uma função em termos de seu valor em entrada menor. Para exemplificar, vamos apresentar a equação de recorrência do *Mergesort* (Intercalação).

T(n)=

Cuja solução é T(n)=O(nlogn).

Apresentaremos a seguir três métodos para resolver recorrência, isto é, para obter limites assintóticos para a solução.

Algoritmos Definidos por Recorrência

1. Quando se deseja especificar um algoritmo para a solução de um determinado problema, podemos utilizar duas abordagens:

1. Definir um algoritmo iterativo.
2. Definir um algoritmo recursivo.

Algoritmos Iterativos:

1. Algoritmo do fatorial

```
1  FAT ← 1
2 para i de 2 até n, faça:
3 FAT ← FAT * i
4 retorne FAT
```

1. Algoritmo de Fibonacci

```
1 Fib(1) ← Fib(2) ← 1
2 para i de 3 até n faça
3 Fib(1) ← Fib(i - 2) + Fib(i – 1)
```

Algoritmos Recursivos

1. Na construção de um algoritmo recursivo devemos ter o cuidado de sempre especificarmos primeiro a condição básica para depois estabelecermos o passo recorrente. Estes dois elementos deverão estar isolados por intermédio de uma cláusula condicional do tipo:

```
Se <condição básica> então:
<ação da condição básica>
Senão,
<ação do passo recorrente>
```

Desafio
Apresente um algoritmo recursivo para calcular o produto de dois inteiros m e n usando apenas adição.

Algoritmos Recursivos:

Algoritmo Fatorial

```
função Fat(n):
        Se n = 0 ou n = 1, então:
                retorne (1)
        Senão,
                retorne(n * Fat(n - 1))
```

Algoritmo de Fibonacci

```
função Fib(n):
        Se n = 1 ou n = 2, então:
```

```
        retorne (1)
Senão,
        retorne (Fib(n - 2) + Fib(n - 1))
```

Recorrências

Uma recorrência é uma fórmula que define uma função sobre os números naturais. Uma relação de recorrência é um caminho para definir uma função por uma expressão envolvendo a mesma função.

Exemplo: A recorrência abaixo define uma função T no conjunto dos números naturais:

T(1) =1 e

T(n)=T(n-1) + 3n + 2 para n=2, 3, 4, ...

Eis os valores de T(n) para valores pequenos de n:

n	1	2	3	4	5	6
T(n)	1	9	20	34	51	71

Seja T(n) uma função de complexidade que representa o número de inspeções nos n elementos do conjunto. O termo T(n) é especificado em função dos termos anteriores T(1), T(2), ..., T(n - 1).

T(n) = T(n/3) + n, T(1) = 1 (para n=1 fazemos uma inspeção).

Por exemplo: T(3) = T(3/3) + 3 = 4; T(9) = T(9/3) + 9 = 12?, e assim por diante.

Seja T(n) uma função de complexidade que representa o número de inspeções nos n elementos do conjunto. O termo T(n) é especificado em função dos termos anteriores T(1), T(2),... , T(n - 1).

Solucionando Recorrência

Resolver uma relação de recorrência nem sempre é fácil. Resolvendo uma relação de recorrência, determina-se o tempo de execução do algoritmo recursivo correspondente.

Exemplo: A sequência T abaixo se encontra definida por recorrência:

1. Condição básica: T(1) = 2
2. Passo recorrente: T(n) = 2 • T(n - 1), para n ≥ 2

Solucionando Recorrência:

De acordo com a definição da sequência T, temos os seguintes termos:

$T(1) = 2$

$T(2) = 2T(1) = 2 \cdot 2 = 2^2$

$T(3) = 2T(2) = 2 \cdot 2^2 = 2^3$

$T(4) = 2T(3) = 2 \cdot 2^3 = 2^4$

$T(5) = 2T(4) = 2 \cdot 2^4 = 2^5$

Podemos concluir que $T(n) = 2n$. Esta equação é denominada de SOLUÇÃO EM FORMA FECHADA para a relação de recorrência sujeita a condição básica T(1). Denomina-se RESOLVER uma recorrência ao processo de se encontrar uma solução em forma fechada para a recorrência.

Exemplo: Através de indução matemática a conjectura $T(n) = 2n$, é verificada da seguinte forma:

Passo básico:

1. $T(1) = 21 = 2$ verdade;

2. Hipótese de Indução:

I. Suponha que $T(k) = 2k$ seja verdade;

3. Passo Indutivo:

Prova-se que $T(k+1) = 2k+1$

Pela definição temos $T(k+1) = 2T((k+1)-1) = 2T(k) = 2 \cdot 2k = 2k+1$

Logo, está provada nossa conjectura.

4. Passo Indutivo:

Prova-se que $T(k) = 3k^2/2 + 7k/2 - 4$

Temos $T(k) = T(k-1) + 3k + 2$ por definição

$T(k) = [3(k-1)^2/2 + 7(k-1)/2 - 4] + 3k + 2$

$T(k) = 3k^2/2 + 7k/2 - 8/2$

$T(k) = (3k^2 + 7k - 8)/2$

Logo, está provada nossa conjectura.

Como a fórmula está correta, prova-se que $T(n) = O(n^2)$.

Como determinar a complexidade de um algoritmo recursivo?

Exemplo: Algoritmo Mergesort

Pode ser escrito pela recorrência:

Técnicas de Recorrência

Apresentaremos a seguir três métodos para resolver recorrências, isto é, para obter limites assintóticos para a solução.

2. Método da Substituição;
3. Método da Árvore de Recursão (iteração);
4. Método Mestre.

Método da Substituição

Este método consiste em propor uma forma para a solução (por inspiração); determinar as constantes envolvidas por indução e mostrar que o limite estabelecido é válido. Substituição vem do fato de substituir a resposta inspirada pela hipótese indutiva aplicada a valores menores. É um método poderoso, mas depende da percepção de quem analisa. É eficiente, mas só pode ser aplicado em casos nos quais seja fácil pressupor a forma de resposta.

Exemplo: Determinar um limite superior para a recorrência $T(n) = 2T(n/2) + \theta(n)$

Supondo que o método da solução visualizada seja $T(n) = O(n \cdot \log n$, o método consiste em provar que $T(n) \leq c \cdot n \cdot \log n$ para uma constante $c > 0$ apropriada.

Assumiremos que este limite é verdadeiro para $\lfloor n/2 \rfloor$, isto é, $T(\lfloor n/2 \rfloor) \leq c \cdot \lfloor n/2 \rfloor \cdot \log(\lfloor n/2 \rfloor)$.

Substituindo na recorrência temos:

$T(n) \leq 2(c \cdot \lfloor n/2 \rfloor \cdot \log(\lfloor n/2 \rfloor) + n$

$\leq c \cdot n \cdot \log(n/2) + n$

$\leq c \cdot n \cdot \log n - c \cdot n \cdot \log 2 + n$

$\leq c \cdot n \cdot \log n - c \cdot n + n$

$\leq c \cdot n \cdot \log n$, para $c \geq 1$

O próximo passo consiste em provar que a nossa solução é válida para as condições de contorno do problema, ou seja, devemos mostrar que podemos escolher a constante c tão grande quanto possível que o limite $T(n) \leq c \cdot n \cdot \log n$ vale para as condições de contorno. Assumindo que $T(1) = 1$ como condição de contorno da recorrência, não conseguiremos encontrar uma

constante c, pois T(1) ≤ c • 1• log1 = 0. A saída para superar esta dificuldade consiste em usar um valor n ≥ n0 sendo n_0 uma constante qualquer maior do que 1. Assim podemos impor T(2)= 4 e T(3)= 5, constantes e usar a recorrência a partir de T(4).

T(2) ≤ c • 2 • log2

4 ≤ 2 • c

c ≥ 2, para todo n ≥ 2.

Como observamos, qualquer valor de c ≥ 2 é suficiente para que os casos básicos de n=2 e n=3 sejam válidos.

A grande dificuldade deste método é que não existe uma forma geral para solucionar recorrências corretamente. Pressupor solução exige experiência e criatividade na área, entretanto existem algumas heurísticas, árvore de recursão, que veremos adiante, que podem ajudar a gerar boas hipóteses.

Se uma recorrência for semelhante a uma que você já tenha visto antes, então será fácil supor uma solução semelhante.

Exemplo: Considere a recorrência.

T(n) = 2T(⌊ n/2 ⌋ +35) + n, que é semelhante à recorrência vista anteriormente, exceto pelo termo 35. Este termo não pode afetar substancialmente a solução da recorrência. Quando n é grande, a diferença entre T(⌊ n/2 ⌋) e T(⌊ n/2 ⌋ +35) não é grande. Concluímos que T(n)=O(nlogn), pode ser verificado usando o método da substituição.

Exemplo: Resolver a recorrência T(n) = 2T(⌊√n ⌋) + logn

Parece difícil, porém tentaremos resolvê-la mediante uma substituição de variáveis. Por conveniência vamos considerar que √n é inteira. Substituindo m = logn obtemos $T(2^m) = 2T(2^{m/2}) + m$. Substituindo a recorrência S(m) = $T(2^m)$, produzimos a recorrência S(m) = 2S(m/2) + m, cuja solução já é conhecida, ou seja, S(m) = O(m • logm) . Substituindo m obtemos T(n) = $T(2^m)$ = S(m) = O(m • logm)= O(logn.log(logn))

O Método de Árvore de Recursão (Iteração)

Embora o método de substituição possa fornecer uma prova de que uma solução para uma recorrência é correta, às vezes é difícil apresentar uma boa suposição. Traçar uma árvore de recursão é um caminho direto para se criar uma boa suposição. Este método consiste em desenhar uma árvore cujos nós representam os tamanhos dos subproblemas. Cada nível i

contém todos os subproblemas de profundidade i. Dois aspectos importantes devem ser considerados: A altura da árvore e o número de passos executados em cada nível. A solução de recorrência (tempo de execução do algoritmo) é a soma de todos os passos de todos os níveis. No caso particular em que o total de passos de cada nível é o mesmo, l(n), por exemplo, a solução é T(n)=h.l(n), onde h é a altura da árvore.

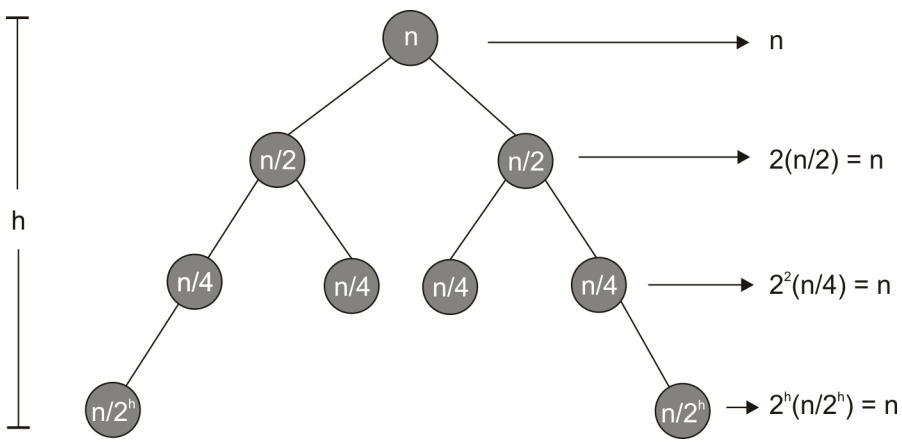
Uma árvore de recursão é usada para gerar uma boa suposição, o qual é verificado pelo método de substituição.

Talvez o método seja mais intuitivo.

Exemplo: Considere a equação de recorrência do Mergesort.

Veremos como uma árvore de recursão forneceria uma boa suposição para a recorrência T(n) = 2T(n/2) + n

1. Supomos que n é potência de 2.



$n/2^h = 1 \Rightarrow h = \log_2 n$ Total = h.n

A altura da árvore é h=logn Logo, O(n . log n)

Mesmo este método não exigindo inspiração, requer mais álgebra que o método da substituição. A ideia consiste em expandir (iterar) a recorrência e expressá-la como um somatório e dependendo apenas de n e das condições iniciais.

Exemplo: Considere a recorrência $T(n) = \begin{cases} 1 \\ 2T(n-1)+1 \end{cases}$

Solução usando a álgebra:

T(n) = 2(2T(n - 2) + 1) + 1

$T(n) = 4T(n - 2) + 2 + 1$

$T(n) = 4(2T(n - 3) + 1) + 2 + 1$

$T(n) = 2^3T(n - 3) + 4 + 2 + 1$

- - - - - - - - - - - - - - - - - - - -

$T(n) = 2^i\ T(n-i) + 2^{i-1} + 2^{i-2} +...+ 2^1 + 2^0$

O caso base é alcançado quando $i = n - 1$

Logo, $T(n) = 2^{n-1} + 2^{n-2} + 2^{n-3} +...+ 2^1 + 2^0$

$T(n) = 2^n - 1 = O(2^n)$

Método Mestre

Este método fornece uma receita de bolo para resolver recorrência da forma: $T(n) = aT(n/b) + f(n)$, onde $a \geq 1$ e $b > 1$ são constantes e $f(n)$ é uma função assintoticamente positiva. O método exige a memorização de três casos, mas a solução de muitas recorrências torna-se mais simples.

Utilizando o método mestre, a solução depende do Teorema Mestre.

Teorema Mestre

Sejam $a \geq 1$ e $b > 1$, constantes. Seja $f(n)$ uma função, e seja $T(n)$ definido no domínio dos inteiros não negativos pela recorrência $T(n) = aT(n/b) + f(n)$, onde n/b pode ser $\lceil n/b \rceil$ ou $\lceil n/b \rceil$. Então, $T(n)$ pode ser limitada assintoticamente por:

1- Se $f(n) = O(n^{\log_b a}-\mathcal{E})$ para uma constante $\mathcal{E} > 0$, então $T(n) = \theta(n^{\log_b a})$.

2- Se $f(n) = \theta(n^{\log_b a})$, então $T(n) = \theta(n^{\log_b a}\log n)$.

3- Se $f(n) = \Omega(n\log ba+\mathcal{E})$ para uma constante $\mathcal{E} > 0$ e se $af(n/b) \leq cf(n)$ para alguma constante $c < 1$ e n suficientemente grande, então $T(n) = \theta(f(n))$.

Para melhor compreender o significado deste teorema, observe que nos três casos estamos comparando a função $f(n)$ com a função $n^{\log_b a}$. A solução da recorrência é dada pela maior das duas funções. No caso 1, a função $n\log ba$ é maior, então a solução é $T(n) = \theta(n^{\log_b a})$. No caso 2, as funções são de mesma dimensão, sendo introduzido um fator $\log n$ e a solução é $T(n) = \theta(n^{\log_b a}\log n) = \theta(f(n)\log n)$. No caso 3, a função $f(n)$ é maior, então, a solução é $T(n) = \theta(f(n))$.

É importante ressaltar que o teorema não cobre todos os casos possíveis, apenas aqueles em que $f(n)$ é menor que $n\log ba$ por um fator polinomial, isto é, $f(n)$ deve ser assintoticamente menor ou maior que $n\log ba$

para os casos 1 e 3 do teorema Mestre. Para o caso 3, deve ser satisfeita a condição de regularidade onde af(n/b) ≤ cf(n).

Exemplo: Considere a seguinte equação de recorrência

1. $T(n) = 9T(n/3) + n$

Neste caso, a = 9, b = 3, f(n) = n $\Rightarrow n^{\log_b a} = n^{\log_3 9} = \theta(n^2)$

Como $f(n) = O(n^{\log_b a - \mathcal{E}})$, onde $\mathcal{E} = 1$, podemos aplicar o caso 1 do teorema mestre e concluir que a solução é $T(n) = \theta(n^2)$.

1. $T(n) = T(2n/3) + 1$

Neste caso, a = 1, b = 3/2, f(n) = 1 e nlogba = 1.

2. Aplica-se o caso 2, desde que $f(n) = \Theta(n^{\log_b a}) = \Theta(1)$ e a solução da recorrência é $T(n) = \Theta(\log n)$.

3. $T(n) = 3T(n/4) + n\log n$

Neste caso, a = 3, b = 4 e f(n) = nlogn, sendo $n^{\log_b a} = n^{\log_4 3} = O(n0{,}793)$. Como $f(n) = \Omega(n^{\log_4 2 + \varepsilon})$, onde ε = 0,2, aplica-se o caso 3, caso a condição de regularidade puder ser preservada.

Método Mestre: Exemplo

Assim, para n tendendo para o infinito, af(n/b) = 3(n/4)log(n/4) ≤ (3/4) nlogn = cf(n), para c = 3/4. Logo, pelo caso 3, a recorrência corresponde a $T(n) = \Theta(n\log n)$.

Exercício 1

1. O que é algoritmo?
2. Forneça um exemplo de aplicação que exija conteúdo algorítmico no nível de aplicação e discuta a função dos algoritmos envolvidos.
3. O que significa dizer que um algoritmo executa em tempo polinomial a n?
4. Comparação entre os tempos de execução

Para cada função f(n) e cada tempo t na tabela a seguir, determine o maior tamanho n de um problema que pode ser resolvido no tempo t, supondo-se que o algoritmo para resolver o problema demore f(n) microssegundos.

	1 seg	1 min	1 hora	1 dia	1 mês	1 ano
log n						
N						
n^2						
n^3						
2^n						

Exercício 2

1. Dois algoritmos gastam n^2 dias e n^3 segundos, respectivamente, para resolverem uma instância de tamanho n. Em alguma situação, o algoritmo quadrático é melhor do que o algoritmo cúbico? Justificar formalmente.
2. Sejam A e B dois algoritmos cujas complexidades são, respectivamente, determinadas pelas funções f(n) e g(n) dadas abaixo. Para cada caso, determine os valores inteiros positivos de n para os quais o algoritmo A leve menos tempo para executar do que o algoritmo B.
   1. $f(n)= 2n^2 -2$ e $g(n)= 3n +5$
   2. $f(n)= 3n^4 + 2n^2 + 1$ e $g(n)=4n^2 + 1$
3. Suponha que estamos comparando uma implementação do algoritmo de ordenação por inserção com uma implementação do *mergesort*. O primeiro consome $8n^2$ unidades de tempo quando aplicado a um vetor de comprimento n e o segundo consome 64nlogn. Para que valores de n o primeiro é mais rápido que o segundo?
4. Suponha dois algoritmos A e B com funções de complexidade de tempo $a(n)=n^2-n+549$ e $b(n)=49n+49$, respectivamente. Determine quais valores de n pertencentes ao conjunto dos números naturais para os quais A leva menos tempo para executar do que B.
5. Para os seguintes pares de funções, determine o menor valor para n, para o qual a segunda função torna-se menor do que a primeira:
   a) n^2, $10n$
   b) 2^n, $2n^3$
   c) $n^2/\log n$, $n(\log n)^2$
   d) $n^3/2$, $n^{2.81}$
6. Para cada um dos seguintes pares de funções f e g, verifique se é possível encontrar uma constante $n_0 \in N$ tal que $(\forall n \geq n_0)\ f(n) \leq g(n)$
   a) n, $n\log_2(n)$
   b) 2^n, 3^{n+1}

7. Quais das afirmações abaixo são verdadeiras? Justifique a sua resposta.
   a) $10n = O(n)$
   b) $10n2 = O(n)$
   c) $2n+1 = O(2n)$
   d) $22n = O(2n)$
   e) $n = O(n2)$
   f) $f(n) = O(v(n))$ e $g(n) = O(u(n)) \Rightarrow f(n) + g(n) = (v(n)+u(n))$
   g) $(3/2)n^2 + (7/2)n - 4 = O(n2)$
8. Qual das seguintes afirmações sobre crescimento assintótico de funções não é verdadeira?
   a) se $f(n)=O(g(n))$ e $g(n)=O(h(n))$, então $f(n)=O(h(n))$
   b) se $f(n) =O(g(n))$, então $g(n)=\Omega(f(n))$
   c) $6n^2 + 8n + 3=O(n^2)$
   d) se $f(n)=O(g(n))$, então $g(n)=Of(n))$
   e) $3n^3 + 30n=O(n^3)$
9. É verdade que $n^2 + 2000n +5466 = O(n^2)$? Justifique.
10. É verdade que $n^2 - 2000n +5466 = O(n)$? Justifique.
11. É verdade que $n^4 -99988889n^2 + 5000008= O(n^3)$? Justifique.
12. Para cada um dos seguintes pares de funções f e g, verifique se é possível encontrar constantes $n_0 \in N$ e $c \in R^+$, tais que $(\forall n \geq n_0)\ f(n) \leq cg(n)$:
    a) 2^5n, n^3 b) 10^nn^2, $n2^n$
13. Suponha que $f(n) = (3/2)\ n^2 + (7/2)n-4$ e que $g(n) = n^2$. Verifique que $f(n)$ é $O(g(n))$, determinando constantes $n_0 \in N$ e $c \in R^*_+$.
14. Suponha que $f(n) = 3n^3 + 2n^2$ e que $g(n) = n^3$. Verifique que $f(n)$ é $O(g(n))$, determinando constantes $n_0 \in N$ e $c \in R^*_+$.
15. Provar que a notação O é transitiva, isto é, se $f(n) \in O(g(n))$ e $g(n) \in O(h(n))$, então $f(n) \in O(h(n))$ para qualquer função $f: N \rightarrow R^*$.
16. Provar que se $f(n) \in O(n)$, então $[f(n)]^2 \in O(n^2)$.
17. Sejam duas funções não negativas $f(n)$ e $g(n)$. Diz-se que $f(n)=\Theta(g(n))$ se $f(n)=O(g(n))$ e $g(n)=O(f(n))$. Mostre que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

EXERCÍCIO III

1. Considere a equação de recorrência a seguir, definindo T(n):

$$T(n)=\begin{cases} 1, & n=1 \\ T(n-1)+n, & n>1 \end{cases}$$

Demonstre por indução que T(n)= n(n +1)/2.

2. Considere a equação de recorrência a seguir, definindo T(n):

$$T(n) = \begin{cases} 1, & n = 0 \\ 2T(n-1), & n \geq 1 \end{cases}$$

Demonstre por indução que $T(n)=2^n$

3. Considere a recorrência:

Mostre que $T(n) \leq 8n \log n$ para $n \geq 8$. Deduza daí que $T(n)=O(n \log n)$.

4. Usar o método da substituição para provar que a solução de T(n)= T(n/2) + 1 é O(logn).
5. Mostre que a solução de T(n)=2T(n/2) + n é O(n2).
6. Através do Método da Substituição, prove que:
    1. T(n)=T(n/2) + 1 é O(logn)
    2. $T(n)=2T(n/2) + n^3$ é $O(n^3)$
7. Em ambos os casos, considere T(1)=1.
    1. Mostrar que a solução para a recorrência

    T(n)= O(1) para n=0

    T(n)= T(n-1) + 1 para n>0
8. Através do Método Mestre, determinar limites assintóticos para as seguintes recorrências.
    a) T(n) = 4T(n/2) + n
    b) $T(n) = 4T(n/2) + n^2$
    c) $T(n) = 7T(n/8) + n^2$
9. Através do Método Mestre, prove que a ordem assintótica de
    1. $T(n)=16T(n/4) + n^2$ é $O(n^2 \log n)$
    2. T(n)=2T(n/2 +10) + n é O(nlogn)

UNIDADE 2

Técnica de Análise de Algoritmos

Resumindo

Esta unidade é reservada para a análise mais específica de algoritmos, em especial, aos algoritmos recursivos, bem como ao estudo dos métodos de resolução de recorrência e a determinação de suas complexidades.

2 TÉCNICA DE ANÁLISE DE ALGORITMOS

ANÁLISE DE ALGORITMOS

> "A análise de algoritmos é uma disciplina de engenharia. Um engenheiro civil, por exemplo, tem métodos e técnicas para prever o comportamento de uma estrutura antes de construi-la.
> Da mesma forma, um projetista de algoritmos deve ser capaz de prever o comportamento de um algoritmo antes de implementá-lo."
>
> — Anônimo

Introdução

A análise de algoritmos é um ramo da ciência da computação que estuda as técnicas de projeto de algoritmos e os algoritmos de forma abstrata, sem estarem implementados em uma linguagem de programação em particular ou implementadas de algum modo. Ela se preocupa com os recursos necessários para a execução do algoritmo, tais como o tempo de execução e o espaço de armazenamento de dados.

Neste capítulo, apresentaremos algumas técnicas muito úteis para analisar a complexidade do pior caso de um algoritmo, envolvendo as principais estruturas de controle normalmente encontradas, bem como mostraremos alguns exemplos de aplicações.

Analisar Algoritmos

Uma metodologia usada para realizar a complexidade de um

algoritmo está baseada na sua estrutura (TOSCANI, 2001). Ela detém a complexidade do algoritmo, passo a passo, através das complexidades de suas componentes.

Serão analisadas as complexidades de cada uma das principais estruturas algorítmicas a seguir, estabelecendo equações para elas.

3. Atribuição: v ← w,
4. Sequência: S; T,
5. Condicional: se b então S senão T ou (se b então S)
6. Iteração definida (incondicional)
para i de j até m faça S.
7. Iteração indefinida (condicional)
enquanto b faça S.

Complexidade de Algoritmos

1. O tempo de execução de um comando de atribuição, leitura ou de escrita pode ser considerado com O(1). Existem exceções para as linguagens que permitem a chamada de funções em comandos de atribuição ou quando envolvem vetores de tamanho arbitrariamente grande.
2. O tempo de execução de uma sequência de comandos é determinado pelo maior tempo de execução de qualquer comando da sequência.
3. O tempo de execução de um comando de decisão dos comandos executados dentro do comando condicional, mais o tempo para avaliar a condição, que é O(1).
4. O tempo para executar um laço é a soma do tempo de execução do corpo do laço mais o tempo de avaliar a condição para determinação, multiplicando pelo número de iterações do laço.
5. Quando o programa possui procedimentos não recursivos, o tempo de execução de cada procedimento deve ser computado separadamente um a um, iniciando com os procedimentos que não chamam outros procedimentos. A seguir, devem ser avaliados os procedimentos que chamam os procedimentos que não chamam outros procedimentos, usando os tempos dos procedimentos já avaliados. Este processo é repetido até se chegar ao programa principal.

6. Quando o programa possui procedimentos recursivos, para cada procedimento é associada uma função de complexidade f(n) e, em geral, a análise desta função envolve a solução de uma relação de recorrência.

Complexidades de Atribuições

A atribuição é uma estrutura sintaticamente simples, cuja semântica depende do tipo de dado atribuído. A atribuição pode ser uma operação simples, como a atribuição de um valor a uma variável, ou uma atribuição mais complexa, como a inserção de um nodo num grafo, a atualização de uma matriz, dentre outros.

Exemplo: Considere as atribuições a seguir:

7. Para as variáveis inteiras i e j:

i ← 0 {inicialização}

j ← i {transferência}

Ambas têm complexidade constante: O(1).

8. Para lista v de inteiros e variável inteira m:

m ← Max(v) {valor máximo}

Esta atribuição envolve (sendo n o comprimento da lista em v).

Determinar o máximo da lista v, com complexidade O(n);

Transferir este valor, com complexidade O(1).

Sua complexidade tem ordem linear: O(n) +O(1)=O(n),

9. Para listas u, v e w;

u←v {transfere lista}

w ←Reversa(v) {inverte lista}

A atribuição u←v transfere cada elemento da lista v com complexidade O(n) para uma lista v com comprimento n.

A atribuição w←Reversa(v) envolve (lista v com comprimento n);

Inverter a lista v com complexidade O(n).

Transferir os elementos da lista invertida com complexidade O(n).

Sua complexidade tem ordem O(n) +O(n) : O(n).

Complexidade de Sequências

Essa estrutura tem a forma: S; T. A complexidade da sequência é a soma das complexidades componentes.

Exemplo: Considere as sequências a seguir:

10. Para variáveis inteiras n e m:

n ← 0; m ← n

Sua complexidade tem ordem 1 + 1: O(1).

11. Para lista v de inteiros e variável inteira i:

i ← Max(v); i ← i + 1;

Sua complexidade tem ordem n + 1: O(n).

12. Para listas u, v e w: (as listas têm comprimento n)

u ← v; w ← Reversa(v). (Lista invertida)

Sua complexidade tem ordem n + n: O(n).

Exemplo: Considere o trecho de algoritmo

v ← Reversa(n);

w ← Ordene(v);

Onde Reversa e Ordene tem complexidade O(n) e $O(n^2)$.

O algoritmo tem complexidade de ordem $n+n^2$, isto é, $O(n^2)$.

Exemplo: Dados algoritmos Prim(u) e Buscab(a,v), considere a sequência:

v ← Prim(u); Buscab(a,v);

Vamos Supor que:

Prim(u) dá como saída a primeira metade da lista em u com comprimento $\lfloor n/2 \rfloor$ e complexidade O(n) (n é o comprimento da lista em u).

Buscab(a,v) procura a na lista v, com complexidade O(log m), para lista v com comprimento m.

O algoritmo composto tem complexidade de ordem $n+\log \lfloor n/2 \rfloor$, ou seja, O(n).

Complexidades Condicionais

A estrutura condicional pode apresentar-se de diversas formas, as mais usuais sendo as seguintes:

Se b então S senão T

e

Se b então S

Análise destas formas:

Vamos iniciar a análise como a forma mais simples

Estrutura Condicional: Se b então S

Exemplo: Considere as estruturas condicionais a seguir:

13. Para variável inteira i:

Se i = 0 então i ← i + 1;

Esta estrutura condicional envolve:

Determinar se o valor de i é 0, sua complexidade é O(1).

Caso afirmativo, executar a atribuição i ← i + 1 com complexidade O(1).

Sua complexidade total tem ordem constante O(1).

1. Para lista v de inteiros e variável inteira m;

Se m = 0 então m ← Max(v);

Esta atribuição envolve (n comprimento da lista):

Determinar se o valor de m é 0, com O(1).

Caso afirmativo, executar a atribuição m ← Max(v) com complexidade O(n).

Sua complexidade no pior caso tem ordem O(n).

1. Estrutura Condicional Se b então S senão T

Exemplo: Considere as estruturas condicionais a seguir:

2. Para variáveis inteiras i e j

Se $i \neq j$ então i ← i + j

senão j ← i + 1

Esta estrutura condicional envolve:

Determinar se os valores de i e j são diferentes com complexidade O(1). Caso afirmativo, executar a atribuição i ← i + j com complexidade O(1).

Caso negativo, executar a atribuição j ← i + 1 com complexidade O(1). Sua complexidade tem ordem constante O(1).

1. Para listas u e v (inteiros)

Se u = v, então v ← Prim(u)

senão u ← Ordene (v)

Esta estrutura condicional envolve:

Determinar se as listas u e v são iguais com complexidade O(n). Caso afirmativo, executar v ← Prim(u) com complexidade O(n) e (Prim(u) dá como saída a primeira metade da lista($\lfloor n/2 \rfloor$).

Caso negativo, executar u ← Ordene(v), usando um dos algoritmos de ordenação com complexidade $O(n^2)$.

Sua complexidade total, no pior caso é: $O(n) + O(n) + O(n^2) = O(n^2)$.

Complexidade de Iteração Definida

A estrutura a ser trabalhada é a iteração definida ou incondicional da forma:

para i de j até m, faça S.

Esta iteração executa S (m – j +1) vezes com valores de i variando de j até m. Considerando-se que os valores de j e m não são alterados na execução de S, o número de iterações é determinado por (m – j +1).

Exemplo: Considere as iterações definidas a seguir:

Para variável inteira m:

para i de 1 até 20, faça m ← m + 1

Sua complexidade tem ordem constante 20 . 1, isto é, O(20)=O(1)

Para vetor A[1 .. n] de inteiros:

para i de 1 até n faça A[i] ← 0

Sua complexidade é do tipo n . 1, isto é, O(n).

Para vetor A[1 .. n] de inteiros:

para i de 1 até n, faça A[i] ← A[i] + 1

Sua complexidade é n . 1: O(n).

Para variável real s e o vetor R[1..n] de reais:

para i de 1 até n, faça s ← s + R[i]

Sua complexidade é n . 1: O(n).

Para variável real t e matriz quadrada Q[1..n,1..n] de reais:

para i de 1 até n, faça t ← t + Q[i,i]

Sua complexidade é O(n).

1. Complexidade de Iteração Indefinida

Esses tipos de laços não são tão fáceis de analisar comparados com os laços “para”. O primeiro fator a ser observado é a regra de parada do laço. Se a regra de parada envolve uma variável inteira e uma constante, o laço pode ser analisado de maneira semelhante ao laço “para”. Quando o controle envolve duas variáveis inteiras, a análise padrão consiste em expressar o comportamento como uma variável única a ser decrementada.

Iterações indefinidas ou condicionais podem tomar várias formas, dentre elas “ENQUANTO” e “REPITA”.

Enquanto b faça S

Exemplo: Considere os trechos de algoritmos a seguir.

Para variável inteira i:

i ← 0

Enquanto i ≤ 10 faça:

i ← i + 1

Sua complexidade tem ordem constante 10.1, isto é, O(10).

Inicialização de vetor A [1..n] de inteiros:

i ← 0

Enquanto i ≤ n, faça:

i ← i + 1;

A[i] ← 0;

Sua complexidade tem ordem n . 1: O(n).

Atualização de vetor A [1..n] de inteiros:

i ← n

Enquanto i > 0 faça

A[i] ← A[i] + 1;

i ← i – 1;

Sua complexidade tem ordem O(n).

OBS: Esses tipos de laços não são fáceis de analisar como os laços "para".

Primeiro, o fator é a regra de parada, o qual envolve variável inteira e uma constante e pode ser analisado semelhante o "para".

Segundo, quando envolvem duas variáveis inteiras, a análise consiste em expressar o comportamento como uma variável única a ser decrementada

Analisar um algoritmo é prever o que o algoritmo irá precisar. Às vezes o *hardware* é importante, mas o que acontece com frequência é medir o tempo que irá demorar. O tempo de execução de um algoritmo depende geralmente do tamanho de sua entrada. Portanto, a análise de algoritmo está baseada no tamanho de sua entrada para compará-lo com outros algoritmos e ter noção de quanto tempo ele vai demorar.

Exemplo: Analisar o seguinte algoritmo:

1. para i ← 1 até n faça
2. para ← 1 até i faça
3. imprima i x j x n
4. fim para
5. fim para

Para medir o custo do algoritmo, nossa análise consistirá em ver quantas vezes cada passo é executado. Mediremos o custo de cada linha (cada passo a ser executado), sempre em função de n. Vamos analisar o algoritmo.

Linha 1 Será executada n+1 vezes

Linha 2 Será executado n x$\sum_{i=1}^{n}$ + n vezes

Linha 3 Será executada n x$\sum_{i=1}^{n}$

Linha 4 Não tem custo.

O laço da linha 1 voltará para si mesmo n vezes, isto é, testará novamente sua condicional e incrementará um. No final ele terá que testar novamente para dizer que já passou de n. Por isso, ele será executado n + 1 vez, ao invés de simplesmente n.

Desafio:
Qual o tempo de execução total para contar de 1 a n em binário, se o tempo necessário para somar 1 ao número i é proporcional ao número de bits na representação binária de i que devem ser alterados para representar i+1?

O laço da linha 2 será executado um número de vezes que a variável i, onde i irá de 1 a n. Portanto, ele será executado o número de vezes que equivale a soma de 1 a n mais n.

O laço da linha 3 será executado o mesmo número que a linha 2, mas sem n.

Exemplo : Faça a análise para o seguinte trecho de programa:

ler(n); $T_1(n)= O(1)$

para i de 1 até n faça

para j de 1 até n faça $T_2(n)=O(n^2)$

A[i,j] ← 0;

para i de 1 até n faça $T_3(n)=O(n)$

A[i,j] ← 1;

$T(n) = T_1(n) + T_2(n) + T_3(n)$

$T(n) = O(1) + O(n2) + O(n) = O(n^2)$

Exemplo: Faça a análise do trecho do programa:

Se A[i,i] = 0, então:

para i de 1 até n, faça:

para j de 1 até n , faça $f(n) = n^2$

A[i,j] ← 0;

Senão:

para i de 1 até n ,faça g(n) = n

A[i,i] ← 1;

$T(n) = f(n) + g(n) = O(max(f(n),g(n))$

$T(n) = O(max(n^2, n)) = O(n2)$

Exemplo: Fatorial

Ler(n);

i ← 2;

Fat ← 1;
enquanto i ≤ n, faça:
Fat ← Fat * i;
i ← i +1;
escreva Fat;
Exemplo: Verificar se um inteiro n é primo
Primo ← verdade
i ← 2
enquanto i * i ≤ n, faça:
Se n mod i = 0, então:
primo ← falsidade
goto 44
senão:
i ← i +1;
44 fim
Exemplo: Considere o algoritmo para ordenar n elementos de um conjunto A.
Para i de 1 até n-1, faça:
min ← i;
para j de i + 1 até n, faça:
Se A[j] < A[min], então:
min ← j;
temp ← A[min];
A[min] ← A[i];
A[i] ← temp;
Exemplo: Algoritmos para avaliação de polinômios de grau n da forma:
$P_n(x) = a_n x^n + a_{n-1}x^{n-1} + ... + a_1 x + a$
E são comparadas as suas complexidades.
Algoritmo 1
Ler (n, a_n,, a_{n-1}-,,..., a_0,,x}
P ← a_0
y ← x
para i de 1 até n-1, faça:
P ← P + ai * y;
y←y + x;
P ← P + an * y;
Cada execução de laço implica em dois produtos e uma soma, 3

operações de tempo constante. Como este é executado (n-1),

Portanto o algoritmo 1 tem complexidade;

$T(n) = (n-1)[O(1)+O(1) +O(1)] = O(n)$

Algoritmo 2

Ler (n, a_n,, a_{n+1}-,,..., a_1, a_0,x}

$P \leftarrow a_0$

para i de 1 até n, faça:

$P \leftarrow P + a_i * x^i$;

O laço é executado n vezes e a cada execução (2 + i) as operações são realizadas,

portanto,

Algoritmo 3

Ler (n, a_n,, a_{n-1}-,,..., a_1, a_0,x}

$P \leftarrow a_n$;

para i de n-1 até 0, faça:

$P \leftarrow ai + x * P$;

Desafio:
Mostre que qualquer algoritmo de ordenação baseado em comparações pode ser transformado em um método estável sem afetar o tempo de execução assintótico do algoritmo.

Neste caso, cada execução do laço realiza uma soma e um produto, ou seja, duas operações de tempo de execução. Assim $T(n) = n. O(1) = O(n)$

Este exemplo mostra claramente a importância de algoritmos eficientes.

Para valores grandes, digamos n = 10000, um número pequeno de operações seriam realizadas (108 ou 104)

Algoritmos de Menor Complexidade para Ordenação

Heapsort

$$O(n \bullet \log n)$$

Mergesort

É possível provar que este limitante é o melhor possível, isto é, estes algoritmos são ótimos.

Teorema: Dados um vetor A [1],..., A [n] e um valor x, verificar se x é elemento do vetor.

Qual é o melhor algoritmo? Depende! Dados ordenados ou não?

Dados ordenados:

Pesquisa binária com complexidade de O(log n).

Dados não ordenados:

Pesquisa sequencial com complexidade de O(n).

A complexidade total será a soma das complexidades da ordenação com:

$O(n \cdot \log n) + O(\log n) = O(n \cdot \log n)$

Pesquisa sequencial é melhor.

Variante: Há m elementos x a serem testados.

Dados não ordenados:

Pesquisa sequencial $O(m \cdot n)$

Ordenar + pesquisa binária

$O(n \cdot \log n) + O(m \cdot \log n) = O((n+m) \cdot \log n)$

Hipótese: $m \cong n$

Agora, a segunda alternativa é a mais eficiente.

$O(n \cdot \log n)$, $O(n^2)$

Exercício

1. Quanto tempo (em número de operações) é gasto para executar o seguinte algoritmo?

```
para i←1 até n-1, faça:
        Para k←1 até n, faça:
        S[i,k] ←S[i,k] – S[i,k]*S[i,i]/S[i,i]
```

2. Determine o tempo consumido na execução do algoritmo abaixo:

```
S←0
para i←1 até n, faça:
S←S + A[i]
m←S/n
k←1
para  i←2 até n, faça:
se (A[i] –m)/2 <(A[k] – m)/2, então:
            k←i

retorne k
```

```
Laço1(n)
s←0
para i←1 até n, faça:
s←s + 1
Laço2(n),
t←1
para t←1 até 2n, faça:
t←t*i
Laço3(n);
t←1
para t←1 até n2, faça:
t←t*i
Laço 4(n);
s←0
para i←1 até 2n, faça:
para j←1 até i, faça:
s←s +i
Laço5(n);
s←0
para i←1 até n2 , faça:
para j←1 até i, faça:
s←s +i
```

Figura 1.8 Algoritmo.

3. Forneça uma estimativa em notação O e em função de n para o método do laço1, dado no algoritmo da figura 1.8.
4. Forneça uma análise similar para o método laço 2 do algoritmo da figura 1.8.
5. Forneça uma análise similar para o método laço 3 do algoritmo da figura 1.8.
6. Forneça uma análise similar para o método laço 4 do algoritmo da figura 1.8.
7. Forneça uma análise similar para o método laço 5 do algoritmo da figura 1.8.
8. Faça a análise do algoritmo de ordenação por inserção

```
1       Ordenação por inserção (A, n)
2       Para j←2 até n, faça:
```

```
3       x←A[j]
4       i←j – 1
5       enquanto i > 0 e A[i] > x, faça:
6       A[i + 1] ←A[i]
7       i←i – 1
8       A[i + 1] ←x
```

UNIDADE 3

Técnica de Projeto de Algoritmos

Resumindo

Esta unidade é dedicada a demonstrar as principais técnicas para elaboração de algoritmos com bons desempenhos e, de acordo com a natureza do problema, propiciar ao leitor decidir qual estratégia utilizar diante de certas situações específicas de resolução. Exemplificaremos as técnicas com alguns dos principais algoritmos relacionados a cada estratégia.

3 TÉCNICA DE PROJETO DE ALGORITMOS

INTRODUÇÃO

> "Ter uma base sólida de conhecimento das técnicas de projeto de algoritmos é uma característica que separa os programadores verdadeiramente qualificados dos novatos. Com a moderna tecnologia da computação, você pode realizar algumas tarefas sem saber muito sobre algoritmos, mas com uma boa experiência em algoritmos, você pode fazer muito, muito mais."
>
> Cormen, Leiserson, Rivers, Stein, "Introduction to Algorithms"

Ao se projetar um bom algoritmo, deve-se levar em conta uma estratégia que possibilite obter uma boa complexidade, tanto temporal quanto espacial. Diante disso, técnicas de projeto de algoritmos são conjuntos de técnicas que compreendem tantos os métodos de codificação de algoritmos quanto a forma de determinar sua complexidade e que propiciam a produção de algoritmos fortes e levando em consideração a forma pela qual determinado algoritmo chega a solução desejada.

FORÇA BRUTA

É a mais simples estratégia de projeto de algoritmos. Solução direta para resolver um problema, usualmente baseada no próprio enunciado do problema e nas definições dos conceitos envolvidos. Consiste em enumerar todos os possíveis candidatos de uma solução e verificar se cada um satisfaz o problema. É considerada por muitos a técnica de projeto mais fácil de aplicar.

Apesar desta técnica, raramente, gerar algoritmos eficientes, é uma importante técnica para projeto de algoritmos, visto que é aplicável a uma ampla variedade de problemas, podendo ser empregada em diversas situações comuns, porém importantes. Por exemplo, encontrar o maior elemento de uma lista, somar n números, os divisores de um número natural n, etc.

Assim, é uma alternativa válida quando se deseja resolver um problema pequeno através de um algoritmo simples e direto. A força bruta é tipicamente usada em problemas cujo tamanho é limitado ou quando há uma heurística usada para reduzir o conjunto de candidatos para um espaço aceitável.

Para alguns problemas, a técnica de força bruta gera algoritmos razoáveis, práticos e sem limitações no tamanho da instância, como por exemplo, o problema da multiplicação de matrizes, casamento de padrões, etc.

O esforço para projetar algoritmos mais eficientes pode não compensar o trabalho, quando o tamanho das instâncias ou a quantidade de problemas a resolver é pequeno, ou seja, se obtém uma velocidade aceitável.

Também pode ser utilizado quando a simplicidade da implementação é mais importante que a velocidade de execução como nos casos de aplicações críticas em que os erros de algoritmo possuem sérias consequências.

DIVIDIR- E-CONQUISTAR

Introdução

Esta técnica consiste em decompor instâncias a serem resolvidas em um número de subinstâncias menores do mesmo problema, resolvendo sucessivamente e independentemente cada uma delas, combinando-as para obter a solução do problema original.

Apresentaremos, a seguir, um exemplo da aplicação desta técnica de projeto de algoritmos aplicada à ordenação.

Ordenação

Seja T[1..n] um vetor de n elementos. Deseja-se ordenar esses elementos em ordem crescente.

Usando a técnica dividir-e-conquistar, devemos dividir T em duas partes iguais ou de tamanhos semelhantes, ordenar cada uma delas recursivamente, e então fazer um "merge" das soluções respeitando a ordenação. O algoritmo a seguir, de nome "mergesort" verifica o tamanho da instância a ser ordenada, caso ela tenha tamanho inferior a c, uma constante determinada experimentalmente, o algoritmo ordena com o uso do algoritmo de ordenação por inserção; caso contrário, o vetor a ser ordenado é dividido em duas partes iguais e o procedimento "mergesort" é chamado recursivamente com cada uma das metades U e V do vetor original T. O procedimento "merge" é invocado quando se dispõe dos dois vetores ordenados U e V, gerando o vetor final ordenado pela mesclagem("merge") de U e V.

Algoritmo Mergesort

```
1 proc mergesort(T[1..n])
2  início
3  se n <= c então ordenar_por_inserção
4  senão
5  U[1..⌊ n/2 ⌋] ← T[1.. ⌊ n/2 ⌋]
6 V[1.. ⌊n/2⌋] ← T[1+⌊n/2⌋ ... n]
7  mergesort(U)
8  mergesort(V)
9  merge(U,V,T)
10 fim
1  proc merge(U[1..m+1],V[1..n+1],T[1..m+n]
2  {U[m+1] e V[n+1] são usados como sentinelas}
3 i,j←1
4  U[m+1],V[n+1] ← ∞
5  para k ← 1 até m+n, faça:
6 se U[i] < V[j] então: T[k] ← U[i]; i ← i+1
7 senão: T[k] ← V[j];j← j+1
```

Exemplo:

Vetor a ser ordenado

3	1	4	1	5	9	2	6	5	3	5	8	9

Vetor dividido em duas metades

3		4	1	5	9

2	6	5	3	5	8	9

Vetor após retorno das duas ordenações recursivas

1	1	3			9		2	3	5	5		8	9

Vetor após o merge

1		2	3	3	4	5	5	5	6		9

Correção:

Pelo teorema de "Jean Pierre basta olhar", esse algoritmo está correto, ou seja, dado um vetor T não ordenado, ao final do algoritmo, teremos um vetor T ordenado.

Mais formalmente, para instância de tamanho menor ou igual a c, as mesmas são ordenadas por um algoritmo já conhecido e correto.

Assumimos que o algoritmo ordena corretamente um vetor de tamanho <=n. Para ordenar um vetor de tamanho n+1, tal vetor será dividido em duas instâncias U,V de tamanho menor ou igual a n. O algoritmo *merge* recebe dois vetores U e V ordenados e gera um vetor T ordenado. Por contradição, assumindo que é possível haver um vetor T não ordenado, e supondo, sem perda de generalidade, que os elementos t_1, t_2, t_{m+n} na ordem de seleção do merge, que corresponde à saída.

Para o vetor não ser ordenado, deve haver algum elemento $t_i > t_j$, para i<j. Pelo algoritmo *merge* é impossível, pois a condição de seleção impõe que seja o menor dentre os dois menores dos vetores U e V.

Complexidade:

O algoritmo é recursivo. A parte de "merge" e inicialização podem ser feitas em O(n), onde n é o tamanho do vetor de determinada instância.

Sendo T(n) o número de passos elementares executados pelo algoritmo, todos concordam que T(n) = 2T(n/2) + n, abstraindo os detalhes técnicos.

Como obter a complexidade desta expressão?

Para responder a esta questão, faremos uso do ferramental apresentado na Unidade anterior, mais especificamente o Método Mestre.

Se T(n) = 2.T(n/2) + n, f(n)=n e $f(n)=\Theta(n^{\log_b a})$, pois a=b=2

Neste caso é aplicado o caso 2 do método mestre.

Então, $T(n) \in \Theta(n.\log n)$

O "*mergesort*" é superior ao algoritmo de ordenação por inserção que lhe serve de base e é considerado algoritmo ótimo juntamente com o "*heapsort*". No entanto, o "*mergesort*" gasta muita memória intermediária, enquanto o "*heapsort*" usa apenas o vetor original.

Outra questão interessante é relativa à divisão do vetor. Será que faz alguma diferença dividir ao meio ou dividir em outras proporções? Supondo que o algoritmo seja como abaixo:

```
1 proc mergesort2(T[1..n])
2 início
3 se n <= c, então: ordenar_por_inserção
4 senão:
5 U[1.. n-1] ← T[1..n-1]
6 V[1] ← T[n]
7 mergesort(U)
8 mergesort(V)
9 merge(U,V,T)
10 fim
```

Desafio:
Projete um algoritmo utilizando a técnica dividir-e-conquistar para encontrar o menor e o maior elemento entre n números usando não mais que 3n/2 comparações.

Qual a complexidade do algoritmo *mergesort*2?

$T(n) = T(n-1)+T(1)+n$, o que corresponde claramente a $\Theta(n^2)$.

Quicksort

O *Quicksort* tem também por objetivo ordenar um vetor. Para isto, o algoritmo escolhe um elemento como pivot, particionando o vetor em dois: elementos à direita e elementos à esquerda do pivot. Cada uma das partições é ordenada recursivamente pelo algoritmo *quicksort*.

Aritmética com Números Inteiros Grandes

Até agora, nós temos considerados que adição e multiplicação são operações elementares, isto é, o tempo necessário para executar estas instruções é limitado superiormente por uma constante que depende somente da velocidade do circuito do computador utilizado. Isto só é válido quando o tamanho do operando pode ser manuseado diretamente pelo *hardware*. Para algumas aplicações, devem-se utilizar inteiros muito grandes. A

representação destes números por ponteiro flutuante não é recomendável, a menos que se esteja interessado, exclusivamente, na ordem de magnitude e com os algarismos mais significativos do resultado. Se o mesmo tiver de ser calculado de forma exata com todos os algarismos, há a necessidade de implementar operações aritméticas no *software*.

Embora uma multiplicação elementar raramente leve mais tempo do que uma adição na maioria dos computadores, isto está longe de ser verdade quando os operandos envolvidos são muito grandes. Há algoritmos que somam dois inteiros em tempo linear, por outro lado, o algoritmo clássico e a multiplicação a "la russe" leva um tempo quadrático para multiplicar estes mesmos operandos. Como melhorar? Seja u e v dois inteiros com n dígitos decimais a serem multiplicados.

Utilizando a estratégia de Dividir-e-Conquistar, é possível separar cada um desses operandos em duas partes do mesmo tamanho, o mais próximo possível: $u = 10^s w + x$; $v = 10^s y + z$, onde $0 \leq x < 10^s$ e $s = \bullet n/2 \bullet$. Consequentemente, os inteiros w e y possuem $\bullet n/2 \bullet$ dígitos. Veja a figura abaixo.

(Por conveniência, diz que um inteiro tem j dígitos se for menor que 10j, mesmo se não for maior ou igual a 10j-1)

U	w	x
V	y	z
	$\bullet n/2 \bullet$	$\bullet n/2 \bullet$

O produto que nos interessa é:

$uv = 10^{2s} wy + 10^s (wz + xy) + xz$

Obtém-se o algoritmo a seguir:

```
1 função mult (u,v: inteiros grandes) : inteiros grandes
2 n ← o menor inteiro tal que u e v sejam de tamanho n
3 se n é pequeno, então multiplicar u por v utilizando o algoritmo clássico
4  retornar o produto computado
5 senão:
6  s ← n div 2
7  w ← u div 10s ; x ← u mod 10^s
```

8 $y \leftarrow v \text{ div } 10s ; z \leftarrow v \bmod 10^s$

9 retornar mult (w,y) × 102s + (mult(w,z) + mult(x,y)) × 10^s + mult (x,z)

Seja T(n) o tempo, no pior caso, necessário para o algoritmo multiplicar dois inteiros de n dígitos, há algoritmos conhecidos que executam em tempo linear divisões e multiplicações por 102s e 10s, bem como as adições. O mesmo ocorre para as operações de módulo, uma vez que estas são equivalentes a uma divisão, uma multiplicação e uma subtração de inteiros. A última parte do algoritmo consiste em quatro chamadas recursivas, cada uma delas serve para multiplicar dois inteiros cujo tamanho é aproximadamente n/2. Então, $T(n) \in 3T(\bullet n/2\bullet) + T(\bullet n/2\bullet) + \Theta(n)$.

Esta equação torna-se $T(n) \in 4T(n/2) + \Theta(n)$, quando n é uma potência de 2. Pode-se, facilmente, verificar que a complexidade deste algoritmo é O(n2).

O artifício que permite acelerar o processo consiste em executar os cálculos wy,wz + xy, e xz em menos de quatro meias multiplicações, mesmo que seja necessário ter de fazer mais adições. Isto é sensível, pois a adição é muito mais rápida do que a multiplicação (que é quadrática) quando os operandos são grandes. Considere o produto:

r = (w + x)(y+ z) = wy + (wz + xy)+ xz.

Depois de apenas uma multiplicação, incluindo os três termos necessários para calcular uv, duas outras multiplicações são necessárias para isolar esses termos. Isso sugere que se deve substituir a última parte do algoritmo por:

r ← mult (w + x, y + z)

p ← mult (w,y); q ← mult (x,z)

retorna 102sp+10s(r–p–q) + q

Seja T(n) o tempo necessário pelo algoritmo modificado para multiplicar dois inteiros no máximo de tamanho n. Sabendo-se que w + x e y + z pode ter até 1 + •n/2• dígitos, supõe-se que exista constante $c \in R^+$ e $n_0 \in N$, tal que:

$T(n) \leq T(\bullet n/2\bullet) + T(\bullet n/2\bullet) + T(1+\bullet n/2\bullet) + cn$ para cada $n \geq n0$. A equação equivale a $T(n) \in 3T(\bullet n/2\bullet) + O(n)$.

Assumindo n, sendo potência de 2, pode-se considerar que:

$T(n) = 3T(n/2) + cn$

Aplicando o método mestre, $n^{\log_b a} = n^{\log_2 3}$ e $f(n)=\Theta(n)$, daí $f(n) \in O(n^{\log_2 3})$. Assim, aplica-se o caso 1 e o algoritmo é $\Theta(n^{\log_2 3})$.

Desta forma, o algoritmo de multiplicação de inteiros de n dígitos tem complexidade $O(n^{1,59})$, para algum valor n > n0, que é melhor do que a do algoritmo clássico $O(n^2)$.

Entretanto, as constantes escondidas são tantas que o algoritmo só se torna interessante na prática quando n é bastante grande. Uma boa implementação provavelmente não usará a base 10, mas de preferência a maior base na qual o hardware pode multiplicar diretamente dois "dígitos".

Exemplo:

Deseja-se multiplicar u=2.345 e v=6.789. A decomposição inicial dos operandos é dada por n=4, s=2, w=23, x=45, y=67, e z=89. Nós obtemos sucessivamente p=23X67=1541, q=45X89=4.005 e r=(23+45)(67+89)=68x156=10.608. Finalmente, o produto desejado uv é obtido, calculando:

$1.541x10^4+(10.608–1.541–4.005)x10^2+ 4.005=15.410.000+506.200+405=15.920.205$.

É claro que este exemplo é muito pequeno e seria mais rápido usar o algoritmo clássico de multiplicação.

A multiplicação não é a única operação interessante que envolve inteiros grandes. Divisão de inteiros, operações, o módulo e o cálculo da parte inteira da raiz quadrada podem ser executados em um tempo de mesma ordem que o necessário para a multiplicação. Um outro ponto importante é que operações com inteiros grandes são de suma importância para a criptologia.

Saiba Mais
A técnica de divisão-e-conquista é parte do folclore do projeto de estruturas de dados e algortimos. O método mestre para resolver recorrências de divisão-e-conquista tem suas origens rastreadas em um artigo de Bentley, Haken e Saxe. O algoritmo de divisão-e-conquista para a multiplicação de inteiros grandes em tempo O $(n^{1,59})$ é normalmente atribuído aos russos Karatsuba e Ofman. O algortimo conhecido como assintoticamente mais rápido para multiplicar dois números de n dígitos executa em tempo O (n log n log log n).

PROGRAMAÇÃO DINÂMICA

Introdução

Programação Dinâmica resolve problemas combinando as soluções dos subproblemas.

Programação refere-se a um método tabular, não à programação no computador. A técnica de dividir-para-conquistar particiona o problema em subproblemas independentes, resolve-os recursivamente e então combina suas soluções para resolver o problema original. A programação dinâmica, por sua vez, é aplicável quando os subproblemas não são independentes, havendo assim a necessidade de uma tabela para armazenamento dos resultados a serem compartilhados e evitar cálculos repetidos.

A programação dinâmica pode ser dividida em quatro passos:

1. Caracterização da estrutura de uma solução ótima;

2. Definir recursivamente o valor de uma solução ótima;
3. Cálculo do valor de uma solução ótima;
4. Construção de uma solução ótima a partir da informação computada.

Os passos de 1 a 3 formam a base de uma solução por programação dinâmica para um problema. O passo 4º é necessário quando necessitamos informação adicional para facilitar a computação do passo 3º.

Multiplicação de Diversas Matrizes

Seja $A_1A_2...A_n$ uma sequência de n matrizes que devem ser multiplicadas. Há diversas formas de se obter o resultado desejado. Para maior clareza, apresentamos um exemplo:

Seja A1 uma matriz 13x5, A_2 5x89, A_3 89x3 e A_4 3x34. Para obter $M=((A_1A_2)A_3)A_4)$, calculamos sucessivamente

A_1A_2 - 5785 produtos

$(A_1A_2)A_3$ - 3471 produtos

$((A_1A_2)A_3)A_4)$ - 1326 produtos

para um total de 10582 produtos escalares. Há 5 maneiras diferentes de se multiplicar essas matrizes e obter o mesmo resultado final:

((AB)C)D - 10582

(AB)(CD) – 54201

(A(BC)D - 2856

A((BC)D - 4055

A(B(CD)) 26418

O modo mais eficiente é cerca de 19 vezes mais rápido que o menos eficiente.

Os problemas consistem em determinar a forma mais eficiente de parentização usando um algoritmo adequado.

Para obter a melhor maneira de calcular o produto, podemos usar o algoritmo da força bruta e verificar todas as possíveis formas. Seja T(n) o número de modos de parentização diferentes em um produto de n matrizes. Fazendo um corte entre a i-ésima e a (i+1)-ésima matrizes do produto, então:

$$M = (M_1M_2...M_i)(M_{i+1}M_{i+2}...M_n)$$

Agora, há T(i) maneiras de parentizar o termo do lado esquerdo e T(n-i) maneiras de parentizar o termo do lado direito. Há T(i)T(n-i) maneiras de parentizar T(n), e i pode assumir valores de 1 a n-1, o que nos dá a

seguinte recorrência:

$$T(n) = \sum_{i=1}^{n-1} T(i)T(n-i)$$

A condição inicial é T(1)=1. A partir daí podemos levantar os seguintes valores:

N	1	2	3	4	5	10	15
T(n)	1	1	2	5	14	4862	2674440

Os valores de T(n) são chamados números de Catalão. T(n) ∈ Ω(4n/n2) o que pode ser verificado provando que

$$\binom{2n}{n} \geq \frac{4^n}{(2n-1)} \quad \text{e que} \quad T(n) = \frac{1}{n}\binom{2n-2}{n-1}$$

No entanto, um algoritmo $\Omega(4^n/n^2)$ é impraticável para valores n grandes.

Uma abordagem seria o uso de algoritmos gulosos. No entanto, dentre os algoritmos gulosos óbvios, não permitirá a obtenção de solução ótima.

Tentaremos verificar se o princípio de otimização pode ser empregado e assim a programação dinâmica resolverá tal problema.

Passo 1: A estrutura de um parentização ótima

O primeiro passo do paradigma da programação dinâmica é caracterizar a estrutura de uma solução ótima. Adotaremos a notação $A_{i..j}$ para a matriz que resulta da avaliação do produto $A_iA_{i+1}...A_j$.

Para obter a solução do problema proposto, devemos obter $A_{1..n}$, que pode ser obtido pelo produto de $A_{1..k}A_{k+1..n}$ cujo custo ótimo é obtido pela soma do custo de $A_{1..k}$ com $A_{k+1..n}$ mais o custo do produto delas. A subcadeia A1..k deve ter parentização ótima, do contrário poderíamos substituí-la por outra com custo menor que o ótimo, o que é uma contradição.

Logo, uma solução ótima para uma instância do problema contém soluções ótimas para as subinstâncias do mesmo problema, o que permite o emprego da programação dinâmica.

Passo 2: Uma solução ótima recursiva

Deve-se definir uma expressão recursiva para a solução ótima em função das subinstâncias.

Será usada uma tabela m_{ij} $1 \leq i \leq j \leq n$ para armazenar a solução ótima para $M_i.M_{i+1}$. O valor m[i,i]=0, pois Ai..i = Ai, não havendo necessidade de qualquer cálculo.

Para i<j, podemos usar a estrutura ótima delineada no passo 1. Assim Ai..j pode ser dividido em duas partes $A_{i..k}A_{k+1..j}$, $i \leq k < j$, e m[i,j] é igual ao menor custo para calcular $A_{i..k}$ e $A_{k+1..j}$ mais o custo para multiplicar essas duas matrizes. O custo para multiplicar $A_{i..k}A_{k+1..j}$ vale $p_{i-1}p_kp_j$

$$m[i,j] = \begin{cases} 0 & se\ i = j \\ \min\{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j\} & se\ i < j \end{cases}$$

O valor m[i,j] dá o custo ótimo, mas não informações para a construção de uma solução ótima.

Para facilitar a indicação de uma parentização ótima, basta armazenar na matriz s[i,j] o valor de k usado para o valor ótimo de m[i,j], ou seja $m[i,j]=m[i,k]+m[k,j] + p_{i-1}p_kp_j$.

Passo 3: Determinando a Solução ótima

Usando a programação dinâmica, passamos a ter Θ(n2) subproblemas (basta observar que m[i,j] armazena os valores ótimos desses subproblemas), bem superior ao algoritmo de força bruta que é exponencial.

Neste ponto deve-se elaborar um algoritmo para resolução do problema, fazendo os cálculos de tal forma que nenhuma solução seja requisitada antes que a mesma já tenha sido calculada.

Fazendo um exemplo, pode-se perceber que o problema deve ser resolvido em ordem crescente de comprimento da cadeia de matrizes, o que equivale a percorrer as diagonais superiores da matriz de custo, a partir da diagonal maior.

```
1 proc Parentizar_Cadeia_Matriz(p:vetor[0..n];s:vetor[1..n,1..n])
2 {p armazena as dimensões das n matrizes }
3 para i:=1 até n, faça:
4  m[i,i]:=0;
5  para d:=2 até n, faça:
6  para i:=1 até n-d+1, faça:
7  j:=i+d-1;
8  m[i,j ]: = min i ≤k< j { m[i,k] + m[ k,j] + p[i−1]p[k]p[j] }
9  s[i,j]:= k , tq m[i,j] = m[i,k]+m[k,j]+p[i-1]p[k]p[j]
```

Desafio:
Dado um conjunto P de n times de futebol, projete um algoritmo para organizar um torneio *round-robin*, em que cada time joga com cada um dos outros exatamente uma vez, assumindo que n é uma potência de 2.

O algoritmo computa, inicialmente, m[i,i]← 0, para i=1,2,...,n (custo mínimo para cadeias de comprimento 1). Então, usa a recorrência para calcular m[i,i+1], para i=1,2,...,n-1 (custo mínimo para cadeias de comprimento d=2) e assim por diante até d=n. Em cada passo, o custo m[i,j] depende apenas dos valores m[i,k] e m[k+1,j] que já foram calculados.

A complexidade do algoritmo pode ser determinada, observando que há dois laços (linhas 5 e 6) executando $O(n^2)$ vezes as linhas 7 e 8. A linha 8 tem complexidade O(n) no pior caso, o que conduz a uma complexidade $O(n^3)$ para o algoritmo todo.

Passo 4: Construindo uma solução ótima

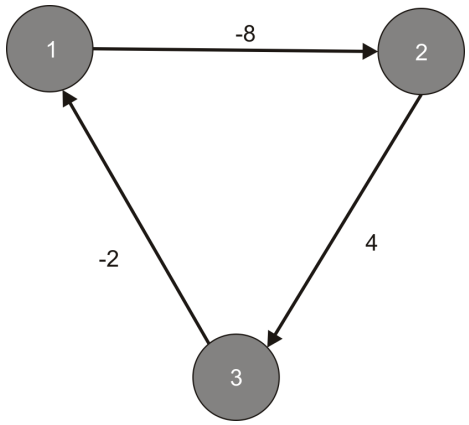
O algoritmo apresentado determina o número ótimo de multiplicações escalares para calcular o produto de uma cadeia de matrizes, mas não mostra diretamente como multiplicar as matrizes. O passo 4 constrói uma solução ótima a partir da informação obtida. A tabela s[1..n,1..n] determina a melhor forma de multiplicar as matrizes através do valor k usado na melhor partição da cadeia. O seguinte algoritmo recursivo calcula o produto da cadeia de matrizes de forma ótima:

A chamada inicial deve ser Produto_Cadeia_Matrizes(A,s,1,n).

O Problema de Caminho Mínimo

Utiliza-se algoritmo que faz uso de técnicas de programação dinâmica, denominado algoritmo de Dijkstra, para determinação de caminho mínimo de um nó para todos os demais nós de um grafo orientado com arestas

ponderadas e com pesos não negativos. Queremos agora resolver um problema amplo que determine a distância mínima e o respectivo caminho mínimo para grafos orientados com arestas ponderadas, sendo permitido o uso de arestas negativas.



Sobre o uso de arestas negativas, queremos alertar desde já que a existência de ciclos negativos representa um erro e modelagem de problemas, pois leva a uma solução ilimitada, ou seja, sempre será possível reduzir o custo em tal ciclo. A figura, a seguir, ilustra um fragmento de grafo contendo um ciclo negativo.

Primeira preocupação é tratar o que tem de bom no algoritmo de Dijkstra, ou seja, armazenar a distância do caminho mínimo especial do nó u ao nó v. De que forma pode armazenar esse valor, observando que desejamos determinar a distância de todo nó u para todo nó v ?

A resposta é simples. Basta usar uma matriz bidimensional para armazenar as distâncias e supor que os nós do grafo são representados por números inteiros.

Para formular o problema como problema de programação dinâmica, vamos imaginar o que acontece quando você joga uma pedra em um lago, forma-se uma onda circular que vai aumentando de raio até atingir uma distância suficientemente grande. No nosso problema, vamos aplicar o conceito da onda procurando primeiro um caminho de um nó u para um nó v usando apenas uma aresta, depois procurar caminhos que possam melhorar os existentes usando 2 arestas, e assim por diante, até n-1 arestas.

Saiba Mais
A técnica de programação dinâmica foi desenvolvida pela comunidade de pesquisa operacional e formalizada por Richard Bellman. A solução para o produto de matrizes encadeadas descritas é atribuída a Godbole. O método assintoticamente mais rápido é atribuído a Hu e Shing.

O algoritmo proposto poderia ser:

1. caminho_geral(G:matriz[1..n,1...n]):matriz[1..n,1...n];
2. Dist:matriz[1..n,1...n];
3. Dist<-G;
4. para k<-2 até n-1 faça:
5. para i<-1 até n-k, faça:
6. j<- i + k
7. Dist[i,j]<- Min{Dist[i,j],

ALGORITMOS GULOSOS

Introdução

É uma família de algoritmos muito utilizada para resolver problemas de otimização. Como o nome sugere a estratégia usada por esses algoritmos consiste na escolha da melhor solução em determinado instante. Isto é, ele faz uma escolha ótima localmente esperando que esta escolha leve a uma solução ótima global. Por vezes conduz a uma solução ótima, mas nem sempre isso ocorre. Esta seção explora problemas de otimização que são solucionáveis por algoritmos gulosos.

São fáceis de implementar e quando trabalham bem, são eficientes.

O Problema do Troco

Suponha que vivemos em um país onde as seguintes moedas são usadas: sonho (100 centavos), 25 centavos, 10 centavos, 5 centavos e 1 centavo. Nosso problema consiste em pagar troco com a menor quantidade de moedas.

Para pagarmos $2,89, a solução ótima consiste em dar 2 sonhos, 3 moedas de 25 centavos, 1 moeda de 10 centavos e 4 de 1 centavo, totalizando 10 moedas.

Para resolvermos esse problema, adotaremos a estratégia gulosa de usar a maior moeda disponível que não ultrapasse o troco a ser pago. O algoritmo pode ser formalizado como abaixo:

```
1  função troco(n):conjunto de moedas
2  constante C={100,25,10,5,1}
3  S<- ∅ {S é o conjunto solução }
4  s<- 0 { s é a soma dos itens em S}
5  enquanto s<n, faça:
6  x<-o maior item em C, tal que s+x ≤ n
7  Se não há tal item, então: retornar 'não há solução'
8  S<- S ◡ {Moeda[x]}
9  s<- s + x
10 retornar S
```

Para o conjunto de moedas do exemplo, é considerado um suprimento ilimitado de cada tipo a ser obtido. Para outros valores, principalmente quando há valores múltiplos, o algoritmo não obtém solução ótima.

O algoritmo é guloso porque em cada passo escolhe a maior moeda que pode e uma vez escolhida uma moeda, ela não será trocada.

Características Gerais

Conjunto ou lista de candidatas

A solução conterá somente elementos desse conjunto. No caso do troco, as candidatas são os tipos de moedas.

Conjunto Solução

Armazena as candidatas selecionadas para fazer parte da solução. Normalmente, há outro conjunto com as candidatas rejeitadas.

Solução válida

É uma função que verifica se o conjunto solução corrente provê uma solução para nosso problema, independente de ser ótimo ou não. Para o problema do troco, essa função verifica que o valor das moedas do conjunto solução não ultrapassa o valor a ser pago.

Função viabilidade

Verifica se um conjunto solução parcial é ou não viável. Essa função permite verificar se a adição de candidatas conduz a alguma solução do nosso problema. Esta função verifica se um dado conjunto solução está ou não viável. No caso do troco, verifica se a soma não ultrapassa o valor do troco. Diferente da função "solução válida" que verifica que a solução atinge todas as restrições do problema, que no caso do troco deve totalizar exatamente o valor do troco.

Função seleção

Indicam quais das candidatas restantes é mais promissora. Esta função implementa a estratégia gulosa e tem por objetivo atingir a otimização.

Função objetivo

Determina o valor de uma solução encontrada. No caso do troco, a função objetivo determina o número de moedas utilizadas.

```
1  função guloso(C: conjunto):conjunto
2  *{C é o conjunto de candidatas}
3  S←∅
4  enquanto C≠ ∅ e não Solução(S, faça:
5  x ← Seleciona(C)
6  C ← C \ {x}
7  se viável (S ◡ {x}) então S ← S ◡ {x}
8  se Solução(S), então: retornar S
9   senão, retornar “Não há solução”
```

Desafio:
Forneça um exemplo de um conjunto de moedas que faça o algoritmo guloso de fazer troco não usar o número mínimo de moedas.

A denominação “guloso” se deve ao fato de que em cada passo, o procedimento escolhe a melhor fatia que ele pode comer, sem se preocupar com o futuro. Ele jamais faz uma troca: uma vez que um candidato foi incluído na solução, jamais é retirado.

Problema da Árvore Geradora Mínima

Seja G=(V,E) um grafo conexo, não orientado, onde V é o conjunto de nós e E o conjunto de arestas. Cada aresta tem um comprimento não negativo associado, representado por uma função Comp: E -> R^+.

O problema é achar um subconjunto T de arestas de G, tal que todos os nós permaneçam conectados usando apenas as arestas de T e a soma dos comprimentos dessas arestas é a menor possível. Dadas duas soluções com soma dos comprimentos iguais, usaremos a solução com o menor número de arestas.

Qual o menor número de arestas que deve haver em T tal que o grafo parcial G’ seja conexo ?

Resposta: n-1 arestas, cada aresta adicional acrescenta pelo menos um ciclo.

O grafo G’ é chamado árvore geradora mínima do grafo G.

1. Claramente, o nosso conjunto de candidatas é o conjunto E de arestas de G.

2. A função solução válida verifica se a adição de uma aresta ao conjunto solução não introduz algum ciclo.

3. A função viabilidade verifica se o conjunto solução é uma árvore.

4. A função de seleção varia com o algoritmo.

5. A função objetivo é minimizar o comprimento total das arestas na solução.

Algoritmo de Kruskal

O algoritmo de Kruskal inicia com um conjunto T de arestas vazio e n conjuntos disjuntos, cada um com um nó do grafo G=(V,E). O conjunto de Arestas E é ordenado em ordem crescente dos pesos. A cada iteração, a aresta de menor peso é candidata, sendo aceita apenas se seus nós extremos não pertencerem ao mesmo conjunto. Caso a aresta estivesse unindo pontos do mesmo conjunto estaria formando ciclo. Observe que cada componente de T (cada conjunto disjunto) forma uma árvore. O algoritmo encerra quando restar apenas um componente (um conjunto), ou seja, quando n-1 arestas forem adicionadas.

O algoritmo de Kruskal será apresentado a seguir.

1 função Kruskal(G=(V,E): grafo; Comprimento: E -> R+): conjunto de arestas { inicialização }
2 Ordenar E por ordem crescente do comprimento
3 n ← |V|
4 T ← ∅ { arestas da árvore geradora mínima }
5 Inicializar n conjuntos, cada um com um nó de V
6 {laço guloso}
7 repita
8 e ← {u,v}// a menor aresta não considerada
9 ucomp ← Achar(u)
10 vcomp ← Achar(v)
11 Se ucomp ≠ vcomp então
12 Juntar (ucomp,vcomp)
13 T ← T ◡ {e}
14 até |T| = n-1
15 retornar T

Exemplo:

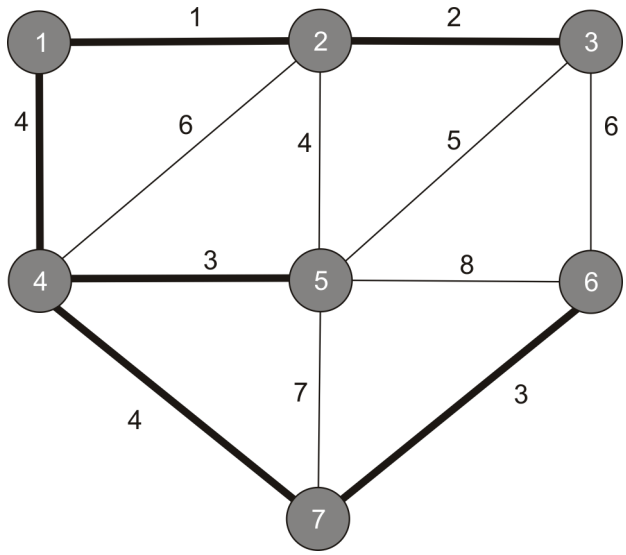


Figura 2 : Um grafo e sua árvore geradora mínima

Ordenando as arestas:

{1,2}, {2,3}, {4,5}, {6,7}, {1,4}, {2,5}, {4,7}, {3,5}, {2,4}, {3,6}, {5,7} e {5,6}

Passo	**Aresta considerada**	**Componentes conexos**
Inicialização	-	{1}{2}{3}{4}{5}{6}{7}
1	{1,2}	{1,2}{3}{4}{5}{6}{7}
2	{2,3}	{1,2,3}{4}{5}{6}{7}
3	{4,5}	{1,2,3}{4,5}{6}{7}
4	{6,7}	{1,2,3}{4,5}{6,7}
5	{1,4}	{1,2,3,4,5}{6,7}
6	{2,5}	rejeitada
7	{4,7}	{1,2,3,4,5,6,7}

O algoritmo encerra com T={{1,2}, {2,3}, {4,5}, {6,7}, {1,4} e {4,7}} e comprimento mínimo total igual a 17.

A complexidade do algoritmo pode ser analisada:

1. Ordenação das arestas = O(m log m) = O(m log n), pois n-1 ≤ m ≤ n(n-1)/2

2. Inicialização dos n conjuntos disjuntos - O(n) são executados no máximo 2m achar, cada um pode gastar até log n operações são executados n-1 juntar, cada um podendo gastar até log n operações.

3. As demais operações exigem no máximo O(m).

Concluímos que o algoritmo é O(mlog m) ou O(mlog n).

Problema do Caminho Mínimo

Considere agora um grafo orientado G=(V,E), onde V é o conjunto de vértices e E o conjunto de arcos. Cada arco tem um comprimento não negativo. Um dos nós é designado nó fonte. O problema é determinar o comprimento do caminho mínimo do nó fonte para cada um dos demais nós do grafo.

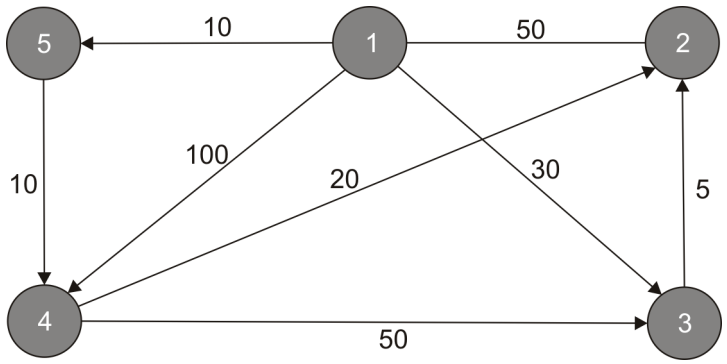
Este problema pode ser resolvido pelo algoritmo guloso de Dijkstra. Esse algoritmo usa um conjunto C que contém inicialmente todos os nós do grafo e um conjunto S onde estão os nós selecionados pelo algoritmo, cuja distância para o nó fonte é mínima.

Os nós do conjunto C possuem distância para o nó fonte passível de melhora. Os conjuntos S e C são disjuntos e V = S ◡ C.

```
1 Algoritmo de Dijkstra
2 {retorna um vetor com a distância mínima do nó fonte a todos os demais nós}
3 função Dijkstra(fonte:vertice;G:Grafo;L:E-> R+):vetor[1..n]
4 vetor Dist[1..n] de inteiro
5 {inicialização}
6 C←V
7 S←∅
8 para ∀u ∈ V faça Dist[u] ← ∞
9 Dist[fonte] ← 0
10 {laço guloso}
11 enquanto C ≠ ∅, faça:
12  u ← Min{Dist[u ]}
        u ∈ C
13 C ← C \ {u}
14 S ← S ◡ {u}
```

15 para ∀w ∈ Adj(u), faça:
16 Dist[w] ← Min{Dist[w] , Dist[u] + L(u,w)}
17 retornar Dist

Exemplo:



Passo	u	C	D
Inicialização	-	{2,3,4,5}	[50,30,100,10]
1	5	{2,3,4}	[50,30,20,10]
2	4	{2,3}	[40,30,20,10]
3	3	{2}	[35,30,20,10]

Observe que o algoritmo só calcula a distância, mas não determina o caminho mínimo. Como modificá-lo para determinar o caminho mínimo?

Solução:

Usar um vetor pred [2..n] para armazenar os predecessores dos nós.

Inicializar Pred da seguinte forma:

Pred[i] ← 0, ∀i ∈ V

Adicionar ao laço interno

D[w] ← D[u] + L(u,w)

Se D[w] > D[u] + L(u,w) então

P[w] ← u

Teorema:

Se o algoritmo de Dijkstra for executado sobre um grafo orientado com as arestas ponderadas com pesos não-negativos e dado um vértice

fonte s, então o algoritmo termina determinando o caminho mínimo entre o vértice s e todos os demais vértices do grafo.

Prova:

Chamaremos dist [u] a distância relaxada do vértice u ao vértice s(ao longo do algoritmo) e δ(s,u) a distância mínima do vértice s para o vértice u.

Provaremos por indução matemática que:

1. dist(u) = δ(s,u), ∀ u ∈ V, a partir de quando o vértice u é inserido no conjunto S.

2. Se um nó u ∉ S, então dist[u] dá o comprimento do menor caminho especial de s para u.

1. Base: inicialmente, somente o nó fonte s está em S, assim que a condição a) é verdadeira.

2. Hipótese de Indução:

Supondo que as condições a) e b) são verdadeiras antes de adicionarmos um nó u ao conjunto S

3. Passo indutivo para a)

Para cada nó pertencente a S, antes da adição do nó u, não há alteração pelo algoritmo, assim a condição a) permanece verdadeira. Ao adicionarmos o nó u ao conjunto S devemos garantir que dist(u)= δ(s,u). Se o caminho encontrado pelo algoritmo é $s \xrightarrow{p_1} x \rightarrow u$, onde x ∈ S.

Supondo que haja outro caminho alternativo, s y z u , tal que y é o único nó de p1 que não pertence a S e z ∈ S e predecessor e u. A distância total por y vale δ(s,y)+ δ(y,u) ≤ dist(y) + δ(y,u) ≤ dist(u), o que nos leva a concluir que dist[y] < dist[u], no momento da seleção de u pelo algoritmo, o que é uma contradição, pois se isso fosse verdade, o algoritmo selecionaria y antes de u. Assim, quando u é adicionado a S, a indução permanece verdadeira.

4. Passo indutivo para b):

Considere um nó w ∉ S diferente de u. Quando u é adicionado a S, há duas possibilidades para o caminho mínimo especial de s para v. O valor dist[w] não troca, ou o caminho de s para w passa por u (e possivelmente por outros nos de S). No segundo caso, sendo x o último nó de S antes de atingir w(pois o algoritmo sempre atualiza dist em relação a um nó inserido em S), o valor Dist[w]=Dist[x]+L(x,w) para todo nó x em S(incluindo u). No entanto, para todo nó x em S, exceto u, esta comparação foi feita quando x foi adicionado a S e Dist[x] não mudou, desde então. Então, o novo valor de Dist[w]= Dist[u]+L(u,w) deve ser comparado com o valor antigo Dist[w]=Dist[x]+L(x,w).

Desde que o algoritmo faz isto explicitamente, ele assegura a parte b) da indução que também permanece válida quando um novo nó u é adicionado a S.

Para completar a prova de que o algoritmo está correto, observe que quando o algoritmo para, todos os nós estão em S e o caminho mínimo do nó s para os demais nós é um caminho especial. Quando Dist[x]=∞ então o caminho δ(s,x) não existe.

Análise do algoritmo:

Considere que a lista de prioridades C é implementada com o uso de lista sequencial. Assim a construção será O(n), a alteração O(1) e remoção O(n).

A inicialização do algoritmo como um todo requer O(n) passos.

A determinação do nó de menor distância requer O(n) operações, executadas n vezes, sendo $O(n^2)$.

A atualização da distância (laço interno) é executada, no pior caso, m vezes, uma para cada aresta, sendo O(m).

Logo, a complexidade do algoritmo é $O(m+n^2)$, o que pode ser limitado superiormente por $O(n^2)$.

É importante ressaltar que o balanceamento de atividades ao longo do algoritmo ajuda muito a reduzir a complexidade do mesmo, isto é, é melhor usar uma estrutura de dados que requer mais operações em um trecho aliviado do algoritmo, mas que reduza a carga em outro trecho mais carregado.

Problema da Mochila

Dada uma mochila com capacidade para transportar no máximo W kg e uma lista de n itens a serem transportados. O i-ésimo elemento da lista tem peso wi e valor vi. Selecionar a carga a ser transportado de tal forma que o valor transportado seja o maior possível.

A modelagem como problema de programação matemática pode ser feita da seguinte forma:

Variáveis de decisão:

$x_i = 1$, caso o i-ésimo item seja alocado

$x_i = 0$, caso contrário

Problema de Programação Matemática (P.P.M.)

$Minx_0 = \sum_{i=1}^{n} x_i v_i$ função objetivo – valor transportado

Sujeito a

$\sum_{i=1}^{n} x_i v_i \leq W$, $x_i \in \{0,1\}$ - restrição referente à capacidade da mochila

Este problema é chamado problema da mochila 0-1 por não permitir o uso de frações de cada item, mas uma escolha binária (sim ou não).

Um algoritmo guloso deve usar como estratégia gulosa a seleção dos itens com maior razão v_i/w_i. Apresentaremos um exemplo para aplicar tal heurística.

Passo	**u**	**C**	**D**
Inicialização	-	{2,3,4,5}	[50,30,100,10]
1	5	{2,3,4}	[50,30,20,10]
2	4	{2,3}	[40,30,20,10]
3	3	{2}	[35,30,20,10]

Se mochila tivesse capacidade W=50, qual seria a solução dada pela heurística gulosa?

Pegaria os objetos 1 e 2, cujos valores totalizam $160. No entanto, se pegássemos os itens 2 e 3 teríamos valor total de $220. Observe que o algoritmo guloso não fornece uma solução ótima.

O problema da mochila fracionário pode ser modelado da mesma forma, permitindo o uso de itens fracionados, ou seja, $0 \leq x_i \leq 1$, i=1,...,n. Para o exemplo dado, a seleção gulosa proposta daria como solução X=(1,1,2/3), dando o valor da função objetivo 1*60 + 1*100 + (2/3)*120 = 60+100+80= $240.

Será que o algoritmo guloso resolve este problema de forma ótima?

Teorema: O algoritmo guloso resolve o problema da mochila fracionário de forma ótima.

Prova:

Sem perda de generalidade, supondo que os itens estejam ordenados pela ordem não crescente da razão v_i/w_i. Por contradição, supondo que seja possível retirarmos uma quantidade q do item i da solução fornecida pelo algoritmo guloso e substituirmos pela mesma quantidade q do item j de forma tal que o valor a ser transportado seja maior. O valor transportado obtido pelo algoritmo guloso vale:

$$x_0 = \sum_{i=1}^{n} x_i \cdot \frac{v_i}{w_i}$$

Com a retirada de uma quantidade q de um elemento i da solução e a colocação da mesma quantidade de um elemento j, o valor a ser transportado passa a ser:

$$x_0^{'} = x_0 - \frac{q}{w_i} v_i + \frac{q}{w_j} v_j$$

A hipótese é de $x_{0'} > x_0$, de onde se conclui que $x_{0'} - x_0 > 0$, e assim,

$$q\left(\frac{v_i}{w_i} + \frac{v_j}{w_j}\right) > 0 \Rightarrow \frac{v_j}{w_j} > \frac{v_i}{w_i}$$

o que é uma contradição.

Logo, o algoritmo guloso sempre acha uma solução ótima para o problema da mochila fracionário.

Complexidade do Problema da Mochila Fracionário:

Para resolver o problema da mochila fracionário, uma abordagem consiste em ordenar os valores v_i/w_i, o que exige O(nlogn) passos, onde m é o número total e elementos.

Resumindo, os passos para o projeto de algoritmos gulosos são:

1. Formule o problema como um problema de otimização (P.P.M.), no qual uma escolha é feita, restando-nos então resolver um único subproblema a resolver.

2. Provar que existe sempre uma solução ótima do problema que atende à escolha gulosa, ou seja, a escolha feita pelo algoritmo guloso e segura.

3. Demonstrar que, uma vez feita a escolha gulosa, o que resta a

resolver é um subproblema tal que se combinarmos a resposta ótima deste subproblema com o(s) elemento(s) da escolha gulosa, chega-se à solução ótima do problema original. Esta é a parte que requer mais “engenhosidade”.

4. Normalmente, a prova começa com uma solução ótima genérica e mostram que ela pode ser modificada possivelmente apos vários passos até que ela inclua o(s) elemento(s) identificado(s) pela escolha gulosa.

Código de Huffman

Esta é uma estratégia eficiente para a compressão de dados, muito utilizada em sistemas para internet, que utiliza uma tabela das frequências de ocorrência dos caracteres para elaborar um modo ótimo de representar cada caractere como uma cadeia binária.

As reduções no tamanho dos arquivos dependem das características dos dados contidos nos mesmos. Em geral, os valores típicos oscilam entre 20% e 90%.

Exemplo: arquivo texto contendo 100.000 caracteres no alfabeto e = {a; b; c; d; e; f}. As frequências de cada caractere no arquivo são indicadas na tabela abaixo.

Saiba Mais
O termo “algoritmo guloso” foi criado por Jack Edmonds em 1971, embora o conceito já existisse desde a segunda grande guerra mundial.

	a	**b**	**c**	**d**	**e**	**f**
Frequência	4	1	1	1	9	5
Código Fixo	0	0	0	0	1	1
Código Variável	0	1	1	1	1	1

A codificação do arquivo consiste em representar cada caractere por uma sequência de bits.

Exemplo: Para armazenar compactamente um arquivo de dados com 100.000 caracteres

Observando a tabela que representa cada caractere por códigos, qual o tamanho (em bits) do arquivo comprimido usando os códigos acima?

Por um código de tamanho fixo (3 *bits*) : 3 x 100.000 = 300.000

Por um código de tamanho variável:

45 x 1 + 13 x 3 + 12 x 3 + 16 x 3 + 9 x 4 + 5 x 4
=
224.000 bits

Códigos Livres de Prefixo

Códigos livres de prefixo são aqueles dados dois caracteres quaisquer i e j representados pela codificação, a sequência de *bits* associada a i não é um prefixo da sequência associada a j. São códigos nos quais nenhuma palavra de código é também um prefixo de alguma outra palavra de código.

Pode-se provar que sempre existe uma solução ótima do problema da codificação que é dado por um código livre de prefixo.

O processo de codificação, ou seja, de geração do arquivo comprimido é sempre fácil, pois se reduz a concatenar os códigos dos caracteres presentes no arquivo original em sequência.

Por exemplo, usando a codificação de tamanho variável do exemplo anterior, o arquivo original dado por "abc" seria codificado por 0101100.

A vantagem dos códigos livres de prefixo se torna evidente quando vamos decodificar o arquivo comprimido.

Como nenhuma palavra de código é um prefixo de qualquer outra, a palavra de código que inicia um arquivo codificado não apresenta ambiguidade.

Pode-se simplesmente identificar este código inicial, traduzi-lo de volta ao caractere original e repetir o processo no restante do arquivo comprimido.

Exemplo: usando a codificação de tamanho variável do exemplo anterior, o arquivo comprimido contendo os *bits* 001011101 divide-se de forma unívoca em 0 0 101 1101, ou seja, corresponde ao arquivo original dado por “aabe”.

O processo de decodificação precisa de uma representação conveniente para o código de prefixo, de forma que a palavra de código inicial possa ser extraída com facilidade. A solução para isso é utilizar árvore binária

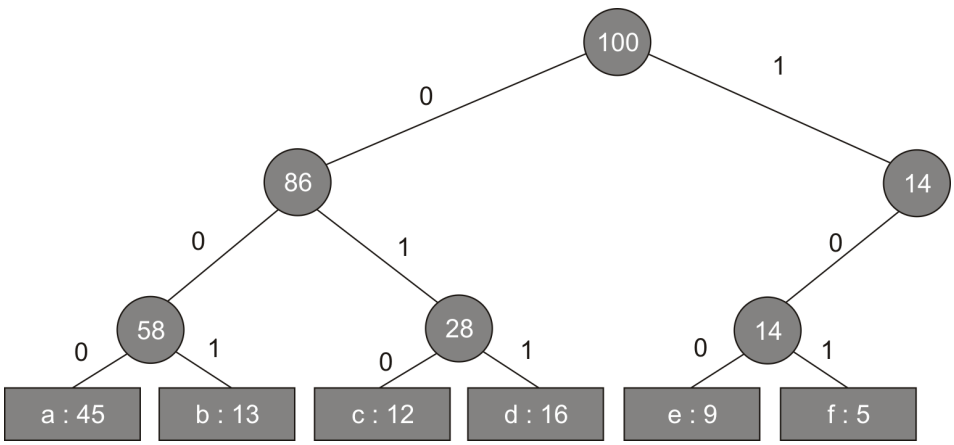
Se C é o alfabeto do qual os caracteres são obtidos e todas as frequências de caracteres são positivas, então a árvore para um código de prefixo ótimo tem exatamente |C| folhas, uma para cada letra do alfabeto e exatamente |C|–1 nós internos. Dada uma árvore T, para cada caractere c no alfabeto C, seja f(c) a frequência de c no arquivo e seja dT(c) a profundidade da folha de c na árvore (ou o comprimento da palavra código). O número de *bits* exigidos para codificar um arquivo (ou custo da árvore) é:

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

O filho esquerdo está associado ao bit 1 enquanto o filho direito fica associado ao *bit* 0. Nas folhas encontram-se os caracteres presentes no arquivo original.

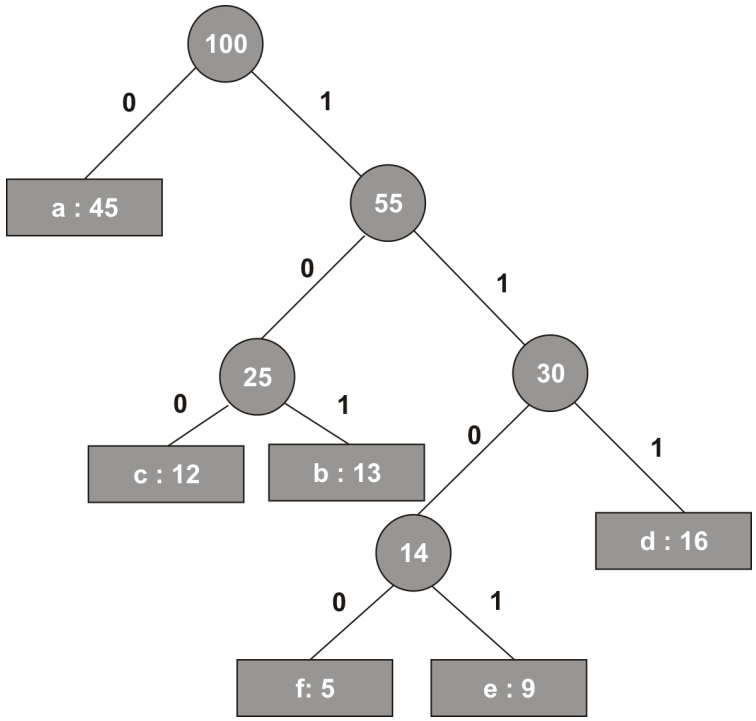
Árvore de Tamanho fixo:

	a	**b**	**c**	**d**	**e**	**f**
Frequência	4	1	1	1	9	5
Código Fixo	0	0	0	0	1	1



Árvore de Tamanho variável:

	a	**b**	**c**	**d**	**e**	**f**
Frequência	4	1	1	1	9	5
Código Variável	0	1	1	1	1	1



Huffman criou um algoritmo guloso que produz um código de prefixo ótimo chamado código de Huffman.

No pseudocódigo, C é um conjunto de n caracteres e cada caractere c ∈ C é um objeto com uma frequência definida f [c].

O algoritmo constrói de baixo para cima a árvore T correspondente ao código ótimo.

Começa com um conjunto de |C| folhas e executa uma sequência de |C|–1 operações de intercalação para criar a árvore final. Uma fila de prioridade mínima Q, tendo f como chave, é usada para identificar os dois objetos menos frequentes a serem intercalados. O resultado da intercalação é um novo objeto cuja frequência é a soma das frequências dos 2 objetos que foram intercalados.

1 Huffman (C)

Entrada: conjunto de caracteres C e as frequências f dos caracteres em C

Saída: raiz de uma árvore binária representando uma codificação ótima livre de prefixos

2 n ← |C|

3 Q ← C # Q é a fila de prioridades dada pelas frequências dos vértices ainda não intercalados

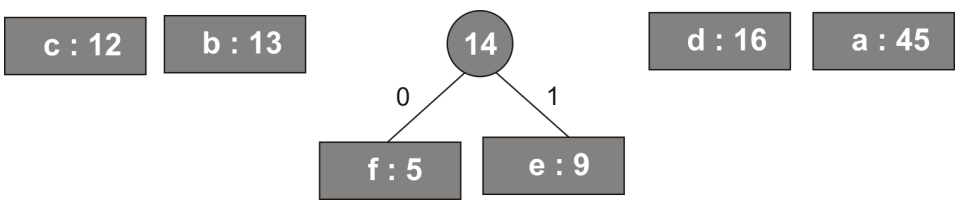
```
4 para i ← 1 até n -1 faça
5 aloque um novo nó z
6 esq[z] ← x ← EXTRACT-MIN(Q)
7 dir[z] ← y ← EXTRACT-MIN(Q)
8 f[z] ← f[x] + f[y]
9 insert(Q,z)
10 retorne EXTRACT-MIN(Q)   # retorna a raiz da árvore
```

A seguir, mostraremos como funciona o algoritmo para as frequências dadas na tabela inicial. Cada passo mostra o conteúdo da fila classificada crescentemente de acordo com a frequência. A cada passo, duas árvores com menores frequências são unidas. Os nós folhas são exibidos com retângulos contendo um caractere e sua frequência. Nós internos são exibidos com círculos contendo a soma das frequências dos nós filhos. Como tratado anteriormente, rotula-se com 0 o conector ao conectar um nó pai com seu filho esquerdo e 1 ao seu filho direito. O código para um caractere é a sequência de rótulos da raiz até chegar ao caractere.

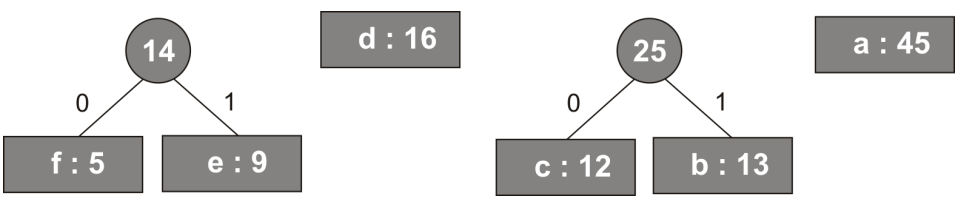
Passo 1



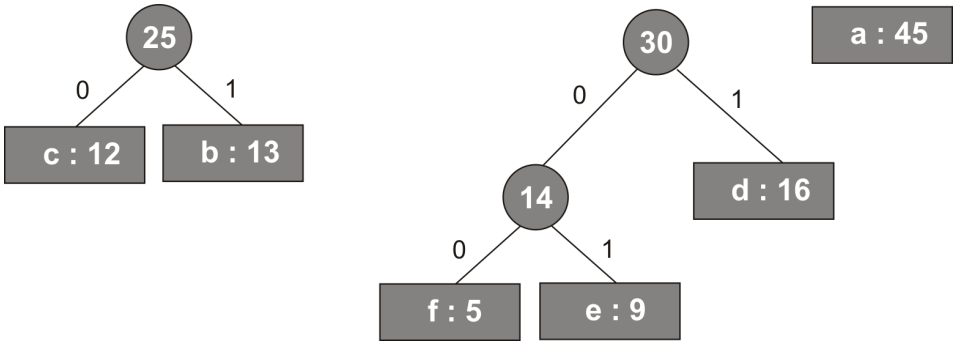
Passo 2



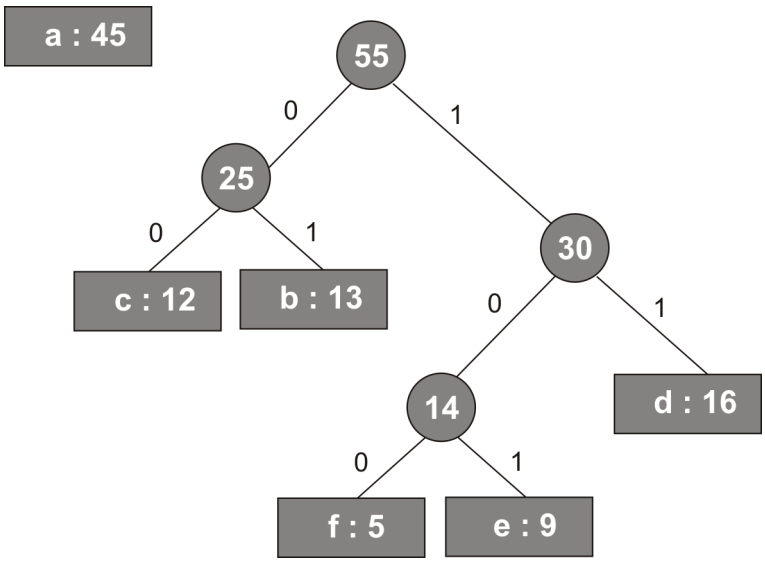
Passo 3



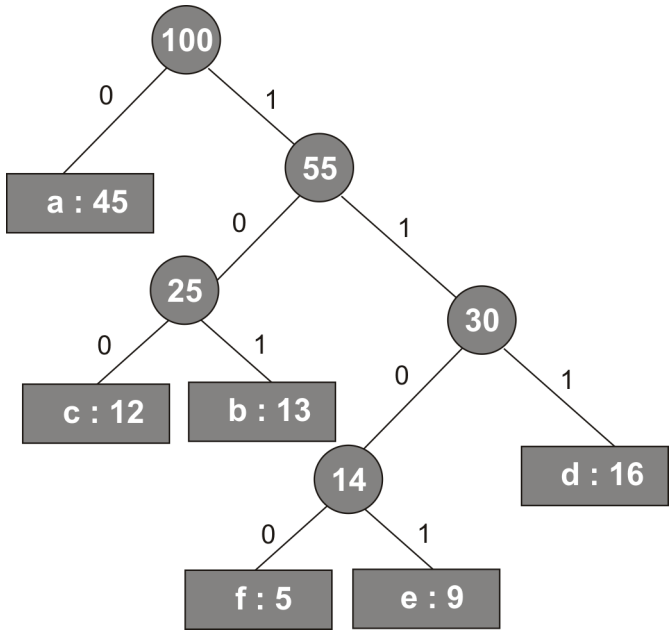
Passo 4



Passo 5



Passo 6 a árvore final



Fazendo a Análise do Algoritmo, temos que os custos são:

5. Construir Heap O(n)

6. (n − 1) iterações, cada uma com 3.O(log n)

Portanto, o algoritmo executa em tempo O(n log n)

Exercício 1

1. O Professor Chiquinho elaborou um algoritmo para realizar o merge de k listas ordenadas, cada uma com n/k elementos. O algoritmo pega a primeira lista e realiza o merge com a segunda lista, fazendo uso de um algoritmo de tempo linear para fazer o merge de duas listas ordenadas como o algoritmo de merge do MergeSort. Então, um merge é feito com a lista resultante de 2n/k elementos e a terceira lista, um novo merge com a lista resultante de 3n/k elementos e a quarta lista, e assim sucessivamente, até conseguir uma única lista ordenada com todos os n elementos. Faça uma análise de pior caso do tempo de execução do algoritmo do Professor Chiquinho. Apresente o resultado em termos de n e k.

2. Representar o polinômio $p(n)=a_0+a_1n+a_2n^2+...+a_dn^d$ de grau d por um vetor P[0,...,d] contendo seus coeficientes. Supondo que você tenha um algoritmo capaz de multiplicar um polinômio de grau k por um polinômio de grau 1 em tempo O(k), bem como um outro algoritmo capaz de multiplicar dois polinômios de grau k em tempo O(k.logk). Sugira um algoritmo eficiente baseado na técnica dividir-e-conquistar que encontre o único polinômio p(n) de grau d cujo coeficiente do termo de maior grau é 1, tal que $p(n_1)= p(n_2)= ... =p(n_d) = 0$, para n_1, n_2 e n_d inteiros. Analise a eficiência do algoritmo sugerido.
3. Considere n pontos distribuídos ao longo do eixo dos x. O objetivo é encontrar os dois pontos com menor distância mútua.

a) Escreva um algoritmo baseado na estratégia dividir-e-conquistar que resolva o problema. Indique a complexidade do algoritmo.

b) Se utilizarmos força bruta, qual a complexidade desta solução?

4. Dados n pontos no plano de coordenadas cartesianas, sugerir um algoritmo capaz de achar o par de pontos mais próximos em O(n.logn) passos no pior caso.
5. Considere um *array* A de números reais. Escreva um algoritmo O(n.log n) que encontre um par de elementos (x, y) de A, em que |x + y| seja mínimo.
6. Um "switch" é um circuito com duas entradas, um controle e duas saídas. A entrada A sairá na saída correspondente; A e B sairá na saída B ou A sairá em B e B sairá em A, dependendo da posição do controle. Use estes "switches" para construir uma rede com n entradas e n saídas tal que qualquer das n! das entradas seja possível na saída. O número de "switches" deve ser O(n.logn).
7. Um problema que está relacionado com o problema de ordenação é o de encontrar o k-ésimo menor elemento de uma lista não ordenada.

a) Escreva um algoritmo que encontre o k-ésimo menor elemento. Esse algoritmo deve ser Θ(n) na média e no melhor caso.

b) Argumente que seu algoritmo está correto.

c) Efetue a análise de complexidade de sua solução para melhor caso, pior caso e caso médio.

8. Dados os pontos $P_1(x_1,y_1,h_1)$ e $P_2(x_2,y_2,h_2)$, onde x_i,y_i são as coordenadas no plano e hi a altura do ponto. Considerando que tais pontos estão sobre um terreno real e que ao terreno foi associada uma matriz InfoTerreno[0..max,0..max] com cada célula representando uma célula de terreno de dimensões kxk em metros e cada célula da matriz armazena os

campos “elevação” (que corresponde à altura do ponto médio do terreno representado pela célula) e tipo de terreno. O tipo de terreno pode ser 1=limpo, 2=vegetação, 3=floresta, 4=urbano, 5=estrada. As coordenadas dos pontos P_1 e P_2 são em metros. Desenvolver um algoritmo para determinar se há uma linha de visada direta entre os mesmos sem obstáculos. São considerados obstáculos a existência de área urbana, vegetação ou floresta por um trecho superior a 20 metros ou o terreno Ter altura superior à altura da linha de visada.

9. Seja um conjunto de n garrafas distintas e n correspondentes rolhas (existe uma correspondência de um-pra-um entre garrafas e rolhas). Não é permitido comparar duas garrafas ou duas rolhas diretamente, mas é permitido comparar uma garrafa e uma rolha para saber quem é maior das duas. Escreva um algoritmo para encontrar o casamento entre garrafas e rolhas. É esperado que, no caso médio, a complexidade de seu algoritmo seja O(n.log n).
10. Considere uma lista com n números distintos x_1, x_2, ..., x_n, em ordem arbitrária, e um valor k < n. Escreva em pseudocódigo, um algoritmo que imprima os k menores valores da lista, em qualquer ordem. Por exemplo, se k = 3, deve ser impresso os 3 menores valores de x_1, x_2, ..., x_n. O algoritmo deve resolver o problema em O(n), independente do tamanho de k.
11. Considere um *array* A, contendo n números distintos. Esse *array* tem a seguinte propriedade: existe um índice m de modo que os números do *subarray* A[1..m] estão em ordem crescente e os números do *subarray* A[m+1..n] estão em ordem decrescente. Escreva (em pseudocódigo) um algoritmo O(log n) que retorna o índice m. Por exemplo, se o *array* de entrada é A[1..9] = [3; 7; 8; 9; 6; 5; 4; 2; 1], a saída deve ser m = 4. Argumente que o seu algoritmo está correto.
12. Implementar em linguagem C o algoritmo para produto de números grandes de forma tal que receba os número de tamanho variável e retorne o produto correto. A implementação deve tornar o algoritmo eficiente para a solução computacional.

Exercício 2

1. Determinar uma parentização ótima para o produto de matrizes cuja sequência de dimensões é [5,10,3,12,5,50,6]

2. Elaborar um algoritmo que imprima a parentização ótima de uma cadeia de matrizes usando a matriz s fornecida pelo algoritmo Parentizar_ Cadeia_Matriz(p:vetor[0..n]).
3. Mostrar que uma parentização completa de uma expressão com n elementos tem exatamente n-1 pares de parênteses.
4. Seja T(n) um número de Catalão, propor um algoritmo eficaz para avaliar

$$T(n)=\frac{1}{n}\left(\frac{2n-2}{n-1}\right)$$

5. Aplicar o algoritmo de caminho mínimo apresentado ao seguinte grafo G representado pela sua matriz de distâncias.

*	V_1	V_2	V_3	V_4
V_1	0	4	7	9
V_2	5	0	5	-
V_3	6	-1	0	6
V_4	7	4	6	0

6. Você dispõe de um container com capacidade para W toneladas e dispõe de diversas mercadorias a serem transportadas. Cada mercadoria é caracterizada por seu peso e valor. Aplicar a técnica de programação dinâmica para conceber um algoritmo para alocação ótima de mercadorias de forma a maximizar o valor transportado. É dado como entrada uma matriz de mercadorias e a capacidade do container. Implementar o algoritmo e resolver o problema abaixo.

W = 15 ton

Item	**Peso(ton)**	**Valor Unit $**	**Quant**
1	3	6,00	2

Item	Peso(ton)	Valor Unit $	Quant
2	4	10,00	3
3	4	12,00	4
4	5	11,00	2

7. Uma cadeia de 3 lanchonetes comprou 8 latões de leite, cada um ao preço de $ 20,00. Cada latão pode ser vendido ao preço de $ 40,00. O fornecedor se compromete a recomprar cada latão não vendido ao fim do dia, pagando $ 10,00 por latão. Constatou-se historicamente a probabilidade de demanda em cada uma das lanchonetes, formando-se a tabela abaixo.

   Qual deve ser a alocação de latões de forma a maximizar o lucro esperado? Sugestão: Construa uma tabela com o lucro esperado para cada quantidade de latões e para cada lanchonete.

	Probabilidade de Demanda		
LOTES	**Lanchonete 1**	**Lanchonete 2**	**Lanchonete 3**
1	0,4	0,5	0,2
2	0,3	0,3	0
3	0	0,2	0,4
4	0,3	0	0,4

8. O Secretário de Segurança do Estado do Piauí constatou que o número de crimes em uma determinada localidade depende do número de patrulhas alocadas, segundo a tabela abaixo:

Número de patrulhas	**0**	**1**	**2**	**3**	**4**	**5**
Redondeza 1	14	10	8	6	5	1
Redondeza 2	25	19	10	6	3	2
Redondeza 3	20	12	8	6	3	0

O secretário conta com 8 viaturas. Use a programação dinâmica para determinar a alocação de patrulhas que minimize o total de crimes.

9. Um investidor tem 6 unidades monetárias (digamos 6 mil reais para serem investidos em lotes de 1 mil reais) para investir em um negócio de risco. O capital ganho numa etapa também pode ser investido. Um determinado investimento oferece as seguintes possibilidades.

Dobrar o valor investido – 30%

Manter mesmo valor – 40%

Perder o valor investido – 30%

1. Qual o maior ganho possível para os próximos quatro anos?
2. Qual a probabilidade de conseguir este ganho?
3. Qual a estratégia adotada se quiser maximizar a probabilidade acumulada de conservarmos no mínimo 25 unidades (25 mil reais) ao fim de quatro anos?

10. Seja B1,..., Bn um conjunto de n caixas todas com um mesmo peso p e tais que a caixa Bi tem um custo Ci.

a) Escreva um algoritmo que dado um valor K, determina um subconjunto de caixas cujo peso é menor ou igual a K e tal que o custo total é máximo.

b) Determine a complexidade de pior caso do seu algoritmo, considerando o número de vezes que cada caixa é analisada.

c) Mostre que o seu algoritmo está correto, isto é, que nenhum outro subconjunto de caixas com peso total menor ou igual a K possui um custo total maior do que o custo total do subconjunto obtido pelo seu algoritmo.

Exercício 3

1. O método guloso sempre fornece uma solução ótima? Justifique a sua resposta através de um exemplo.
2. Sobre métodos gulosos:

a) Apresente sua definição.

b) Quais as suas vantagens?

c) Quais as suas desvantagens?

3. Para o problema do troco, suponha que as moedas disponíveis são c^0, c^1,..., c^k, para inteiros c>1 e k>1. Mostrar que o algoritmo guloso sempre leva a uma solução ótima.
4. Suponha que temos um conjunto de m atividades a serem alocadas a um grande número de locais. Deseja-se alocar todas as atividades usando o menor número possível de locais. Cada atividade possui um tempo

inicial e um tempo final. Projetar um algoritmo guloso para resolver este problema.

5. Considerando o problema de selos de postagem, argumente que o seu algoritmo está correto e efetue a análise de complexidade de sua solução.

a) Mostre que qualquer postagem de valor inteiro maior do que 7 centavos pode ser formada, utilizando apenas selos de 3 e 5 centavos.

b) Escreva um algoritmo que dado um valor da postagem retorna o número de selos de 3 e 5 centavos, respectivamente.

c) Mostre que o algoritmo está correto.

d) Indique a complexidade do algoritmo.

6. Considere o seguinte problema de coleta de cupons. Existe uma certa quantidade de diferentes tipos de caixa de biscoito. Em cada caixa de biscoito encontramos um cupom que dá um desconto na compra de uma outra caixa de biscoito (não necessariamente do mesmo tipo). É possível utilizar múltiplos cupons para comprar uma nova de caixa de biscoito até o valor de obtê-la grátis. Não é possível receber dinheiro de volta, mesmo se sobra cupom. É necessário comprar uma caixa de cada tipo de biscoito, gastando o menor valor possível. Descreva um algoritmo eficiente, que recebe como entrada, para cada tipo de biscoito, seu preço, o valor do cupom e a marca contemplada no cupom e retorna a ordem ótima de compra das caixas de biscoito.

7. Dados seis DVD+Rs, cada um com capacidade de armazenamento de 10GB, necessitamos armazenar seis arquivos de tamanhos 5GB, 6GB, 3GB, 7GB, 5GB e 4GB nestes DVD+Rs. O problema é encontrar o número mínimo de DVD+Rs necessários para armazenar todos estes arquivos. Considere que os arquivos não podem ser particionados.

a) É possível obter uma solução exata para este problema? Caso positivo, descreva a sua solução.

b) É possível obter uma solução exata para este problema utilizando um algoritmo de eficiência polinomial? Explique a sua resposta.

c) Utilizando a estratégia força bruta, projete um algoritmo para determinar o número mínimo de DVD+Rs necessários para armazenar todos estes arquivos. Apresente o pseudocódigo ou a descrição dos passos, utilizando linguagem natural.

d) Qual o menor número de DVD+Rs encontrado pelo seu algoritmo para armazenar os seis arquivos?

e) É possível utilizar a estratégia dividir-e-conquistar para resolver este problema? Justifique sua resposta.

8. O professor João está planejando uma viagem entre as cidades A e B. Ele dispõe da distância total entre as duas cidades, a localização de todos os postos de abastecimento ao longo do trajeto e a autonomia do veículo. O professor deseja fazer o menor número possível de paradas. Descrever uma estratégia eficiente para determinar em quais postos parar e provar que sua estratégia conduz a uma solução ótima.
9. Modificar o algoritmo de Dijkstra implementando a lista de prioridade com o uso de “heap”. Determinar a complexidade desse algoritmo. É melhor do que o algoritmo apresentado?
10. Modificar o algoritmo de Dijkstra implementando a lista de prioridade com o uso de “lista encadeada ordenada”. Determinar a complexidade desse algoritmo. É melhor do que o algoritmo apresentado?
11. Mostrar como resolver o problema da mochila em tempo O(n), onde n é o número de itens disponíveis.
12. O problema conhecido como “task-scheduling” para um processador tem como entrada as durações das tarefas t1, t2,...,tn. Programar a ordem das tarefas tal que o tempo médio gasto por cada uma delas no sistema seja mínimo. Propor um algoritmo guloso que resolva o problema e provar sua otimização usando matróides. Qual a complexidade do algoritmo?
13. Seja o problema para achar um subconjunto T de arestas de um grafo conexo G, tal que todos os nós permaneçam conexos quando somente as arestas de T são usadas e a soma das arestas é a menor possível. Imagine a possibilidade de haver arestas com comprimentos negativos. A solução pode não ser uma árvore.
14. Considere um arquivo texto onde encontramos apenas os seguintes caracteres: a(25), b(12), c(4), d(4), e(7), i(30), z(18). Entre parênteses está indicada a frequência de ocorrência de cada caractere. Aplicar a estratégia de Huffman para a compressão de arquivos. Qual seria o fator de compressão neste caso?

UNIDADE 4

Classes de Problemas

Resumindo

Com esta unidade finalizaremos o nosso estudo de Projeto de Algoritmos e sua análise classificando os problemas de acordo com a sua natureza e complexidade. Visamos apenas oferecer ao leitor uma visão geral dos problemas, enfatizando os de ordem polinominal e os problemas não determinístico polinominalmente (NP).

4 CLASSES DE PROBLEMAS

INTRODUÇÃO

> "Alguns problemas computacionais são difíceis. Tentamos e tentamos
> encontrar algoritmos eficientes para resolvê-los, mas falhamos
> repetidamente. Seria bom se pudéssemos provar que é impossível
> encontrar um algoritmo eficiente nestes casos. Tal prova seria útil,
> pois poderíamos nos tranquilizar sabendo que um algoritmo
> desse tipo não pode ser achado."
>
> — Goodrich, Michael T., Tamassia, Roberto

Há alguns problemas computacionais que são difíceis de serem resolvidos ou que é impossível se provar que não existe solução eficiente.

Vimos a conveniência de se utilizar medidas de complexidade como medida de eficiência. Aprendemos que um algoritmo é eficiente quando a sua complexidade for polinomial em relação ao tamanho de sua entrada. Um algoritmo é dito ser de tempo polinomial se for O(nk), para alguma constante k > 0. Qualquer outro algoritmo que não for polinomial é exponencial.

Contudo, essa classificação não é absoluta. Algumas vezes pode ser apenas satisfatória, mas, na maioria dos casos, é aceitável.

Solucionabilidade de Problemas

Problemas Resolvíveis e Não Resolvíveis

De uma maneira geral, os problemas podem ser divididos em Resolvíveis e Não Resolvíveis:

Resolvíveis - Há pelo menos um algoritmo que o descreve e o resolve.

Não Resolvíveis - Não há um algoritmo para resolvê-los. São também chamados de Problemas Halting, ou seja, problemas onde: Sejam dados um programa qualquer e um conjunto de entradas válidas e genéricas. O programa irá terminar?

Problemas Tratáveis e Intratáveis

Problemas Tratáveis

São todos os tipos de problemas da natureza que podem ser resolvidos em tempo hábil, independentemente do tamanho da entrada. Nestes casos, compensa investir em melhorias nas velocidades de processamento do *hardware* ou até mesmo em reprojetar processos, como por exemplo, uma implementação do algoritmo ótimo por meio de processamento paralelo ou distribuído, para que seja possível a obtenção de respostas em tempo mais breve, uma vez que o tempo de resposta dos algoritmos ótimos dos problemas tratáveis é previsivelmente de natureza igual ou abaixo à complexidade polinomial. Entre os exemplos, podemos destacar:

1. Problemas de Ordenação;
2. Soma, multiplicação, produto, divisão, MDC, MMC e exponenciação em aritmética ilimitada;
3. Problemas de Atribuição;
4. Problema de Filas de Prioridade;
5. Problemas de seleção em estruturas ordenadas;

Problemas Resolvíveis, mas Intratáveis

São todos os tipos de problemas da natureza que podem ser resolvidos, porém não em tempo hábil. A solucionabilidade desses problemas necessita de um grande poder de computação ainda não desenvolvido, o que torna inviável a sua tratabilidade. Como exemplos, temos o Problema do Caixeiro Viajante, Problema do Ciclo Hamiltoniano, o Problema de encontrar subconjuntos mutuamente disjuntos de um conjunto, dadas condições de formação (Restrições), o Problema de encontrar o melhor preenchimento de

uma mochila, sabendo que ela tem um limite de peso para carregar itens de valor, dentre muitos itens existentes de valor conhecido, etc.

Problemas Intratáveis e Indecidíveis

Esta classe de problemas é extrema, em termos de complexidade, uma vez que estes não têm nenhuma garantia sobre sua programabilidade, ou até mesmo, se um algoritmo existe para tratá-lo de alguma forma, tal algoritmo não há prova dele ser ótimo a menos que P=NP, o que será visto mais adiante.

Sendo assim, nem sempre é possível afirmar que a solução ótima desejada é possível de ser obtida, uma vez que não existe método generalista que a encontre para toda instância. Como exemplos, pode-se citar: o problema do Número Cromático; o problema de Steiner Euclidiano; o problema de Empacotamento de Vértices (Vertex Packing), etc. Tais problemas aproximam-se àqueles do tipo Halting, porém, não são Halting.

Formas de Problemas

Para melhor abordar a questão da complexidade de problemas e o enquadramento dos mesmos nas classes P e NP, é conveniente classificá-los nas seguintes categorias:

1. Problemas de decisão;
2. Problemas de localização;
3. Problemas de otimização.

Problemas de Decisão

Os problemas de decisão são problemas cuja solução corresponde a responder sim ou não ao que foi formulado. Por exemplo, dado um grafo G(V,E), orientado e ponderado nas arestas, dois vértices u, v ∈ V e um inteiro não negativo k. Existe um caminho em G entre u e v com comprimento menor ou igual a k? Então, dada a instância I=<G,u,v,k>, a resposta será “sim” se houver tal caminho; ou “não”, caso contrário.

Problemas de Localização

Os problemas de localização são aqueles nos quais o objetivo é encontrar, caso exista, determinada estrutura satisfazendo requisitos especificados por uma questão. No caso do problema de caminho, dado o grafo G(V,E) ponderado nas arestas, os vértices u, v e um número inteiro k. O problema formulado consiste em localizar, caso exista, um caminho com comprimento menor ou igual a k. A resposta seria exibir uma sequência de vértices u, ...,v.

Problemas de Otimização

Os problemas de otimização são os que têm por objetivo encontrar uma determinada estrutura satisfazendo critério ótimo predefinido. Para problema do caminho, um critério ótimo poderia ser o caminho de menor comprimento possível.

Apesar da classificação, tais classes apresentam um íntimo relacionamento entre si. O problema de decisão parece ser bem mais fácil que os de localização e otimização, o que não é totalmente verdadeiro. Podemos afirmar que o problema de decisão apresenta dificuldade não maior que a do problema de localização, e este, por sua vez, apresentam dificuldade não maior que a do problema de otimização correspondente. Há casos em que os Problemas de Localização e Otimização são tão difíceis quanto o Problema de Decisão respectivo.

Sendo o problema de decisão normalmente mais fácil e pelo fato do tamanho da saída do mesmo ser constante, sim ou não, torna-o padrão para análise de intratabilidade de problemas.

Problemas de Decisão da Classe P

Estudaremos agora a classe de problemas que possuem solução em tempo polinomial e são também chamados de tratáveis. Cumpre ressaltar que um problema que requer tempo $\Theta(n^{100})$ é intratável, bem como qualquer problema que exige tempo polinomial com grau muito elevado.

Podemos afirmar que um problema é tratável, caso ele possa ser solucionado em um tempo oportuno para o fim a que se destina.

Outro aspecto a considerar é que muitos problemas que podem ser

resolvidos em tempo polinomial em um modelo computacional pode ser resolvido em tempo polinomial em outro modelo.

Definição - Classe P

P é a classe de problemas de decisão que podem ser resolvidos por um algoritmo de tempo polinomial.

A classe de problemas polinomiais tem propriedades de fechamento em relação à adição, multiplicação e composição. Assim, um algoritmo polinomial que gera entradas para outro algoritmo polinomial forma um algoritmo composto também polinomial.

O problema de caminho apresentado no parágrafo anterior é um caso típico de problema da classe P.

Observe que dado um problema para o qual não se conhece algoritmo polinomial não significa, necessariamente, que tal problema não pertença à classe P. Para tal afirmação, deve ser exibida uma prova formal de que todo algoritmo possível para resolver tal problema não é polinomial.

Apresentaremos, a seguir, dois problemas para os quais não se conhece algoritmo exato e tempo polinomial:

Desafio:
Suponha que um oráculo deu a você um computador mágico C que recebe qualquer fórmula booleana B em FNC (forma normal conjuntiva) e responde uma etapa se B pode ser satisfeita ou não. Mostre como usar C para construir uma atribuição de valores booleanos que satisfaçam qualquer fórmula booleana B que possa ser satisfeita. Quantas chamadas você precisa fazer a C para fazer isso, no pior caso?

Problema de Satisfabilidade (*Satisfiability Problem - SAT*)

Dada uma expressão lógica E(x1,x2,...,xn) na forma normal conjuntiva, isto é, constituída de uma conjunção de cláusulas, cada uma delas constituída de uma disjunção de literais.

Exemplo:

$$E(x_1, x_2, x_3) = (x_1 \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (\overline{x_3})$$

onde x_1, x_2, x_3 são variáveis lógicas denominadas literais e as expressões entre parênteses são as cláusulas.

O problema da Satisfabilidade consiste em determinar se a expressão é verdadeira para uma data atribuição das variáveis lógicas. Dado x_1=V, x_2=F, x_3=F, a resposta exige o teste de cada uma das cláusulas o que conduz à resposta F. No problema dado, a expressão será verdadeira quando $x_1 = x_2 = x_3$ =F.

Problema do Ciclo Hamiltoniano

Dado um grafo G=(V,E), onde V é um conjunto de vértices e E é um conjunto de pares (u,v) tal que u,v ∈ V, denominados arestas. O grafo G é dito não orientado, pois (u,v) = (v,u) denotam uma só aresta. O conjunto V tem n vértices e o conjunto E tem m arestas.

Um ciclo hamiltoniano consiste em um caminho fechado $v_1,v_2,...,v_n,v_1$ e o problema de decisão correspondente pode ser formalizado da seguinte forma:

Entrada: Grafo G

Questão: G possui um Ciclo Hamiltoniano?

Até o momento, não se conhece um algoritmo polinomial que resolva este problema.

Classe NP

Algoritmos de Verificação

Consideramos um problema bem mais fácil. Dados um grafo G hamiltoniano e uma sequência de vértices C, provar que C forma um ciclo hamiltoniano. Todos concordam que é um problema bem mais fácil e para resolvê-lo basta percorrer cada um dos vértices, na ordem, verificando a existência de arestas. Este algoritmo de verificação pode ser implementado em $O(n^2)$, onde n é o número de vértices.

Um algoritmo de verificação é um algoritmo A com 2 argumentos onde um argumento é uma cadeia x de entrada e o outro argumento é uma cadeia binária y denominado certificado. O algoritmo A verifica a entrada x para obter um certificado y, tal que A(x,y)=1. Intuitivamente, um algoritmo A verifica uma linguagem L em busca de um certificado y para a entrada x ∈ L. Entretanto, para alguma entrada x ∈ L e não deve haver certificado provando que x ∈ L. No caso do Problema Ciclo Hamiltoniano(PCH), o certificado é uma lista de vértices do ciclo hamiltoniano.

Definição da Classe NP

A classe de problemas NP é a classe de linguagens que podem ser verificadas por um algoritmo de tempo polinomial.. Uma linguagem L pertence

a NP se e somente se existe um algoritmo A que verifique um certificado em tempo polinomial.

Então, podemos concluir que o Problema do Ciclo Hamiltoniano pertence a NP.

Relação entre P e NP

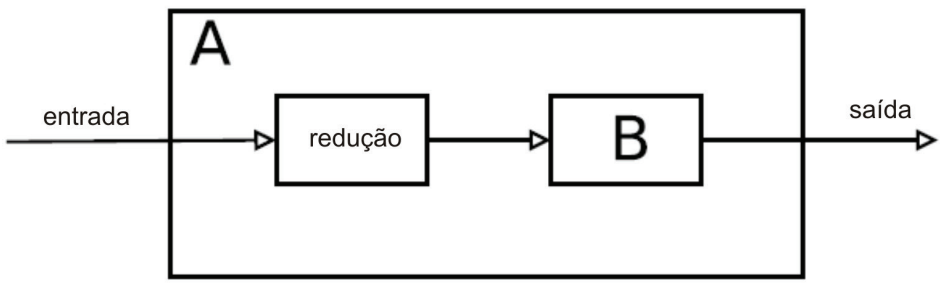
O que falar de P em relação a NP?

Pela definição da classe NP, podemos afirmar que se $L \in P$, então, $L \in NP$. A justificativa é imediata, pois se L pode ser resolvido em tempo polinomial, a verificação também será em tempo polinomial.

A classe Co-NP

Redução Linear

Determinar a complexidade exata de um dado problema não é uma tarefa fácil na prática. Uma solução consiste em comparar a dificuldade relativa de diferentes problemas. Dizemos que um problema reduz a outro se podemos transformar eficientemente instâncias do primeiro problema em instâncias do segundo problema, tal que, resolvendo o segundo problema, estaremos resolvendo o primeiro.



Sejam os problemas de multiplicação e quadrado (um número elevado ao quadrado) de grandes números inteiros. Já estudamos uma forma de resolver o problema de multiplicação de inteiros grandes com um algoritmo $O(n^{1,59})$, que é bem melhor em relação ao algoritmo tradicional quadrático. Podemos fazer tal cálculo com menor complexidade? Achar um algoritmo melhor é função do desenvolvedor de algoritmos; provar que um algoritmo

é ótimo é do escopo da complexidade. No entanto, não se conhece limites inferiores para a multiplicação e para o quadrado.

Multiplicar dois números requer Ω(n), pois uma entrada com n dígitos exige que cada digito seja verificado. Procura-se um algoritmo ainda não conhecido que processe a multiplicação em tempo O(n).

Considerando agora o problema quadrado, ele parece ser mais simples do que a multiplicação.

Como resolver o problema quadrado usando multiplicação?

$x^2 = x.x$

Desta forma, estamos reduzindo o problema quadrado ao problema multiplicação.

Como resolver o problema multiplicação usando quadrado?

$x.y = ((x+y)^2-(x-y)^2)/4$

Desta forma, estamos reduzindo o problema multiplicação ao problema quadrado.

Quando um problema X é redutível a um problema Y e este é redutível ao problema X, dizemos que X e Y são computacionalmente equivalentes, ou seja, a dificuldade para resolver X e Y é de mesma ordem. Para formalizar a transformação, basta exibir os algoritmos:

```
função quadrado(x)
retornar multiplicar(x,x)
função multiplicar(x,y)
retornar ((quadrado(x+y) - quadrado(x-y))/4)
```

Definição: Sejam A e B dois problemas. Diz-se que A é linearmente redutível a B, denotado por $A \leq_l B$, se a existência de um algoritmo para B que trabalha em um tempo O(t(n)) para uma função arbitrária t(n) implica que existe um algoritmo para A que também trabalha em um tempo O(t(n)).

Quando $A \leq_l B$ e $B \leq_l A$, diz-se que A e B são linearmente equivalentes e se escreve $A \equiv_l B$.

A Classe NP-Completo

Há um grande número de problemas do mundo real para os quais não se conhece algoritmos computacionais eficientes para resolvê-los e cuja dificuldade intrínseca ainda não foi provada. No entanto, há diversos problemas, tais como Caixeiro Viajante, Número Cromático em grafos, Problema da Mochila, Ciclo Hamiltoniano, SAT, Programação Inteira para os

quais há provas formais da dificuldade intrínseca em resolvê-los. Todos os problemas desta classe são problemas reconhecidamente intratáveis, mas se for desenvolvido algum algoritmo polinomial para qualquer um dos problemas desta classe, todos os outros problemas da classe estarão resolvidos em tempo polinomial, conforme veremos ao longo desta Unidade, não são poucos os problemas reconhecidamente intratáveis.

Já vimos o questionamento quanto à inclusão das classes P e NP. Intuitivamente, descobrir uma prova é mais difícil do que verificá-la. Esta intuição pode conduzir à conjectura de que P ≠ NP o que até hoje não foi possível verificar. Por outro lado, há diversos problemas da classe NP que com certeza não estão em P e que são muito difíceis de resolver. Tais problemas foram agrupados na classe NP-Completo, definida a seguir.

Definição: Um problema de decisão X é NP-completo se,

i) $X \in NP$, e

ii) $Y \leq^{p}_{t} X$, para todo problema $Y \in NP$.

Pela definição, um problema Y para pertencer à classe NP-completo, primeiro deve pertencer à classe NP e em segundo, todos os problemas da classe NP devem ser redutíveis a ele.

Usando a definição acima, caso tenhamos um problema X NP-completo e um problema Z da classe NP, se $X \leq^{p}_{t} Z$, o que pode-se afirmar ? Pode-se afirmar que Z é tão difícil quanto X ou pior.

Será que Z pertence a NP-completo ?

Teorema: Seja X um problema NP-Completo. Considerar um problema de decisão Z pertencente a NP tal que $X \leq^{p}_{t} Z$. Então, Z também pertence a NP-completo.

Prova:

Para z ser NP-Completo deve satisfazer as duas condições da definição imediatamente anterior. Primeiro $Z \in NP$, o que é uma hipótese do teorema; a segunda condição considera um Y arbitrário em NP. Como X é NP-Completo e $Y \in NP$, segue que $Y \leq^{p}_{t} X$. Pela hipótese $X \leq^{p}_{t} Z$.

Agora, dado um novo problema Z, como provar que ele pertence a NP-Completo? Aplicando a definição de NP-Completo, primeiro deve-se provar que Z pertence a NP, conforme a definição anterior. A seguir, deve-se provar que todos os problemas da classe NP são redutíveis a Z. Como provar isto?

Este teorema é um mecanismo de grande valia para provar que um novo problema Z pertence à classe NP-Completo. Basta provar que Z pertence a NP e escolher um problema X conveniente do conjunto de problemas NP-Completo conhecido e mostrar que X é Turing polinomialmente redutível a Z.

Saiba Mais
O primeiro problema NP-Completo foi o problema SAT pela prova apresentada por Stephen Cook em 1971.

Os seis problemas NP-completo básicos são:

1. 3-SATISFATIBILIDADE (3-SAT);
2. 3-Matching Dimensional (3-DM);
3. Cobertura de Vértices (VC);
4. Clique;
5. Circuito Hamiltoniano(HC);
6. Partição

As reduções obtidas:

1. SAT -> 3-SAT
2. 3-SAT -> 3-DM e 3-SAT -> VC
3. 3-DM -> Partição
4. VC -> HC e VC -> Clique

Algumas Reduções

Teorema: 3-SAT é NP-completo

Prova:

É fácil de ver que 3-SAT pertence a NP, pois dada uma expressão com cláusulas de 3 literais cada, a verificação de uma atribuição é feita em tempo polinomial.

Para transformar SAT em 3-SAT, seja $U=\{u_1,u_2,...,u_n\}$ um conjunto de variáveis e $C=\{c_1,c_2...,cm\}$ um conjunto de cláusulas construindo uma instância arbitrária de SAT.

Construiremos um conjunto $C^{'}$ de cláusulas com 3 literais sobre um conjunto U' de variáveis, tal que $C^{'}$ satisfaz se, e somente se, C conseguir satisfazer.

Conjuntos Independentes

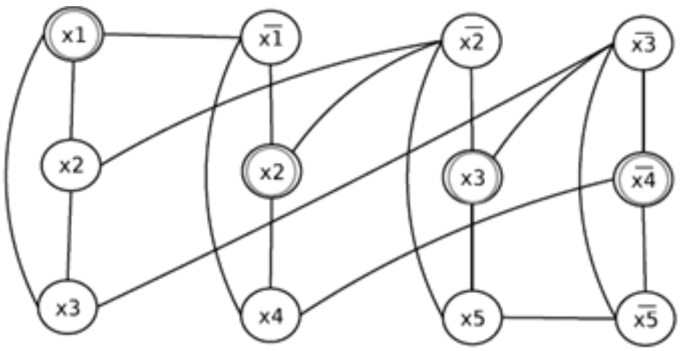
Cada literal de cada cláusula representa um vértice. Ligam-se os vértices referentes à mesma cláusula e ligam-se os vértices que representam

literais complementares.

Considerando uma expressão E com m cláusulas, E é satisfazível se e somente se G tem um conjunto independente de tamanho m.

Exemplo de Redução:

$$(x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4) \wedge (\overline{x_2} \vee x_3 \vee x_5) \wedge (\overline{x_3} \vee \overline{x_4} \vee \overline{x_5})$$



A Classe NP-Difícil

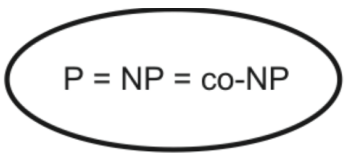
Definição: Um problema de decisão X é NP-Difícil se

i) $Y \leq^{p}_{t} X$, para todo problema $Y \in NP$.

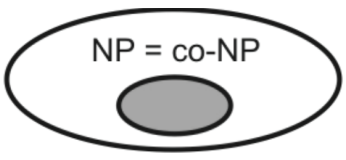
Conclui-se que um problema NP-Difícil é pelo menos tão difícil quanto qualquer problema em NP.

Relações entre Classes de Problemas

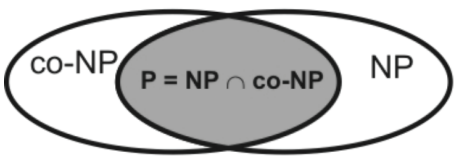
Há basicamente quatro possibilidades de relacionamentos entre as classes de problemas.



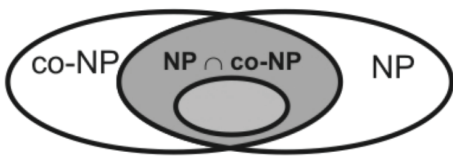
(a)



(b)



(c)



(d)

A maioria dos pesquisadores considera a possibilidade (a) a mais improvável, ou seja, P = NP = co-NP.

Em (b), se NP é fechado para complemento, então NP=co-NP, mas isso é necessário para o caso de não se ter P=NP. A região em laranja são os problemas P. Em (c), P=NP ∩ co-NP, mas NP não é fechado para complemento.

Por outro lado, boa parte dos pesquisadores considera a possibilidade (d) como a mais provável, onde NP≠co-NP e P ≠ NP ∩ co-NP. A região em bege representa os problemas P.

Backtracking* e *branch-and-bound

Backtracking

É uma estratégia para sistematicamente determinar uma lista de possíveis soluções, eliminando (explicitamente) a verificação de uma boa parte dos possíveis candidatos. Pode ser considerado como uma variação de busca em profundidade, pois utiliza uma árvore implícita.

O projeto com *backtracking* é uma forma de construir algoritmos para um problema difícil L. Esse algoritmo procura sistematicamente em um grande (possivelmente exponencial) conjunto de possibilidades. A estratégia de procura é geralmente otimizada para evitar simetrias em instâncias para L e para percorrer o espaço de busca de forma que seja encontrada uma solução fácil para L, se tal solução existir.

A técnica de *backtracking* tira a vantagem da estrutura inerente a muitos problemas NP-completos. Lembre que a aceitação de uma instância x de um problema NP pode ser feita em polinomial, dado um certificado de tamanho polinomial. Frequentemente, esse certificado consiste em uma série de escolhas tais como valores para variáveis booleanas, um subconjunto de vértices de um grafo ou um conjunto de objetos para colocar em uma mochila. De forma similar, a verificação de um certificado, frequentemente, envolve um teste simples para verificar se o certificado fornece uma configuração bem-sucedida para x, tal como satisfazer uma fórmula, cobrir todas as arestas de um grafo ou estar de acordo com algum critério de desempenho. Nesses casos podemos usar um algoritmo de *backtracking* para procurar sistematicamente pela solução do problema, se esta solução existir (GODRICH, TAMASSIA, 2004).

O algoritmo *backtracking* percorre os possíveis caminhos de procura para localizar soluções ou pequenas saídas. A configuração no final de um caminho consiste em um par (x,y), onde x é o subproblema que ainda deve ser resolvido e y é o conjunto de escolhas feitas para chegar a esse subproblema a partir do problema original. Inicialmente, fornecemos ao algoritmo de *backtracking* o par(x,0), onde x é o problema original. Se o algoritmo descobrir em algum momento que uma configuração (x,y) não conduz a uma solução válida, não importando quantas escolhas adicionais sejam feitas, então ele evita todas as procuras a partir dessa configuração e retorna à outra.

1. Algoritmo *Backtrack*(x):
2. Entrada: uma instância x de um problema difícil
3. Saída: uma solução para x ou sem solução se nenhuma existir
4. F←{(x,0)} {F é o conjunto de fronteira de configurações de subproblemas}
5. enquanto F≠ 0 faça
6. retire de F a configuração (x,y) mais promissora
7. expanda (x,y) fazendo um pequeno conjunto de escolhas adicionais
8. sejam $(x_1,y_1),(x_2,y_2),...,(x_k,y_k)$ o conjunto de novas configurações
9. para cada nova configuração (x_i,y_i), faça:
10. verifique a consistência de (x_i,y_i)
11. se a verificação retornar “solução encontrada”, então:
12. retorne a solução derivada de (x_i,y_i)
13. se a verificação retornar “sem saída”, então:
14. descarte a configuração (x_i,y_i) {*Backtrack*}
15. senão
16. F← F $\cup$ {(x_i,y_i)} {(x_i,y_i) inicia uma procura mais promissora}
17. retorne “sem solução”

Resumindo, de uma maneira geral, a ideia para se resolver um problema utilizando *backtracking* é usando espaço de solução, onde as soluções são representadas por n-tuplas ou vetores de solução $<v_1, v_2, ..., v_n>$, em que cada vi é escolhido a partir de um conjunto finito de opções Si.

Inicia-se com um vetor vazio, onde em cada etapa o vetor é estendido com um novo valor. O novo vetor é, então, avaliado, sendo que se não for solução parcial, o último valor do vetor é eliminado e outro candidato é

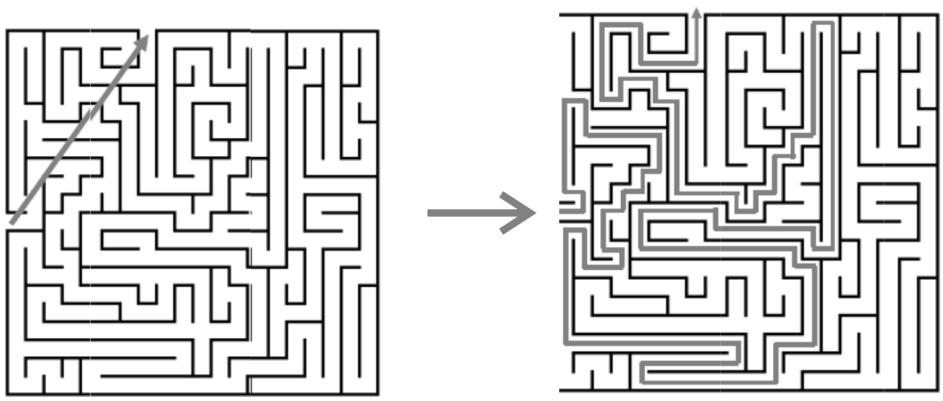
considerado.

As restrições explícitas correspondem às regras que restringem cada vi em tomar valores de um determinado conjunto. Está relacionado com a representação do problema as escolhas possíveis. Já as restrições implícitas determinam como os vi's se relacionam entre si.

Exemplos de Problemas:

Alguns exemplos de problemas onde se pode utilizar a técnica de *backtracking*.

Problema do Labirinto



Jogo da Troca de Bolas

O problema consiste em n bolas amarelas e n bolas azuis, com tabuleiro (uma linha) com 2n + 1 posições, onde bolas com a mesma cor em extremidades diferentes, e um espaço vazio separando o conjunto de bolas diferentes.

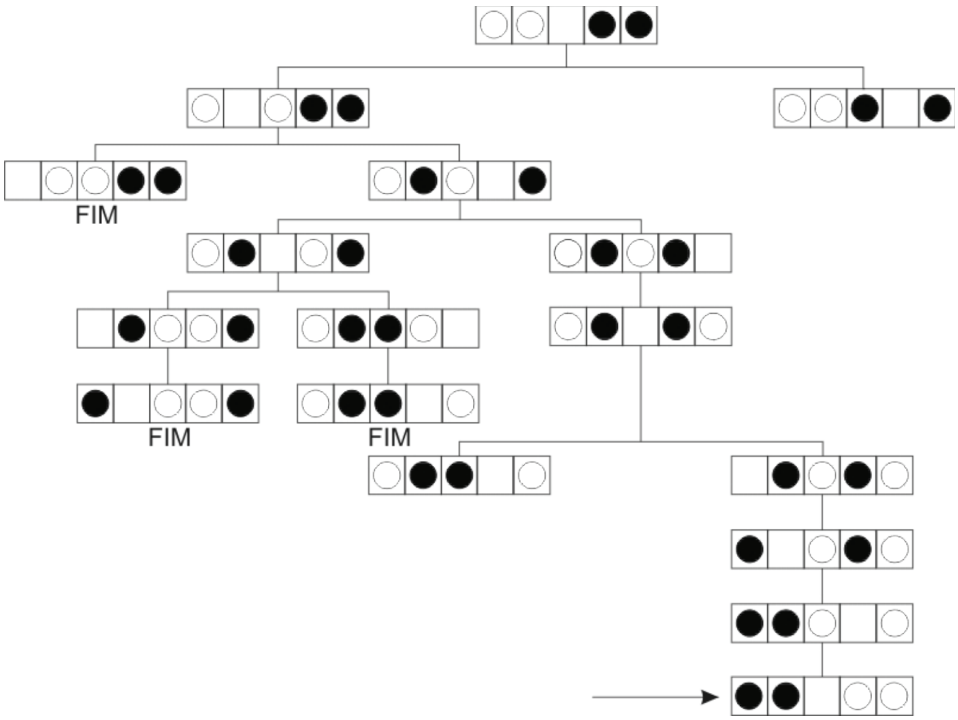
Os possíveis movimentos são:

1. Bola azul para a esquerda e amarela para a direita;
2. Mover um espaço se o espaço está vazio;
3. Pular sobre exatamente uma bola de cor diferente, se o espaço logo após a bola estiver vazio.

Alvo a atingir:



Árvore com possibilidades e alvo:



Observa-se nesses problemas que:

1. Toma-se uma série de decisões entre várias opções;
2. Cada decisão leva a um novo conjunto de decisões;
3. Alguma(s) sequência(s) de decisões pode(m) conduzir a solução do problema;
4. Encontrar uma solução consiste em:

1) Fazer uma lista com todos os candidatos possíveis;
2) Examinar todas as respostas ou alguma delas;
3) Retornar a solução.

Assim, se fossemos aplicar Força bruta, na prática, esta abordagem não seria muito eficiente, visto que a lista de candidatos é grande.

Branch-and-bound

O algoritmo de *backtracking* é eficiente para problemas de decisão, mas não foi planejado para problemas de otimização, em que, além de termos

de satisfazer alguma condição de validade para um certificado y, associado a uma instância x, também desejamos que uma dada função f(x) seja maximizada ou minimizada (sem perda da generalidade, vamos assumir que a função deve ser minimizada). Mesmo assim, podemos estender o algoritmo *backtracking* para trabalhar com um problema de otimização e, ao fazer isso, obteremos o padrão de algoritmos chamada de *brach-and-bound*.

O padrão *brach-and-bound* dado pelo algoritmo seguinte tem todos os elementos de *backtracking*, mas ele não termina simplesmente ao achar a primeira solução. Em vez disso, continuamos até a melhor solução ser encontrada. Além disso, o algoritmo tem um mecanismo de pontuação para escolher sempre a configuração mais promissora a ser explorada em cada iteração. Devido a essa abordagem, *brach-and-bound* é chamado por vezes de estratégia de busca best-first (GODRICH, TAMASSIA, 2004).

1. Algoritmo Branch-and-Bound(x):
2. Entrada: uma instância x de um problema difícil de otimização (minimização)
3. Saída: uma solução ótima para x ou sem solução se nenhuma existir
4. F←{(x,0)} { conjunto de fronteira de configurações}
5. b←{(+∞,0)} {custo e configuração da melhor solução conhecida}
6. enquanto F≠0, faça:
7. retire de F a configuração (x,y) mais promissora
8. expanda (x,y) obtendo um conjunto $(x_1,y_1),\ldots, (x_k,y_k)$ de configurações adicionais
9. para cada nova configuração (x_i,y_i), faça:
10. verifique a consistência de (x_i,y_i)
11. se a verificação retornar “solução encontrada”, então:
12. se o custo c da solução para (x_i,Y_i) for melhor que b, então:
13. b<- (c, (x_i,y_i))
14. senão,
15. descarte a configuração (x_i,y_i)
16. se a verificação retornar “sem saída”, então:
17. descarte a configuração (x_i,y_i) {*Backtrack*}
18. senão,
19. se min b(x_i,y_i) é o menor custo de b, então:
20. F← F $\cup$ {(x_i,y_i)} {(x_i,y_i) inicia um novo caminho promissor}

21. senão,
22. descarte a configuração (x_i,y_i) {o valor é alto demais e já não interessa}
23. retorne b

Para fornecer um critério de otimização e poder escolher a configuração mais promissora, adicionamos mais uma hipótese às três hipóteses usadas para um algoritmo de *backtracking*:

"Para qualquer configuração (x,y), assumimos que temos uma função min b(xi, yi) que retorna um limite inferior para o custo de qualquer solução derivada desta configuração."

A única exigência sobre min b(xi, yi) é que ela seja menor ou igual ao custo de qualquer solução derivada. Como deve ser claro a partir do algoritmo de *branch-and-bound*, quanto mais preciso for esse limite inferior, mais eficiente será o algoritmo.

Exercício 1

1. Julgue as seguintes assertivas com verdadeiro (V) ou falso (F), justifique ou dê contraexemplos:

( ) Existem problemas na classe NP que não estão na classe NP-completo.

( ) A classe NP-completo contém todos os problemas NP-difíceis.

( ) Se houver uma solução polinomial para o problema SAT, então P = NP.

( ) NP-completo é a classe dos problemas computacionais para os quais não existe um algoritmo polinomial.

( ) Assim como os problemas P são problemas polinomiais, os NP são os problemas não polinomiais.

( ) Se um problema A, de custo polinomial, é redutível polinomialmente a outro B, então B também terá custo polinomial.

( ) Se um problema A, NP-difícil, é redutível a outro B, então B também será NP- difícil.

( ) Se um problema NP-difícil é redutível polinomialmente a outro NP, então eles são polinomialmente equivalentes. Neste caso, ambos são NP-completo.

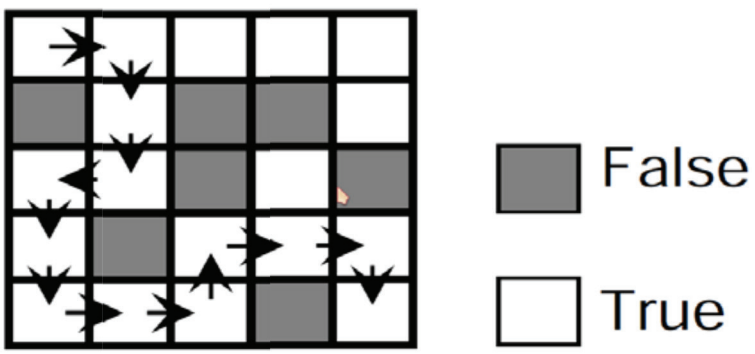
( ) Se dois problemas NP-difícil são polinomialmente equivalentes, então ambos são NP-completo ou ambos não o são.

( ) Se um problema NP-difícil reduz-se polinomialmente a um P, então P =

NP, e todos os problemas NP-completo farão parte do conjunto P também.

( ) As soluções dos problemas NP são implementáveis, havendo várias técnicas para isso, como algoritmo exponencial, probabilístico e de aproximação.

2. Mostrar que se π ∈ NP é um problema, então uma resposta SIM ou NÃO para π pode ser obtida em tempo exponencial com o tamanho de sua entrada.
3. Prove que a seguinte afirmativa é verdadeira: “Se o complemento de um problema NP-Completo está em NP, então NP = Co−NP”
4. Pesquise como o Problema do Ciclo Hamiltoniano pode ser reduzido ao Problema de Decisão do Caixeiro Viajante.
5. Um array booleano M[1..n,1..n] representa um labirinto quadrado. Estando em um determinado ponto do labirinto é possível se movimentar para pontos adjacentes na mesma linha ou mesma coluna. Se M[i,j] é True, é possível passar pelo ponto (i,j); se M[i,j] é False, não é possível passar pelo ponto (i,j). É possível usar uma solução por backtracking que encontra um caminho, se existir, do ponto (1,1) para o ponto (n,n). Abaixo encontramos um exemplo de um labirinto 5x5. Defina restrições implícitπas e explícitas para a sua solução de backtracking e monte a árvore de espaço de estados, indicando a solução.



Referências

AZEREDO, Paulo A. **Métodos de classificação de dados e análise de suas complexidades**. Rio de Janeiro: Campus, 1996.

.

BALAKRISHNAN, J. & RANGANATHAN, K. **A Textbook of Graph Theory.**, Ed. Springer-Verlag,1999.

BERG, M.; KREVELD, M. V.; OVERMARS, M.; SCHWARZKOPF, **O. Computational Geometry, Algorithms and Applications**. 2nd edition, Springer, 2000.

BOAVENTURA, P. O. **Grafos: Teoria, Modelos, Algoritmos**. Blucher: Ed. Edgard, 1996.

BRASSARD, G., BRATLEY, P. **Algorithmics: Theory and Practice. Prentice-Hall.**

CAMPELLO, R.E. & MACULAN, N. **Algoritmos Heurísticos: desenvolvimento e avaliação de perfomance.** Rio de Janeiro: Ed. EDUFF, 1994.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: Teoria e Prática**, Ed. Campus. Rio de Janeiro, 2002.

DIESTEL, R. **Graph Theory.** Ed. Springer, 1997.

FUIRTADO, A. L. , **Teoria dos Grafos: Algoritmos**. Ed. LTC, 1973.

GAREY Michael R. and JOHNSON David S. **Computers and Intractability -**

A Guide to the Theory of NP-Completeness.

GERSTING, Judith L. **Fundamentos matemáticos para a Ciência da computação**. Rio de Janeiro, LTC, 1995.

GOODRICH, M. T. & TAMASSIA, R. , **Projeto de Algoritmos: Fundamentos, análise e exemplos da internet,** Ed. Bookman. Porto Alegre, 2004.

GOULD, R. , **Graph Theory**. The Benjamim/Cummings Publishing Company, 1988.

GRAHAM, Ronald L. Knuth, DONALD, E. Patashnik, Oren. **Matemática concreta: fundamentos para a ciência da computação.** Rio de Janeiro, LTC, 1995.

KLEINBERG, J. Kleinberg, TARDOS, E. **Algorithm Design.** Addison Wesley, 2005.

KNUTH, D. E.; GRAHAM, L. R.; PATASHNIK, **O. Concrete Mathematics - A Foundation for Computer Science**. Addison-Wesley, 1989.

LEVTIN, A. **Introduction to the design and analysis of algorithms**. Addison-Wesley, 2003.

LEWIS, Harry R. and PAPADIMITRIOU, Christos H. **Elementos de Teoria da Computação.**

MANBER, U. **Introduction to Algorithms: a creative approach**. Addison-Wesley, 1988.

MIYAZAWA, F. K. **Notas de Aula de Complexidade de Algoritmos.** Relatório Técnico, Unicamp, 1999.

RESENDE, P. J.; STOLFI, J. **Fundamentos de geometria computacional**. IX Escola de Computação, 1994.

SZWARCFITER, J. L. **Grafos e Algoritmos computacionais.** Rio de Janeiro:

Campus, 1984.

_____. **Estruturas de Dados e seus Algoritmos**. LTC Editora, 1994.

TERADA, R. **Desenvolvimento de Algoritmos e Estrutura de Dados.** São Paulo: Makron Books, 1991.

TOSCANI, L.V. & VELOSO, P.A.S. **Complexidade de Algoritmos**. Porto Alegre: Ed. Sagra Luzzatto, 2001.

WEST, D. **Introduction to Graph Theory**. Prentice Hall, 1996.

WILSON, R. J., **Introduction to Graph Theory**, Ed. AddisonWesley, 1996.

ZIVIANI, N. **Projeto de Algoritmos com implementação em JAVA e C++.** São Paulo: Ed. Thomson, 2007.
_____. **Projeto de Algoritmos com implementação em PASCAL e C.** São Paulo: Ed. Thomson, 2005.

_____. **Projeto de Algoritmos com implementação em JAVA e C++.** São Paulo: Ed. Thomson. 2007.

WEB BIBLIOGRAFIA

Algorithms and Data Structures http://www.csse.monash.edu.au/~lloyd/tildeAlgDS/

Algoritmos Animados em Java - http://gdias.artinova.pt/projecto/pt/menu_applets.php

Algoritmos em grafos - http://www.ime.usp.br/~pf/algoritmos_em_grafos/index.html

Algoritmos para grafos - http://www.ime.usp.br/~pf/algoritmos_para_grafos/

Análise de algoritmos – uma coletânea de textos - http://www.joinville.udesc.

br/portal/professores/gilmario/materiais/Analise_de_algoritmos__.pdf

Associação Brasileira de Educação a Distância – ABED - www.abed.org.br

Dictionary of Algorithms and Data Structures - http://www.itl.nist.gov/div897/sqg/dads/

Graph Theory Tutorials - http://www.utm.edu/departments/math/graph/

Graphviz - http://www.graphviz.org/

Open Problems – Graph Theory and Combinatorics, de Douglas West: - http://www.math.uiuc.edu/~west/openp/

Projeto de Algoritmos - http://www.dcc.ufmg.br/algoritmos/

Secretaria de Educação a Distância do MEC – SEED - www.seed.mec.gov.br

Teoria dos Grafos - http://www.icmc.usp.br/manuals/sce183/grafos.html

Teoria dos Grafos - http://www.inf.ufsc.br/grafos/

Teoria dos Grafos, na Wikipédia: http://pt.wikipedia.org/wiki/Teoria_dos_grafos

Uma Introdução Sucinta à Teoria dos Grafos - http://www.ime.usp.br/~pf/teoriadosgrafos/

Universidade Aberta do Brasil – UAB - www.uab.gov.br

Universidade Aberta do Piauí – UAPI - www.ufpi.br/uapi

Minicurrículo

Francisco José de Araújo

CV. http://lattes.cnpq.br/9112511053307536

Possui graduação em Licenciatura Plena em Matemática pela Universidade Federal do Piauí UFPI (1976) e Mestrado em Sistemas e Computação pela Universidade Federal da Paraíba (Campina Grande, 1980). Foi Professor do Departamento de Informática da Universidade Federal da Paraíba-João Pessoa (1979 a 1989) e do Departamento de Informática e Estatística da Universidade Federal do Piauí UFPI (1989 a 2004). Atualmente, está aposentado pela Universidade Federal, mas continua exercendo atividade docente no curso de Ciência da Computação do Centro de Ensino Unificado de Teresina - CEUT e sendo coordenador do Curso de Sistemas de Informação da Faculdade das Atividades Empresariais de Teresina FAETE.

José Messias Alves da Silva

CV. http://lattes.cnpq.br/0683740460851435

Possui graduação em Licenciatura Plena em Matemática pela Universidade Estadual do Piauí-UESPI (2001), Bacharelado em Ciência da Computação pela Universidade Federal do Piauí UFPI (2003). Especialista em Matemática e Estatística e Especialista em Administração e m Redes Linux pela Universidade Federal de Lavras-MG (2005). Foi Professor substituto do Departamento de Matemática-DM e do Departamento de Informática e Estatística – DIE, da Universidade Federal do Piauí - UFPI. Tem experiência na área de Ciência da Computação com ênfase em Sistemas de Computação. Seus principais interesses de pesquisa e atuação incluem *Software* Livre, Álgebra Computacional, Segurança e Criptografia, Análise de Desempenho, Processamento Paralelo, Sistemas Distribuídos e Computação baseada em *Clusters* e em Grades (*Grid Computing*).





www.uapi.ufpi.br